

ՀՀ ԿՐԹՈՒԹՅԱՆ, ԳԻՏՈՒԹՅԱՆ, ՄՇԱԿՈՒՅԹԻ ԵՎ ՍՊՈՐՏԻ ՆԱԽԱՐԱՐՈՒԹՅՈՒՆ

ՀԱՅԱՍՏԱՆԻ ԱԶԳԱՅԻՆ ՊՈԼԻՏԵԽՆԻԿԱԿԱՆ ՀԱՄԱԼՍԱՐԱՆ (ՀԻՄՆԱԴՐԱՄ)

Տեղեկատվական և հաղորդակցական տեխնոլոգիաների ու էլեկտրոնիկայի ինստիտուտ

Ալգորիթմական լեզուների և ծրագրավորման ամբիոն

ՀԱՇՎԵԲԱՑԱՏՐԱԳԻՐ

Վեր Տեխնոլոգիաներ առարկայի կուրսային աշխատանքի

Թեմա: Մինի էլեկտրոնային խանութ

ակադեմիական խումբ

Ուսանող՝ Հովհաննիսյան Դավիթ Միքայելի (ստորագրություն)

(ազգանուն, անուն)

Ղեկավար՝ Մանուկյան Լ. (ստորագրություն)

(ազգանուն, անուն)

Ամբիոնի վարիչ՝ Ավետիսյան Ս. (ստորագրություն)

(ազգանուն, անուն)

ԵՐԵՎԱՆ – 2025

ՀԱՅԱՍՏԱՆԻ ԱԶԳԱՅԻՆ ՊՈԼԻՏԵԽՆԻԿԱԿԱՆ ՀԱՄԱԼՍԱՐԱՆ (ՀԻՄՆԱԴՐԱՄ)

Տեղեկատվական և հաղորդակցական տեխնոլոգիաների ու էլեկտրոնիկայի ինստիտուտ

Ալգորիթմական լեզուների եվ ծրագրավորման ամբիոն

SS224

ակադեմիական խումբ

ԿՈՒՐՍԱՅԻՆ ԱՇԽԱՏԱՆՔԻ ԱՌԱՋԱԴՐԱՆՔ

Վեր Տեխնոլոգիաներ

(առարկայի անվանումը)

Հովհաննիսյան Դավիթ Միքայելի

(ուսանողի ազգանուն, անուն, հայրանուն)

1. Աշխատանքի թեման՝ Մինի էլեկտրոնային խանութ

2. Աշխատանքի նախնական տվյալները՝

3. Հաշվեքացատրագրի բովանդակությունը

3.1.

3.2.

3.3.

3.4.

4. Գրաֆիկական մասի ծավալը

5. Կատարման ժամանակացույցը

6. Աշխատանքի ղեկավար՝ Մանուկյան Լ.

ստորագրություն

(Ա. Ա. Հ.)

7. Ամբիոնի վարիչ՝ Ավետիսյան Ս. Ս.

ստորագրություն

(Ա. Ա. Հ.)

8. Ուսանող՝ Հովհաննիսյան Դավիթ Միքայելի

(ամսաթիվ, ուսանողի ստորագրություն)

(Ա. Ա. Հ.)

Բովանդակություն

Բովանդակություն	3
Տեսական.....	6
Գլուխ1 SEO	6
Ներածություն.....	6
1.1. Հիմնաբառերի հետազոտություն(keywords)	8
1.1.1 Ի՞նչ է հիմնաբառերի հետազոտությունը:	8
1.1.2. Ինչու՞ է կարևոր հիմնաբառերի հետազոտությունը:	9
1.1.3. Հիմնաբառերի հետազոտության էական հասկացությունները	10
1.1.4. Ինչպե՞ս անել հիմնաբառերի հետազոտություն:	11
1.1.5. Հիմնաբառի հետազոտության խորհուրդներ	18
1.1.6. Արագ հիմնաբառերի հետազոտություն	21
1.1.7 Հիմնաբառերի հետազոտության գործիքներ.....	22
1.2. Ի՞նչ է SEO-ն.....	33
1.2.1. SEO-ի տեսակները՝ ըստ կիրառման մեթոդների.....	34
1.2.2 SEO-ի տեսակներ	42
Գլուխ 2 Front-End	56
2.1. HTML (HyperText Markup Language)	58
2.1.1. Ի՞նչ է HTML-ը.....	58
2.1.2. HTML փաստաթղթի կառուցվածքը	60
2.1.3. Օգտագործված HTML թեգերի նկարագրությունը	63
2.1.4. Semantic HTML-ի կիրառությունը	65
2.1.5. HTML և SEO-ի կապը	66

2. 2. CSS (Cascading Style Sheets)	68
2.2.1. Ի՞նչ է CSS-ը և ինչու է այն կարևոր.....	68
2.2.2. CSS սինտաքսը և կառուցվածքը.....	69
2.2.3. CSS-ի միացման եղանակները	70
2.2.4. Ընտրիչներ (Selectors) և դրանց տեսակները	71
2.2.5. Տեքստի ձևավորում CSS-ով.....	72
2.2.6. Տեքստի ձևավորում CSS-ով (գույն, չափ, տառատեսակ և այլն).....	73
2.2.7. Էլեմենտների չափսեր և տեղակայում (width, height, margin, padding)	73
2.2.8. Box Model-ի բացատրությունը.....	74
2.2.9. Գույների օգտագործում՝ HEX, RGB, HSL	74
2.2.9. Դասավորության մոդելներ (Display, Position)	75
2.2.10. Flexbox-ի կիրառումը	75
2.2.11. CSS Grid Layout-ի կիրառումը.....	76
2.2.12. CSS փոփոխականներ (Custom Properties / Variables)	77
2.2.13. Մաքուր և վերօգտագործելի CSS գրելու խորհուրդներ	78
2.3. JS (JavaScript).....	79
2.3.1. JavaScript-ը ինչ է և ինչու է կարևոր	79
2.3.2. DOM-ը (Document Object Model)	82
2.3.3. JS իրադարձություններ (Events) և դրանց կառավարում.....	86
2.3.4. JavaScript Հիմնական Թեմաներ	89
Գլուխ 3 Back-End	93
3.1. Օգտագործողների կառավարում PHP-ով (User Authentication & Authorization)	94
3.2. CRUD գործողություններ PHP-ում (Տեսական հատված).....	96
3.3. REST API Մշակում PHP-ով.....	99
Գլուխ 4 Database (MySQL)	103

Օրինակ SQL հարցումներ MySQL-ում	104
Գործնական	106
Գլուխ 1 SEO	106
1.1. Կայքի Հիմնաքառեր	106
1.2. Մեթա տեգերի օպտիմալացում (Meta Tags Optimization).....	106
1.3. Մրցակիցների SEO վերլուծություն.....	110
1.3.1 Մրցակից 1 SEO վերլուծություն – AZAT MARD	110
1.4.Robots.txt	116
1.5. Sitemap.xml	118
1.6. Internal Linking	121
Գլուխ 2 Front-End–	124
Գլուխ 3 Back-End	135
3.1.Login.php.....	135
3.2.Logout.php	137
3.3. Register.php	137
Գլուխ 4 Database (MySql)	139
Եզրակացություն.....	150
Գրականություն	152

Տեսական

Գլուխ1 SEO

Ներածություն

Այսօր, երբ առցանց առևտուրը արագ զարգանում է, մինի խանութի կայքերը դարձել են հագուստի նման նեղ շուկաների համար հզոր գործիք: Գիտե՞ք, որ 73%-ը առցանց գնորդների նախընտրում են կայքեր, որոնք հեշտ է օգտագործել և օպտիմիզացված են որոնողական համակարգերի համար: SEO (Որոնողական Համակարգերի Օպտիմիզացիա) կիրառելը կարևոր է այնպիսի կայքերի համար, որպեսզի դրանք տեսանելի լինեն հաճախորդների համար: Մինի խանութի կայքերը, ի տարբերություն մեծ էլեկտրոնային առևտրի հարթակների, ունեն յուրահատուկ առավելություններ, ինչպիսիք են արագությունը, պարզությունը և թիրախային լսարանին հասնելու ունակությունը:

SEO-ն (Search Engine Optimization) որոնողական համակարգերում կայքի օպտիմալացումն է՝ որոնման արդյունքում առաջին հորիզոնականներում հայտնվելու նպատակով: Այն որոշակի գործողությունների համախումբ է, որոնք հնարավորություն են տալիս բացահայտել կայքի առաջխաղացման վրա ազդող տեխնիկական, կառուցվածքային ու բովանդակային խնդիրները և միաժամանակ մշակել դրանց շտկմանն ուղղված ռազմավարություն: Որքան բարձր է կայքի դիրքը որոնման արդյունքներում, այնքան ավելի շատ օգտվողներ են տեսնում և այցելում կայք: Իսկ կայքում ունենալով օգտվողի կողմից որոնվող համապատասխան ապրանքը կամ ծառայությունը, դուք ունենում եք այն վաճառելու մեծ հավանականություն: Հետևաբար, օնլայն շուկայում վաճառքի ցանկալի արդյունքների հասնելու համար առաջ է գալիս կայքի առաջխաղացման անհրաժեշտություն:

SEO-ի աշխատանքը հիմնված է որոնողական համակարգերի կողմից սահմանված կանոնների վրա: Յուրաքանչյուր որոնողական համակարգ օգտագործում և պարբերաբար թարմացնում է իր վարկանիշային ալգորիթմները, որոնք կախված են բազմաթիվ գործոններից: Չնայած այդ բանաձևերը գաղտնի են պահվում, SEO մասնագետները գիտեն, թե որ գործոններն են առավելագույնս ազդում կայքի առաջխաղացման վրա: Առաջնորդվելով այդ կանոններով և կիրառելով համապատասխան քայլերի հաջորդականություն, հնարավոր է բարելավել կայքի դիրքը որոնման արդյունքներում:

SEO-ի կարևորությունը նաև կայանում է նրանում, որ այն օգնում է բրենդի ճանաչելիությունը բարձրացնել: Ավելի մեծ վստահություն են առաջացնում այն կազմակերպությունները, որոնք որոշակի բանալի բառերով զբաղեցնում են թոփ հորիզոնականներ: Նրանք միշտ տեսանելի են որոնողներին և միշտ հիշեցնում են իրենց մասին, ինչի արդյունքում տպավորվում է և՛ բրենդային անվանումը, և՛ գործունեության ոլորտը: Բրենդի ճանաչելիությունը դրական ազդեցություն է ունենում նաև հետագա վաճառքների վրա, քանի որ բարձր հեղինակությունը հանգեցնում է վստահության բարձրացման և նոր հաճախորդների ներգրավման:

Այս աշխատանքը ուսումնասիրում է մինի խանութի կայքի օպտիմիզացման հիմնական ռազմավարությունները, մասնավորապես՝ SEO տեխնիկաները, օգտագործողի փորձը (UX) և բջջային օպտիմիզացիան: SEO-ի ճիշտ կիրառումը կարող է բարձրացնել կայքի տեսանելիությունը որոնողական համակարգերում, գրավել ավելի շատ հաճախորդներ և մեծացնել վաճառքները: Անկախ նրանից՝ դուք ստեղծում եք մինի խանութի կայք գրոյից կամ օպտիմիզացնում եք արդեն գոյություն ունեցողը, SEO-ի հիմունքները հասկանալը կարևոր է հաջողության հասնելու համար: Հաջորդ բաժիններում մենք կանդրադառնանք հիմնաբառերի հետազոտությանը, on-page SEO տեխնիկաներին և դիզայնի լավագույն պրակտիկաներին, ինչպես նաև կուսումնասիրենք, թե ինչպես կարելի է ստեղծել օգտագործողի համար հարմար և գրավիչ կայք:

1.1. Հիմնաբառերի հետազոտություն(keywords)

Հիմնաբառերի հետազոտությունը SEO-ի գրելու գործընթացի առաջին քայլն է և ձեր SEO ռազմավարության կարևոր մասն է: [SEO](#). Նախքան ձեր վեբ կայքի բովանդակությունը ստեղծելը, դուք պետք է իմանաք, թե ինչ որոնման տերմիններ է օգտագործում ձեր լսարանը: Դրանց որոնման պայմաններն են *vos* հիմնաբառեր.

Այս հիմնաբառերի հիման վրա դուք կարող եք սկսել գրել բարձրորակ, օգտակար բովանդակություն: Այստեղ մենք կանցկացնենք հիմնաբառերի հետազոտության մեջ ներգրավված բազմաթիվ քայլերի միջով:

1.1.1 Ի՞նչ է հիմնաբառերի հետազոտությունը:

Հիմնաբառերի հետազոտությունը մաս է կազմում [SEO](#) -ի(որոնողական համակարգի օպտիմալացում): Դա այն աշխատանքն է, որը ինչ-որ մեկը անում է հիմնաբառերի երկար ցուցակը կազմելու համար, որոնց համար կցանկանայիք, որ վեբ կայքը դասակարգվի: Նման ցուցակ ստանալու համար կայքերի սեփականատերերը պետք է հետազոտություն կատարեն իրենց լսարանի և որոնման համակարգերի հետ.

- *Որոնման ի՞նչ տերմիններ են մարդիկ մուտքագրում Browser(Google, Yandex, Safari...)-ում, երբ փնտրում են իրենց արտադրանքը, ծառայությունը, բիզնեսը կամ բիզնեսի տեսակը:*
- *Եվ ի՞նչ են նրանք սպասում գտնել:*

Այս ցուցակի միջոցով կայքի սեփականատերերը կարող են ստեղծել բովանդակություն, որը կգրավի ավելի բարձրորակ երթևեկություն: Հիմնաբառերի հետազոտությունը երբեք չի ավարտվել. Պարբերաբար կրկնել այն՝ տեղեկացված լինելու համար:

1.1.2. Ինչու՞ է կարևոր հիմնաբառերի հետազոտությունը:

Պատշաճ հիմնաբառերի ուսումնասիրությունը կարևոր է, քանի որ այն կնվազեցնի ձեր լսարանի կողմից օգտագործվող որոնման բառերը: Հաճախ ենք հանդիպում բիզնեսի տերերի, ովքեր օգտագործում են բառերի մեկ հավաքածու՝ իրենց արտադրանքը նկարագրելու համար, մինչդեռ նրանց նպատակային լսարանը օգտագործում է բոլորովին այլ բառերի շարք: Արդյունքում, պոտենցիալ հաճախորդները չկարողացան գտնել այս կայքերը՝ բառերի օգտագործման հետաձգման պատճառով:

Օրինակ, երբեմն շուկայավարման բաժինը որոշում է արտադրանքին տալ ոչ այնքան տարածված անուն: Մա կարող է լինել խելացի շուկայավարման որոշում. մարդիկ կարող են ավելի հեշտությամբ հիշել ձեր ապրանքը:

Եթե ձեր տեքստը օպտիմիզացնում եք այս տերմինների համար, ապա, ամենայն հավանականությամբ, լավ դասակարգված կլինեք այդ հատուկ պայմանների վրա: Այնուամենայնիվ, դուք չեք անում այս պայմաններով, և դուք կկարոտեք ձեր հավանական լսարանի մեծ մասը, քանի որ ձեր հաճախորդը այլ բառեր է օգտագործում:

Դուք կհասկանաք, որ անիմաստ է օպտիմալացնել այն բառերով, որոնք մարդիկ չեն օգտագործում: Հիմնաբառերի մանրակրկիտ ուսումնասիրությունը օգնում է համոզվել, որ օգտագործում եք նույն բառերը, ինչ ձեր նպատակային լսարանը, ինչը ձեր կայքի օպտիմալացման ջանքերը շատ ավելի արժանի է դարձնում: Բացի այդ, նայելով [հետազոտական մտադրություն](#), դուք կիմանաք, թե ինչ է փնտրում ձեր հանդիսատեսը: Այս հարցումները պետք է պատասխանվեն որակյալ բովանդակության տեսքով:

1.1.3 Հիմնաբառերի հետազոտության էական հասկացությունները

Հիմնաբառերի հետազոտություն գործիքներ

«Հիմնաբառ» (Keyword) կամ «հիմնական արտահայտություն» այն բառը կամ արտահայտությունն է, որը ցանկանում էք, որ ձեր կայքի որոշակի էջ գտնվի Google- ում: Դուք որոշելու եք արտահայտությունների մի ամբողջություն՝ կատարելով հիմնաբառերի հետազոտություն:

Եթե ցանկանում եք հեշտությամբ օպտիմալացնել հաղորդագրությունը կամ էջը ձեր ստեղծագործության մեկի համար, ապա հարկավոր է այդ արտահայտությունը մուտքագրել գրառման կամ էջի ֆոկուս ստեղծագործության դաշտում: Այն ձեզ կտրամադրի հետադարձ կապ այն մասին, թե ինչպես օպտիմալացնել ձեր բովանդակությունն այդ արտահայտության համար՝ վարկանիշի հնարավորությունները բարելավելու համար:

Երկար հիմնաբառերն ավելի առանձնահատուկ են և ավելի քիչ են որոնվում, քան, այսպես կոչված, գլխի հիմնաբառերը: Նրանք կենտրոնանում են մի խմբի վրա: Որքան երկար և ավելի կոնկրետ լինեն որոնման պայմանները, այնքան հեշտ կլինի դասակարգել դրանք, քանի որ ավելի քիչ մրցակցություն կլինի: Նույնիսկ եթե ավելի քիչ մարդիկ կան, ովքեր փնտրում են այս տերմինները, նրանք կարող են ավելի շատ դրդապատճառներ գնել, բաժանորդագրվել, բաժանորդագրվել կամ որևէ այլ բան, որը դուք կցանկանայիք անել:

Ձեր հիմնաբառերի ռազմավարություն այն որոշումների մասին է, որոնք դուք կայացնում եք ձեր հիմնաբառերի հետազոտության հիման վրա:

Օրինակ, ի՞նչ բովանդակություն եք նախ ստեղծելու: Դուք կենտրոնանալու եք գլխի՞ վրա, թե՞ պոչի վրա: Ինչպե՞ս և որտե՞ղ եք այն հրապարակելու: Պատրաստվու՞մ եք ստեղծել գրվածք, հոդված կամ արտադրանքի էջ, տեսանյութերի ձեռնարկ կամ ինֆոգրաֆիկա:

Դուք պետք է պարզեք, թե իրականում ինչ է ուզում կամ կարիք ունի հետազոտողը: Դուք ոչ միայն նայում եք հիմնաբառերին, այլ հիմքում ընկած նպատակներին, թե ինչ է ուզում

որոնողը իմանալ, անել կամ գնել: Ձեր բովանդակությունը պետք է լուծում տա որոնողի «խնդրին»: Այն կոչվում է նաև բովանդակության ձևավորում.

1.1.4 Ինչպե՞ս անել հիմնաբառերի հետազոտություն:

Կարծում ենք, որ հիմնաբառերի հետազոտություն կատարելիս կա ութ կարևոր քայլ: Այստեղ մենք անցնելու ենք փուլ առ փուլ, թե ինչպես հեշտությամբ սկսել մեր սեփական հիմնաբառերի ուսումնասիրությունը:

Որոշեք ձեր առաքելությունը

Նախքան ինչ-որ բան սկսելը, **մտածիր քո առաքելության մասին**, Մտածեք այնպիսի հարցերի մասին, ինչպիսիք են.

- *Ո՞րն է ձեր կամ ձեր բիզնեսի հիմնական նպատակը:*
- *Ինչո՞վ է դա առանձնահատուկ:*
- *Դուք կոնկրետ ում եք փորձում հասնել:*
- *Եվ, ի՞նչ խոստումներ եք տալիս ձեր կայքում:*

Մի շտապեք և բառացիորեն գրեք ձեր առաքելությունը: Երբ կարողանաք մանրամասնորեն պատասխանել այս հարցերին, դուք կկատարեք առաջին և ամենակարևորը քայլը ձեր հիմնաբառերի ռազմավարության մեջ: Շուկան, որում գտնվում եք, որոշում է՝ արդյո՞ք ձեր առաքելությունը այնքան մեծ կլինի, որ բարձր դասի: Որոշ շուկաներ շատ մրցունակ են, խոշոր ընկերությունները գերակշռում են որոնման արդյունքներում:

Այս ընկերություններն ունեն հսկայական բյուջեներ ընդհանուր առմամբ շուկայավարման և, մասնավորապես, SEO- ի համար: Այս շուկաներում մրցակցությունը բարդ է, ուստի այս շուկաներում դասակարգումը նույնպես դժվար կլինի:

Գուցե դուք վաճառում եք Հավայան կոուիզներ: Դուք առաջարկում եք հիանալի հարմարություններ երեխաների համար, ինչը նավարկություններին հատկապես հարմար

Է դարձնում երիտասարդ ծնողների կամ միայնակ ծնողների համար: Երիտասարդ ծնողների համար Հավայան կղզիների լավագույն նավարկությունների առաջարկը կարող է լինել այն, ինչը եզակի է դարձնում ձեր ծառայությունը:

Այնպես որ, փնտրեք այն իրը, որը ձեր արտադրանքը առանձնացնում է մրցակցությունից: Սա պետք է լինի ձեր առաքելությունը, ձեր տեղը և դա այն է, ինչ դուք պետք է առաջարկեք ձեր հաճախորդին:

Եթե դուք մտնում եք մրցակցային շուկա, ձեր լավագույն խաղաղրույքը մեկնարկելն է: Երբ սկսեք «վաստակել» այդ խորշի մի փոքր մասը և ավելի մեծ անուն ձեռք բերել “Հավայան կռուիզային” բիզնեսում, կարող եք փորձել հարթեցնել և վաճառել ձեր նավարկությունները ավելի մեծ (ավելի ընդհանուր) լսարանին: Ձեր առաքելությունն այնուհետև կդառնա ավելի ընդհանուր:

Կազմեք հիմնաբառերի ցուցակ

Երկրորդ քայլը ձեր հիմնաբառերի ցուցակի ստեղծումն է, գերադասելի է աղյուսակում, ինչպիսիք են Google Sheets- ը կամ Excel- ը: Ձեր առաքելությունը նկատի ունենալով՝ փորձեք “մտնել” ձեր ցանկալի լսարանի միտքը:

- *Ի՞նչ են փնտրելու այս մարդիկ:*
- *Ի՞նչ տեսակի որոնման պայմաններ կարող են նրանք օգտագործել ձեր գարմանալի ծառայությունը կամ ապրանքը որոնելիս:*
- *Ձեր արտադրանքից ո՞ր մեկն է լուծում ձեր արտադրանքը:*

Գրեք որքան հնարավոր է շատ պատասխաններ: Եթե ձեր առաքելությունը պարզ է, ապա դուք կունենաք բավականին հստակ պատկերացում ձեր տեղը և յուրօրինակ վաճառքի կետերի մասին (այն բաները, որոնք ձեր բիզնեսը առանձնացնում են մյուսներից): Սրանք այն պայմաններն են, որոնց համար ցանկանում եք գտնել:

Որոնեք ձեր հիմնաբառերը

Երբ ստեղծեք այդ առաջին ցուցակը, ժամանակն է մի փոքր խորանալ ձեր հիմնաբառերի մեջ: Բարեբախտաբար, կան գործիքներ, որոնք ձեր հիմնաբառերի հետազոտությունը մի փոքր հեշտացնում են:

Առաջինը հենց Google-ն է: Երբ դուք մուտքագրում եք այն հիմնաբառերը, որոնք եկել եք, Google-ն առաջարկում է դրանք ձեզ, երբ դա անում եք: Սրանք այն հարցերն են, որոնք մարդիկ իրականում տվել են Google-ում: Կարող եք նաև ստուգել «նման որոնումները» Google-ի արդյունքների էջում: Այս գործիքը ձեզ տրամադրում է ձեր հիմնական արտահայտությունների, հոմանիշների և հարակից հիմնական բառակապակցությունների բոլոր տատանումները:

Շատերը, երբ սկսում են հիմնաբառերի հետազոտություն, հակված են կենտրոնանալ հայտնի և բարձր մրցակցային «գլխավոր» տերմինների վրա: Սակայն, դժբախտաբար, այս հիմնաբառերը հաճախ գրավում են խոշոր ընկերությունները, ինչը դժվարացնում է փոքր և միջին բիզնեսների համար դրանցով բարձր դիրք զբաղեցնելը:

Ի տարբերություն դրանց, **երկար պոչի հիմնաբառերը** (long-tail keywords) ավելի կոնկրետ են և ունեն ավելի քիչ որոնման տրաֆիկ, բայց միևնույն ժամանակ՝ ավելի ցածր մրցակցություն: Սա նշանակում է, որ ավելի հեշտ է դասավորվել այս հիմնաբառերով որոնման արդյունքներում: Բացի այդ, երկար պոչի հիմնաբառերն ունեն **ավելի բարձր փոխակերպման տոկոս**, քանի որ դրանք ավելի կենտրոնացած են օգտվողի կոնկրետ պահանջարկին: Օրինակ, եթե «գլխավոր» տերմինը «լակոտների մարզում» է, ապա երկար պոչի հիմնաբառը կարող է լինել «Ամստերդամում լաբրադորի համար դրական ամրապնդման մարզում»:

Երկար պոչի հիմնաբառերը սովորաբար ավելի երկար են և կոնկրետ, ինչը թույլ է տալիս ավելի ճշգրիտ հասնել թիրախային լսարանին: Օրինակ, եթե ձեր գլխավոր տերմինը «կանացի հագուստ» է, ապա երկար պոչի հիմնաբառը կարող է լինել «գնել կանացի կաշվե ջինսեր Երևանում»: Մի մոռացեք երկար պոչի հիմնաբառեր ավելացնել ձեր աղյուսակին:

Առաջին սյունակում դրեք առաջատար տերմինները և ավելացրեք սյուններ երկար պոչի հիմնաբառերի համար: Սա կօգնի ձեզ ավելի ուշ ստեղծել պատշաճ կայքի կառուցվածք: Որքան երկար է ձեր որոնման տերմինը, այնքան ավելի ցածր էք գնում ձեր կայքի կառուցվածքում:

Վերլուծեք մրցակցությունը

Եթե ստիպված եք գնալ երկար պոչի հիմնաբառերի, ապա պետք է իմանաք նաև, որ դրանք մեծապես կախված են ձեր մրցակցությունից: Եթե ձեր խորշի մրցակցությունը բարձր է, ապա դժվարությամբ եք դասակվելու մրցակցությանը: Եթե քիչ մրցակցություն ունեք, ապա կկարողանաք նույնիսկ դասվել բարձրակարգ պայմանների վրա: Այսպիսով, դուք պետք է կատարեք SEO- ի որոշ չափանիշներ:

- Google- ում որոնեք ձեր որոնումից դուրս եկած հիմնաբառերը: Սկսեք ձեր «ամենակարճ» ժամկետից: Ստուգեք որոնիչի արդյունքների էջը (SERP): Սրանք այն կայքերն են, որոնց հետ դուք կմրցակցեք, երբ ձեր բովանդակությունը օպտիմալացնեք նման հիմնաբառի համար: Ավելի ուշադիր նայեք.
- *Տեսնո՞ւմ եք պրոֆեսիոնալ կայքեր:*
- *Կորպորատիվ կայքեր*
- *Դուք «հավասարվա՞ծ» եք այս ընկերություններին:*
- *Ձեր կայքը համապատասխանո՞ւմ է սրա հետ:*
- *Ձեր գործը չափի՞նք նման է, և այն նույնքան ազդեցություն ունի՞ ձեր տեղը:*

Դժվար է դասակարգել, երբ ուժեղ մրցում ես կայքերի դեմ: Եթե ապրանքանիշերը հայտնի են հեռուստատեսության կամ ռադիոյի գովազդներից, ապա բարձր վարկանիշ ստանալու ձեր շանսերն էլ ավելի են փոքրանում: Բայց չի խանգարի նայել դրանց բովանդակությանը: *Արդյո՞ք դրանց բովանդակությունը լավ գրված է և լավ օպտիմիզացված:* Եթե ձեր մրցակցությունն իսկապես աղքատ բովանդակություն ունի, հնարավոր է, որ նրանցից գերազանցելու հնարավորություն ունենաք:

Նաև դիտեք Google-ի գովազդները: *Կան որևէ?*

Եթե ունեք Google Adwords հաշիվ, կարող եք հաստատել վճարումը մեկ քիքի համար: Որոնման տերմինները, որոնք ունեն բարձր քիքներ, ընդհանուր առմամբ, նույնպես ավելի դժվար են դասակարգվել որոնիչների արդյունքներում: Համոզվեք, որ ձեր աղյուսակում նշումներ կատարել եք ձեր ուսումնասիրած հիմնաբառերի արդյունքների վերաբերյալ:

Ավելի մանրամասն նայեք որոնման նպատակին

Այսօրվա բիզնեսը, մեծ մասամբ, պետք է պատրաստ լինի մարդկանց հարցերին պատասխանելու կամ նրանց «խնդրի» լավագույն լուծումը տալու շուրջ: Ամեն անգամ, երբ ինչ-որ մեկը որոնման հարցում է մուտքագրում որոնման համակարգ, նրանք ինչ-որ բան են փնտրում: Հարցերի յուրաքանչյուր տեսակ կոնկրետ պատասխանի կարիք ունի:

- Փորձեք պարզել, թե որն է ձեր հանդիսատեսի նպատակը, երբ նրանք մուտքագրում են որոշակի հիմնական բառակապակցություն Google- ում. Արդյո՞ք դա տեղեկատվական նպատակ ունի (փորձեք տեղեկություններ գտնել որոշակի թեմայի վերաբերյալ), դիտելու մտադրություն (ցանկություն ունի՞ նա մուտք գործել որոշակի կայք), բիզնեսի նպատակ: (նա ինչ-որ բան փնտրո՞ւմ է գնելուց առաջ) կամ գործարքային մտադրություն (այժմ նա փնտրո՞ւմ է պատրաստ է՞ ինչ-որ բան գնել): Դուք կարող եք ավելին իմանալ որոշակի հարցումների որոնման նպատակի մասին՝ մանրակրկիտ ուսումնասիրելով այն էջերի տեսակը, որոնք արդեն դասավորված են այդ հարցման համար:
- *Հիմնականում տեսնո՞ւմ եք ապրանքի էջերը:*
- *Տեսնո՞ւմ եք բլոգի վերաբերյալ տեղեկատվական շատ գրառումներ:*
- *Տեսանյութեր տեսնո՞ւմ եք:*
- *Թե՞ խառնուրդ եք տեսնում:*

Սրանք բոլորն են, որոնք Google- ը ենթադրում է որպես որոշակի հարցման որոնման նպատակ:

Բացահայտեք, թե դիտավորության ինչ տեսակներն են կիրառվում ձեր բանալիների արտահայտությունների վրա և, նորից, ավելացրեք ձեր արդյունքները ձեր աղյուսակում:

Որոշեք հիմնաբառերի ռազմավարությունը

Ձեր հավաքած տվյալների հիման վրա կարող եք որոշել հիմնաբառերի ռազմավարությունը: Եթե դուք հետևել եք վերոնշյալ քայլերին, ապա պետք է ունենաք աղյուսակ՝ զգալի քանակությամբ հիմնաբառերով և այդ բառապաշարների համար ձեր լսարանի հետազոտության նպատակի վերաբերյալ տեղեկատվությամբ:

Հիմա մտածեք այս հարցի շուրջ. Ինչպե՞ս է իմ կայքը պահվում, համեմատած SERP-ի կայքերի հետ:

Դուք հավասար չափի՞ եք և նույն շուկայավարման բյուջեով եք. Առաջ գնացեք և կենտրոնացեք այդ լավագույն տերմինների վրա:

Եթե ոչ. Նախ փորձեք երկար պոչի հիմնաբառերը: Երկար պոչի հիմնաբառերի մի ամբողջ փնջի վրա կենտրոնանալը կարող է շատ լավ գրավել մեծ երթևեկություն: Երբ հասցնեք դասվել այդ երկար պոչի հիմնաբառերի շարքում, առաջատար տերմինների նպատակը կդառնա մի փոքր հեշտ:

Երբ որոշեք, մտածեք բովանդակության տեսակի մասին.

- *Ո՞րն էր որոնման նպատակը իմ հիմնական արտահայտությունների համար:*
- *Ի՞նչ է փնտրում իմ հանդիսատեսը:*
- *Բայց նաև ի՞նչ բովանդակություն կարող եմ ստեղծել, որը դեռ չկա, և ինչպե՞ս կարող եմ առանձնանալ՝ որակի կամ առաջարկվող լուծումների տեսանկյունից:*

Սա կօգնի ձեզ որոշել, թե ինչ տեսակի բովանդակություն եք ստեղծելու:

Սկսեք ստեղծել վայրէջք էջեր

Տեսականորեն, այս քայլը դուրս է բուն բառերի հետազոտության հնարավորությունից: Այնուամենայնիվ, ստեղծելով զարմանալի վայրէջք էջեր իհարկե, կարևոր է, եթե ցանկանում եք երթևեկություն ստանալ դեպի ձեր վեբ կայքը: Այսպիսով, դուք պետք է ստեղծեք վայրէջք էջեր ձեր որոնման տերմինների համար, բայց դուք չպետք է անմիջապես ստեղծեք այդ բոլոր էջերը. դա կարող է երկար ժամանակ տևել: Ձեր հիմնաբառերի ռազմավարությունը կօգնի ձեզ առաջնահերթություն սահմանել:

Ստեղծեք լավ վայրէջքի էջ և օգտագործեք այն պատճենել դրանք արագ նորերը պատրաստելու համար: Ձեր ամենակարևոր հիմնական արտահայտությունների համար դուք կստեղծեք անկյունաքարային բովանդակության հոդվածներ.

Ստեղծեք հնարավոր լավագույն բովանդակությունը այս հիմնաբառի վրա՝ հեղինակավոր: Ձեր երկար պոչի բոլոր հոդվածները կապված կլինեն այս անկյունաքարային բովանդակության հետ: Դա պետք է լինի ձեր ռազմավարության մի մասը , որը Yoast SEO Premium- ը կարող է օգնել ձեզ իրականացնել:

1.1.5. Հիմնաբառի հետազոտության խորհուրդներ

Գուցե այդ ամենը բավական պարզ է թվում, բայց մենք գիտենք, որ դա շատ աշխատանք է և ավելի հեշտ է ասել, քան անել: Գործնականում կիրառվելիս կարող է բախվել որոշ ընդհանուր հարցերի կամ հարցերի: Այստեղ մենք կտանք մի քանի խորհուրդ՝ այն գործելու համար:

Գերադասեք ձեր հիմնաբառերի ցուցակը

Քանի՞ հիմնաբառ պետք է ունենաս:

Դե, մենք չենք կարող ճշգրիտ ասել հիմնաբառերի ճշգրիտ քանակը, բայց կարող ենք ասել, որ դրանցից շատերը մեզ հարկավոր են, այնքան շատ, որքան կարող եք մտածել: Այնուամենայնիվ, ավելի քան 1000 հիմնաբառեր, հավանաբար, շատ են: Նույնիսկ եթե ողջամտորեն փոքր բիզնես եք, հավանաբար ձեզ պետք կգան մի քանի հարյուր հիմնաբառեր:

Բայց ձեզ հարկավոր չէ միանգամից էջեր ստեղծել այս բոլոր հիմնաբառերի համար: Կարող եք քիչ-քիչ բովանդակություն ավելացնել: Մտածեք այն հիմնաբառերի մասին, որոնց համար ցանկանում եք դասակարգել հենց հիմա. Միգուցե ամենաերկար պոչերը, որոնք այժմ այնքան էլ կարևոր չեն: Հասկացեք ձեր առաջնահերթությունները և պլանավորեք ձեր բովանդակության ստեղծումը:

Ուշադրության կենտրոնացումը և դրա հոմանիշները միայն մեկ էջի կարիք ունեն

Նախկինում յուրաքանչյուր հիմնաբառ, որը ցանկանում էիք գտնել, ուներ իր ուրույն էջը: Սակայն այսօր որոնիչները այնքան խելացի են, որ հիմնականում օգտագործում են որոնման նպատակ՝ որոնողներին իրենց հարցերի լավագույն պատասխանը տալու համար: Էջը, որը լավագույնս կպատասխանի այս հարցերին, դասվելու է վերևում:

Որոնիչները հասկանում են նաև հիմնաբառերի նուրբ տարբերությունները, որպեսզի ստիպված չլինեք վայրէջքի էջեր ստեղծել հիմնաբառի բոլոր նուրբ տատանումների համար, ինչպես հոմանիշները:

Դա չի նշանակում, որ դուք չպետք է օգտագործեք հոմանիշներ: Իրականում, հոմանիշները իսկապես կարող են բարելավել ձեր օրինակի ընթերցանությունը, այնպես որ օգտագործեք դրանք ձեր տեքստում: Yoast SEO Premium հավելվածը կարող է օգնել ձեզ այս հարցում. այն թույլ է տալիս օպտիմալացնել ձեր բովանդակությունը հոմանիշների և հարակից հիմնական բառակապակցությունների միջոցով:

Կապակցված հիմնաբառերի արտահայտությունները բառեր և հասկացություններ են, որոնք խորացնում և ընդլայնում են ձեր հիմնաբառի ըմբռնումը: Դրանք նույնիսկ օգնում են Google- ին ավելի լավ հասկանալ ձեր ասած թեման: Օգտագործելով հոմանիշներ և հարակից հիմնական բառակապակցություններ ձեր տեքստում, կարող եք նկարել ձեր հոգվածում ձեր առանց ֆոկուսային հիմնական արտահայտությունների ամբողջական պատկերը:

Դիտեք եզակի և հոգնակի հիմնաբառերի արդյունքները

Նպատակ ունե՞ք հիմնաբառի եզակի կամ հոգնակի թվին:

Դե, դա կախված է հարցումից: Քանի որ Google- ը կարող է ավելին իմանալ ձեր հարցման որոնման նպատակի մասին, այն ավելի լավ կարող է կռահել, թե ինչ եք փնտրում: Օրինակ, եթե փնտրում եք գիրքը, դուք այլ արդյունք եք ստանում, քան եթե որոնում եք գրքերը.

Ըստ երևույթին, Google-ը կարծում է, որ առաջին դեպքում դուք փնտրում եք սահմանում կամ ինչ-որ պատմություններ, իսկ երկրորդ դեպքում՝ կարծում է, որ մտադիր եք գիրք գնել: Այսպիսով, համոզվեք, որ գիտեք, թե ինչ եք առաջարկում ձեր էջում, և որ այն համապատասխանում է հարցումին և արդյունքներին, որոնք տալիս է Google-ը:

Բառի ձևն աջակցում է, ուստի այն ավտոմատ կերպով հայտնաբերում է ձեր հիմնաբառի բոլոր տարբեր ձևերը (հայտնի է որպես հիմնաբառերի ածանցում): Այսպիսով, դուք այլևս

ստիպված չեք լինի ձեր հաղորդագրությունը օպտիմալացնել որոշակի բառաձևի համար: Ձեր հաղորդագրության օպտիմալացումը կդառնա շատ ավելի բնական գործընթաց:

Այնուամենայնիվ, կան պատճառներ, թե ինչու դուք միշտ կցանկանաք օպտիմալացնել հիմնաբառի շատ հատուկ բառաձևը: Այդ դեպքում ֆոկուսի բանալին կարող եք դնել մեջբերումների մեջ. «Բոլոր ժամանակների լավագույն գրքերը»: Yoast SEO- ն այժմ ձեր բովանդակությունը ստուգելիս կքննարկի միայն այս ճշգրիտ ֆոկուսային բանալին:

Օգտագործեք հիմնական արտահայտությունը միայն մեկ անգամ

Զգուշացեք, չպետք է մեկ անգամից ավելի օգտագործեք ձեր ճշգրիտ ֆոկուսային հիմնաբառը: Եթե դա անեք, ձեր վարկանիշը կարող է տուժել: Google-ը դժվարանում է տարբերակել բովանդակությունը, որը շատ նման է: Հետևաբար, այն կարող է դասակարգել շատ նման գրառումներ կամ ավելի ցածր էջեր:

Գտե՞լ եք, որ նախկինում օգտագործել եք նույն կամ շատ նմանատիպ հիմնաբառեր տարբեր հաղորդագրությունների և էջերի վրա:

Հավանաբար իմաստ ունի ստուգել ձեր բովանդակությունը և միգուցե միաձուլել / հեռացնել / վերահղել դրա մի մասը:

Փորձեք, գնահատեք և փորձեք կրկին

Երբ դուք մանրակրկիտ վերլուծել եք որոշակի ժամկետի համար վարկանիշ ստանալու ձեր հնարավորությունները, հաջորդ քայլը զարմանալի և համապատասխանաբար օպտիմալացնել այն: Եվ հրապարակելու համար, տարածեք այն սոցիալական ցանցերում և ձեր տեղեկագրում:

Համոզվեք, որ կգրավեք որոշ հետադարձ կապեր:

- *Տեսե՞ք ձեր վարկանիշը: Ձեր հոդվածը հայտնվո՞ւմ է:*
- *Դա Google- ի SERP- ների առաջին էջո՞ւմ է:*
- *Թե՞ դա թաքնված է 2-րդ կամ 3-րդ էջում:*

Համոզվեք, որ գնահատում եք Google SERP- ներում ձեր հոդվածների օպտիմիզացված պայմանները: Ստուգեք՝ ձեր ջանքերը արդյունք տալի՞ս են, թե ոչ:

Եթե ի վիճակի չեք դասակարգվել առաջին էջում, փորձեք գրել մեկ այլ գրառում՝ կենտրոնանալով (դեռ) ավելի երկար պոչով հիմնաբառի վրա: Դարձրեք այն մի փոքր ավելի հատուկ: Եվ տեսեք, թե ինչպես է այն ընթանում: Կրկին գնահատեք: Շարունակեք այս գործընթացը մինչև հասնեք SERP- ների այդ առաջին էջին:

1.1.6. Արագ հիմնաբառերի հետազոտություն

Իդեալական աշխարհում դուք կկատարեիք ձեր հիմնաբառերի հետազոտությունը, կկազմեիք գեղեցիկ սեղան և յուրաքանչյուրի համար վայրէջքի էջեր կստեղծեիք: Ձեր կայքի կառուցվածքն անթերի կլինի, և դուք ամեն օր բլոգ կուղարկեք և կստեղծեք, ինչը ձեր կայքը Google- ում ավելի ու ավելի բարձր կդարձնի: Բայց մենք ապրում ենք իրական աշխարհում: Իհարկե, ձեր հիմնաբառերի ուսումնասիրությունը միշտ չէ, որ այդքան ծավալուն կլինի: Եվ որոշ գրառումներ կամ հոդվածներ չեն գրվել որպես զարմանալի ռազմավարության մի մաս, այլ պարզապես այն պատճառով, որ թեման նորություններում է կամ ինչ-որ բան ձեզ ոգեշնչել է այն գրել: Դա այդպես է գործում: Բայց դա պարտադիր չէ, որ խնդիր լինի: Եթե դուք գրում եք մի բան, որը լիովին չի համապատասխանում ձեր ռազմավարությանը, դա չի նշանակում, որ դուք չպետք է փորձեք դասակարգել այդ բովանդակությունը: Դուք դեռ կարող եք օգտագործել այն՝ ձեր հիմնաբառերի ռազմավարության ցանկի պայմաններին առնչվող ինչ-որ բան դասավորելու համար:

[Google Trends](#) արագ ստուգելու համար այն հիմնաբառը, որի համար ցանկանում եք դասակարգել: Գոնե ժամանակ տրամադրեք մտածելու, թե ինչպես ձեր հոդվածը կամ բլոգը համապատասխանեցնել ձեր ռազմավարությանը: Ի վերջո, եթե դուք արժեքավոր բովանդակություն եք գրում, կարող եք նաև այն դասակարգել:

Պատրաստ եք Սկսեք գրել:

Հիմնաբառերի հետազոտությունը պետք է լինի ցանկացած կայուն SEO ռազմավարության սկիզբ: Արդյունքը կլինի հիմնաբառերի երկար ցուցակ, որոնց համար ցանկանում եք

գտնել: Բայց ամենադժվարը մնում է անել՝ գրել այս ամբողջ բովանդակությունը:



1.1.7 Հիմնաբառերի հետազոտության գործիքներ

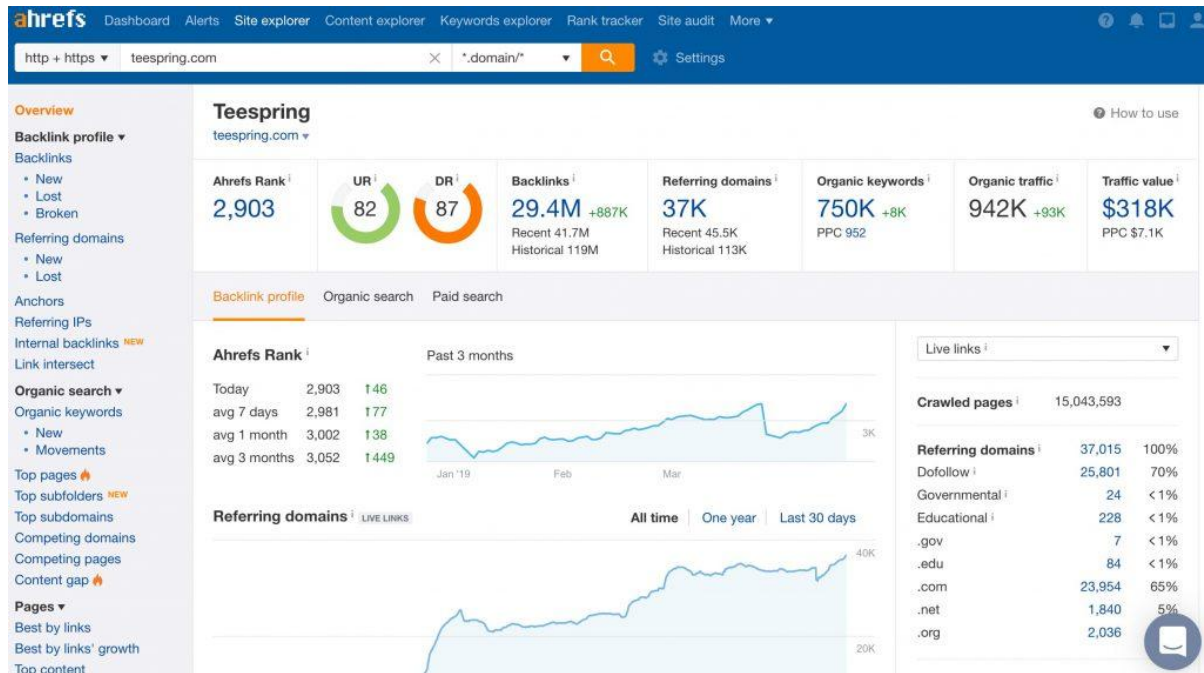
Ahrefs

Ahrefs-ը հիմնաբառերի հետազոտության ամենահզոր գործիքներից մեկն է: Այն օգնում է ձեզ հասկանալ, թե ինչու է ձեր մրցակցությունն այդքան բարձր դասվում և ինչ պետք է անեք որոնման արդյունքներում գերազանցելու համար:

Ahrefs-ն ամեն օր ստղում է ավելի քան 6 միլիարդ էջ, ավելի քան 12 տրիլիոն հղում իր ավելի քան 200 միլիոն տիրույթի անունների ցուցակում: Դա շատ տվյալներ է, բայց իրական գեղեցկությունն այն է, թե ինչպես է նրանց պլատֆորմը օգնում ձեզ օգտագործել այդ տվյալները ձեր օգտին:

Այն ունի ինտուիտիվ ինտերֆեյս, որը բաժանում է բոլոր տվյալները տարբեր բաժինների:

Պարզապես մուտքագրեք, տիրույթի անունը՝ որոնման դաշտում և Ahrefs-ը կբերի մեծ քանակությամբ տեղեկատվություն հեշտ հասկանալի ձևաչափով:



Այն ձեզ ցույց կտա արդյունքների ակնարկ՝ հետադարձ կապերի ընդհանուր թվով, հղում կատարելով տիրույթներին, keyword օրգանական և բովանդակության վերանայում: Դուք կարող եք սեղմել այս բաժիններից որևէ մեկի վրա՝ հետագա գնալու համար:

Կտտացրեք «Օրգանական հիմնաբառեր» զեկույցին, և Ahrefs-ը ձեզ ցույց կտա հիմնաբառերի ցանկը՝ որոնման ծավալով, որոնման վարկանիշով, URL-ով և այլն:

ahrefs Dashboard Site Explorer Keywords Explorer Site Audit Rank Tracker Content Explorer More Academy

http + https ahrefs.com/ Subdomains

Organic keywords How to use

Monthly volume United States Position Volume KD CPC Traffic SERP features Word count

100,878 keywords 6 Mar 2023 Compare with: 6 Feb 2023 Main positions only Multiple URLs only

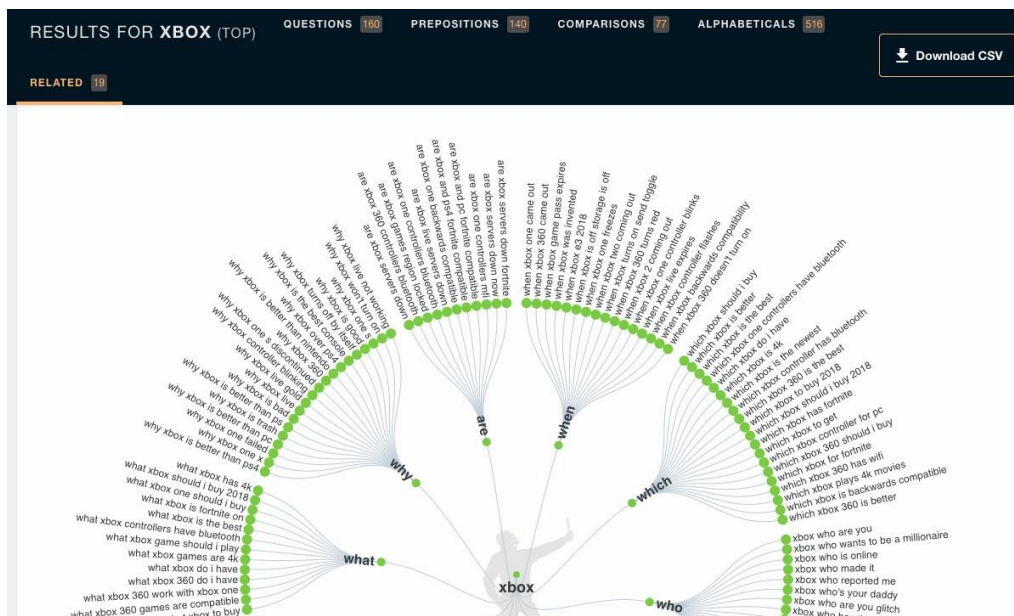
Keyword	SF	Volume	KD	CPC	Traffic	Change	Paid	Position	Change	URL
✓ ahrefs	3	43,000	63	5.32	40,546		0	1		https://ahrefs.com/ 4 more
✓ backlink checker	2	8,000	88	7.38	6,182	-3	0	1		https://ahrefs.com/backlink-checker 4 more
✓ affiliate marketing	4	179,000	92	5.78	4,252	-1,042	0	6 → 7	▼1	https://ahrefs.com/blog/affiliate-marketing/
✓ ahrefs backlink checker	2	3,000	86	N/A	3,559		0	1		https://ahrefs.com/backlink-checker 14 more
✓ affiliate marketing for beginners	3	5,200	72	5.49	2,549		0	1		https://ahrefs.com/blog/affiliate-marketing/ https://ahrefs.com/blog/affiliate-marketing/
✓ ahrefs keyword explorer	3	2,100	83	4.55	2,260	+1	0	1		https://ahrefs.com/keywords-explorer 31 more

Կարող եք նաև առաջարկել հիմնաբառերի գաղափարներ՝ որոնման դաշտում մուտքագրելով հիմնաբառեր: Ahrefs- ի հիմնաբառերի արդյունահանման գործիքը ստեղծում է հիմնաբառերի առաջարկների ցանկ՝ որոնման ծավալով, դժվարության միավորով և կտտոցներով:

Ahrefs- ը նաև գալիս է հզոր գործիքներով՝ բովանդակության վերլուծության, դասակարգի հետևման, վեբ մոնիտորինգի և այլնի համար: Բոլոր հաշվետվությունները կարող եք արտահանել CSV կամ PDF ձևաչափով, այնուհետև աշխատել դրանց վրա ձեր նախընտրած աղյուսակային ծրագրակազմում:



AnswerThePublic- ը անվճար հիմնաբառերի և բովանդակության գաղափարի հետազոտման գործիք է: Այն օգտագործում է Google- ի և Bing- ի ինքնաշխատ առաջարկը և ներկայացնում է տվյալներն ավելի հասկանալի տեսողական ձևաչափով: Պարզապես այցելեք կայք և մուտքագրեք հիմնաբառ կամ արտահայտություն: Դրանից հետո գործիքը բեռնելու է հիմնաբառերը և դրանք ներկայացնելու է տեսողական քարտեզներում: Հաշվետվությունները բաժանված են հարցերի, նախդիրների, համեմատությունների, այբուբենների և հարակից հիմնաբառերի: Կարող եք սեղմել հիմնաբառի վրա՝ Google որոնման արդյունքները զննարկչի նոր ներդիրում ցուցադրելու համար: Մա թույլ է տալիս արագորեն դիտել այն հարցերը, որոնք մարդիկ փնտրում են և ինչպես կարող եք պատասխանել դրանց համապատասխան բովանդակությամբ: Հիմնաբառերի ամբողջ հետազոտությունը ներկայացված է մեկ էջում: Դուք կարող եք ներլցնել տվյալները տեսողական ձևաչափով որպես պատկերներ կամ դրանք արտահանել CSV ֆայլ և օգտագործել դրանք ձեր նախընտրած աղյուսակային ծրագրակազմի հետ:



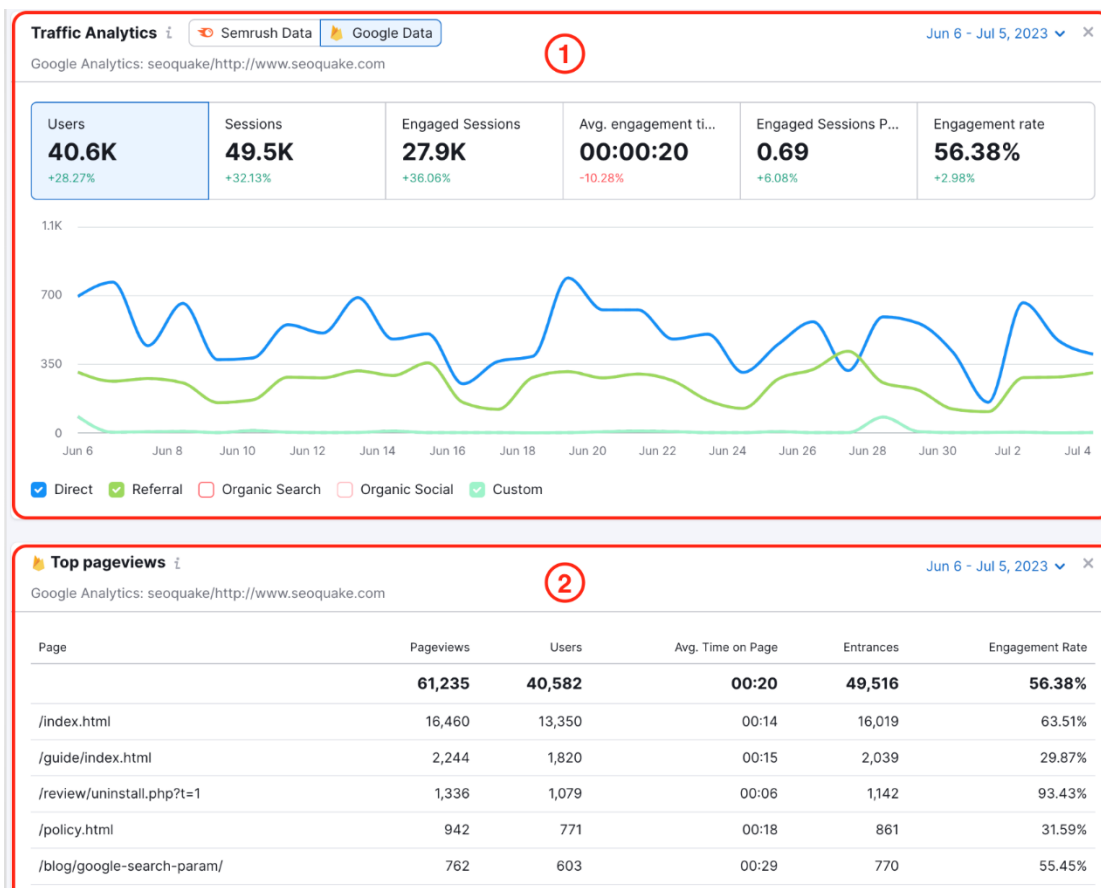


SEMRUSH

SEMRush- ը մեկն է լավագույն է SEO գործիքների շուկայում: Դա SEO- ի ամբողջական հավաքածու է՝ օրգանական որոնում կատարելու, վճարովի գովազդի հետազոտություն, հիմնաբառերի հետազոտություն և մրցակցության խոր վերլուծություն կատարելու գործիքներ:

Սկսելու համար անհրաժեշտ է միայն գնալ SEMRush կայք և մուտքագրել հիմնաբառ կամ ձեր կայքի URL:

Դուք կտեսնեք ձեր մուտքագրված հիմնաբառի ակնարկ, որը ցույց է տալիս որոնման ծավալը (միջին որոնման հաշվարկ), վճարովի գովազդի CPC և որոնման արդյունքների քանակը:



Մի փոքր
շարժվեք
ներքև և
կտեսնեք
համապա
տասխան
դ
հիմնաբառ
երի և
հարակից
հիմնաբառ
երի

ակնարկ: Կարող եք կտտացնել «Դիտել ամբողջական զեկույցը» կոճակը՝ ամբողջական ցուցակները ուսումնասիրելու կամ դրանք CSV ֆայլ արտահանելու համար:

Ոլորեք ներքև և կտեսնեք ձեր մուտքագրած հիմնաբառի որոնման լավագույն արդյունքների ցուցակը: Կարող եք կտտացնել այս հղումներից յուրաքանչյուրին՝ ամբողջական զեկույցը դիտելու համար, ինչը կօգնի ձեզ հասկանալ, թե ինչու են այս էջերը դասվում այդ հիմնաբառերի համար:

Կարող եք նաև օգտագործել դրանց նոր գործիքը՝ **Հիմնաբառ կախարդական գործիք**՝ ինչը հնարավորություն է տալիս ավելի արագ մուտք գործել հարակից տերմինների հիմնաբառերի ավելի լայն հետազոտություն: Կարող եք կտտացնել Ավելացնել կոճակին հիմնաբառի կողքին, որպեսզի դրանք ավելացնեն հիմնաբառերի վերլուծիչին:

Երբ գտնեք որոնման ամենաբարձր ծավալով լավագույն հիմնաբառերը, հաջորդ քայլը կլինի վերլուծել այդ հիմնաբառերի մրցակցությունը: Կտտացրեք այդ հիմնաբառերի համար արդեն դասավորված հղումներին՝ մանրամասն վերլուծություն տեսնելու համար:

Դուք կկարողանաք տեսնել հետադարձ կապերը տվյալ URL- ի համար, այլ հիմնաբառեր, որոնց համար դասվում է այդ էջը, և որքան որոնման երթևեկություն է ստանում:

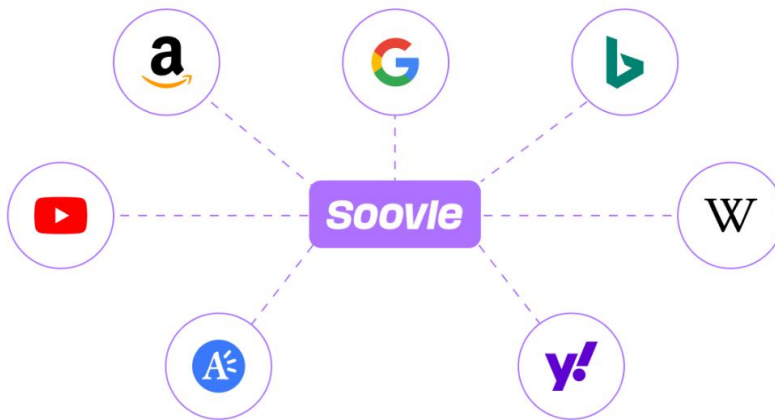
Ընդհանուր առմամբ, SEMRush- ը հիմնաբառերի հետազոտության լավագույն գործիքն է: Այն ոչ միայն ձեզ տալիս է հիմնաբառերի գաղափարներ, այլ նաև օգնում է ձեզ պարզել, թե ինչպես կարող եք դասակարգվել այդ բոլոր հիմնաբառերի համար:

Հիմնաբառեր գտնելու մասին ավելի շատ խորհուրդներ

Վերոնշյալ բոլոր գործիքները ձեզ կտրամադրեն տվյալների գանձ: Ահա ավելի շատ խորհուրդներ, թե ինչպես կատարելագործել հիմնաբառերի հետազոտությունը և գտնել առավել շահավետ գաղափարներ:

- Սկսեք ձեր որոնումը հիմնաբառերի հետ ավելի լայն հիմնաբառերով, ապա նեղացրեք այն՝ ցածր մրցակցություն ունեցող և ավելի շատ որոնման ծավալի հիմնաբառեր գտնելու համար:

- Մուտքագրեք ձեր սեփական URL- ը SEMRush- ում կամ Ahrefs- ում ` ձեր սեփական հիմնաբառի կատարումը տեսնելու համար, այնուհետև այն համեմատելու ձեր մրցակիցների հետ:
- Կենտրոնացեք ձեր հաճախորդների հարցերին պատասխանող հիմնաբառեր գտնելու վրա և կօգնեն նրանց որոշում կայացնել գնումների հարցում:
- Մի սահմանափակեք ձեր հիմնաբառերի հետազոտությունը առավել որոնվող հիմնաբառերով: Կարող եք հեշտությամբ դասակարգվել ավելի երկար հիմնաբառերի համար և կառուցել ` օգտագործելով դրանք:



Soovle-ը որոնման հարթակ է, որը թույլ է տալիս օգտվողներին միաժամանակ որոնել տարբեր կայքերից, ինչպիսիք են Google, Bing, Amazon, eBay, Wikipedia և այլն: Այն հարմար է այն օգտվողների համար, ովքեր ցանկանում են տեղեկատվություն ստանալ մի քանի աղբյուրներից առանց յուրաքանչյուր կայք առանձին այցելելու: Soovle-ն առաջարկում է նաև ավտոմատ լրացման հնարավորություն, ինչը կարող է օգնել արագացնել որոնման գործընթացը

1. Բանալի բառերի հետազոտություն

Soovle-ն թույլ է տալիս տեսնել, թե ինչպես են տարբեր հարթակներում (օրինակ՝ Google, YouTube, Amazon, Wikipedia) օգտվողները որոնում են տեղեկատվություն: Սա կարող է օգնել.

- Բացահայտել հաճախակի որոնվող բանալի բառեր:
- Հասկանալ, թե ինչպես են օգտվողները ձևակերպում իրենց հարցումները տարբեր հարթակներում:
- Գտնել նոր բանալի բառեր, որոնք կարող են օգտագործվել SEO-ի համար:

2. Ավտոմատ լրացման առանձնահատկություն

Soovle-ն օգտագործում է ավտոմատ լրացման (autocomplete) տեխնոլոգիա, որը ցույց է տալիս, թե ինչպես են որոնման համակարգերը առաջարկում որոնման տարբերակներ: Սա կարող է օգտակար լինել.

- Հասկանալու համար, թե որոնք են ամենատարածված որոնումները:
- Վերլուծելու համար, թե ինչպես են որոնման համակարգերը (օրինակ՝ Google) կառուցում իրենց առաջարկությունները:
- Գտնելու երկար պոչ բանալի բառեր (long-tail keywords), որոնք կարող են ավելի քիչ մրցակցային լինել, բայց ավելի ուղղված:

3. Տարբեր հարթակների վերլուծություն

Soovle-ն թույլ է տալիս վերլուծել, թե ինչպես են բանալի բառերը օգտագործվում տարբեր հարթակներում (օրինակ՝ Amazon-ում ապրանքների որոնում, YouTube-ում տեսանյութերի որոնում, Wikipedia-ում հոդվածների որոնում): Սա կարող է օգնել.

- Հարմարեցնել բովանդակությունը տարբեր հարթակների համար:
- Հասկանալ, թե ինչպես են օգտվողները փնտրում տեղեկատվություն կոնկրետ հարթակներում:

4. Բովանդակության ռազմավարության մշակում

Soovle-ի տվյալները կարող են օգտագործվել բովանդակության ռազմավարություն մշակելու համար.

- Ստեղծել բովանդակություն, որը համապատասխանում է օգտվողների որոնման միտումներին:
- Կենտրոնանալ այն բանալի բառերի վրա, որոնք արդյունավետ են տարբեր հարթակներում:

5. Մրցակցային վերլուծություն



Soovle-ն կարող է
օգտագործվել
մրցակիցների
վերլուծության
համար.

- Հասկանալ,
թե ինչ բանալի
բառեր են
օգտագործում
մրցակիցները:

- Վերլուծել, թե ինչպես են մրցակիցները դասակարգվում տարբեր հարթակներում:

Google Keyword Planner

Google Keyword Planner-ը թույլ է տալիս.

- Գտնել նոր բանալի բառեր՝ հիմնված ձեր բիզնեսի, ապրանքի կամ ծառայության վրա:
- Տեսնել բանալի բառերի որոնման ծավալները (ամսական որոնումների քանակը):
- Վերլուծել բանալի բառերի մրցակցայինությունը:
- Ստանալ գնահատումներ Google Ads-ում գովազդային արշավների արժեքի վերաբերյալ:

2. Հիմնական հնարավորություններ

ա) Բանալի բառերի որոնում

- Մուտքագրեք բառեր, արտահայտություններ կամ կայքի URL, և Keyword Planner-ը կառաջարկի հարակից բանալի բառեր:
- Օրինակ, եթե մուտքագրեք "SEO services", գործիքը կառաջարկի այնպիսի բառեր, ինչպիսիք են "best SEO services", "affordable SEO services", "local SEO services" և այլն:

բ) Որոնման ծավալներ

- Keyword Planner-ը ցույց է տալիս, թե որքան հաճախ են որոնվում կոնկրետ բանալի բառերը ամսական:
- Օրինակ, "SEO services" բառը կարող է ունենալ 10,000 որոնում ամսական, իսկ "local SEO services"՝ 5,000:

գ) Մրցակցայինություն

- Գործիքը ցույց է տալիս, թե որքան մրցակցային է բանալի բառը (ցածր, միջին կամ բարձր):
- Սա օգնում է հասկանալ, թե որ բառերն են ավելի հեշտ կամ դժվար օպտիմալացնել:

դ) Գնահատված գներ

- Keyword Planner-ը տրամադրում է գնահատումներ, թե որքան կարող է արժենալ բանալի բառի համար գովազդը Google Ads-ում:
- Օրինակ, "SEO services" բառի համար մեկ կտտոցի արժեքը կարող է լինել \$5-10:



1. 2. Ի՞նչ է SEO-ն

Որոնողական համակարգի օպտիմալացում ([անգլ.](#)՝ *search engine optimization, SEO*), [որոնողական համակարգերում կայքի](#) կամ մի վեբ էջի տեսանելիությունը բարձրացման գործընթաց: SEO-ն վերաբերում է միայն օրգանիկ դիտումներին («բնական» կամ «օրգանաան» արդյունքներ) և չի ներառում վճարված դիտումները (մասնավորապես, չի ներառում «Pay-per-click» [PPC] և ընդհանրապես «Search Engine Advertisement» [SEA] դիտումները): SEO-ի իրականացման նպատակն է բարձրացնել կայքի այցելուների թիվը՝ մեծացնելով որոնողական համակարգերից եկող կայքի այցելուների թիվը («տրաֆիկը»), որն էլ իր հերթին բարձրացնում է կայքի վարկանիշը որոնողական համակարգերում: Այդ այցելուները կարող են նաև վերածվել հաճախորդների^[1]: SEO-ն կարող է թիրախավորել տարբեր տեսակի որոնումներ, այդ թվում, նկարների, վիդեոների, ակադեմիական^[2], նորությունների որոնում: SEO-ն լոկալ որոնողական համակարգերի օպտիմիզացումից տարբերվում է նրանով, որ այն կենտրոնացած է միջազգային որոնումների վրա, իսկ լոկալ որոնողական համակարգերի օպտիմիզացումը կենտրոնանում է բիզնեսի առցանց տեսանելիության օպտիմիզացման վրա, որպեսզի դրա կայքը ցուցադրվի որոնման համակարգերով, երբ այցելուն մուտք գործի տվյալ տարածաշրջանից՝ իր ապրանքի

կամ ծառայության համար: SEO-ի գործառնություններից է համարվում ուսումնասիրել, թե ինչպես են աշխատում որոնողական համակարգերը, համակարգչային ակցորիթմները, որոնք թելադրում են որոնողի վարքագիծը, որոնողական համակարգերում մուտքագրված բանալի բառերը, և որ որոնողական համակարգերն է թիրախային լսարանը նախընտրում: Կայքի օպտիմիզացումը կարող է ներառել կոնտենտի ավելացում և խմբագրում, HTML-ի և ծրագրավորման կոդի փոփոխում՝ կայքի կոնտենտի համապատասխանությունը կոնկրետ բանալի բառերին բարձրացնելու և որոնողական համակարգերում ինդեքսավորման խոչընդոտները վերացնելու համար (կայքի «տեխնիկական օպտիմիզացիա»): SEO-ի կարևոր մարտավարություններից են նաև. կայքի նեքին և արտաքին հղումների թվի աճը (*link-building, internal re-linking, backlink building*)

1.2.1. SEO-ի տեսակները՝ ըստ կիրառման մեթոդների

SEO-ի տեսակներն են ըստ կիրառման մեթոդների՝

- Ներքին SEO
- Արտաքին SEO
- Տեխնիկական SEO
- Սև գլխարկ SEO
- Մոխրագույն գլխարկ SEO
- Բացասական SEO

Ներքին SEO

Էջի SEO-ն ամենատարածված ռազմավարությունն է, որի վրա բիզնեսները կենտրոնանում են իրենց կայքերի համար: Պարզ ասած, էջի SEO-ն վերաբերում է ձեր կայքի փոփոխություններին, որոնք դրականորեն ազդում են դրա կատարման վրա:

Էջի SEO-ի ամենատարածված օպտիմալացումներից մեկը հիմնաբառերի օպտիմալացումն է: Երբ օգտվողներն իրականացնում են որոնումներ, նրանք օգտագործում են հիմնաբառեր՝ համապատասխան որոնման արդյունքների ցուցակ ստեղծելու համար:

Որպեսզի օգնեք արժեքավոր առաջատարներին գտնել ձեր բիզնեսը, դուք պետք է կատարեք հիմնաբառերի հետազոտություն՝ բացահայտելու հիմնաբառերն ու արտահայտությունները, որոնք մարդիկ փնտրում են առցանց՝ ձեր ապրանքներն ու ծառայությունները գտնելու համար: Օրինակ, կեղտաջրերի մաքրման կայանը, որը զբաղվում է SEO-ով, կարող է թիրախավորել վերականգնված մետաղ գնելու հետ կապված հիմնաբառեր:

Այնուհետև դուք կարող եք ներառել այդ հիմնաբառերը ձեր կայքի առանցքային վայրերում, ինչպիսիք են ձեր էջի վերնագրերը, մետա նկարագրությունները և

վերնագրերը: Բացի այդ, դուք կարող եք ստեղծել հատուկ բովանդակություն հիմնաբառերը հասցեագրելու համար:

Թիրախավորելով ձեր բիզնեսի հետ կապված հիմնաբառերը և ստեղծելով արժեքավոր բովանդակություն՝ դուք կարող եք օգնել ձեր բիզնեսին ավելի բարձր երևալ որոնման արդյունքներում, և, ի վերջո, ավելի շատ առաջատարներ և եկամուտներ կվաստակեք:

Էջի SEO-ի մեկ այլ ռազմավարություն URL-ի կառուցվածքների օպտիմալացումն է: Երբ օգտվողները ձեր էջում են, դուք ցանկանում եք օգնել նրանց հիշել ձեր էջը: Հարմարեցված URL կառուցվածքը կօգնի ձեզ օպտիմալացնել ձեր կայքը և հեշտացնել ձեր առաջատարների համար ձեր կայքը հիշելը:

Արտաքին SEO

Ի՞նչ է էջից դուրս SEO-ն: Արտաէջ SEO-ն ներառում է ձեր կայքի օպտիմալացումներ, որոնք տեղի չեն ունենում ձեր կայքում: Մրանք արտաքին գործոններ են, որոնք ազդում են ձեր կայքի աշխատանքի վրա:

Էջից դուրս SEO-ի ամենամեծ գործոններից մեկը սոցիալական լրատվամիջոցների փոխանակումն է: Երբ դուք կիսում եք ձեր կայքի հղումը ձեր սոցիալական մեդիայի

էջերում, դուք չեք կարող վերահսկել, թե ինչպես է ձեր լսարանը փոխազդում ձեր սոցիալական էջերի ձեր հղումների հետ: Սա արտաքին վարկանիշային գործոն է, որը կարող է օգնել ձեր կայքին ավելի լավ աշխատել:

Կարևոր է նշել, որ սոցիալական ազդանշաններն ուղղակիորեն չեն ազդում ձեր SEO-ի վրա: Դրանք ուղղակի ազդեցություն չունեն ձեր վարկանիշի բարելավման վրա, բայց կարող են հանգեցնել վարքագծի, որն օգնում է բարելավել ձեր վարկանիշը:

Երբ դուք կիսում եք ձեր բովանդակության հղումները ձեր սոցիալական պրոֆիլներում, դուք հնարավորություն ունեք ձեր էջի համար հղումներ վաստակելու: Մարդիկ կարող են կիսվել ձեր հղումով իրենց սեփական պրոֆիլներում և օգնել ձեր էջին երթևեկել: Համապատասխան տրաֆիկ դեպի ձեր էջ վարելը կարող է օգնել բարելավել ձեր կայքի վարկանիշը:

Որոշ այլ էջից դուրս SEO գործոններ ներառում են ակնարկներ և հետադարձ կապեր: Այս բաները այլ վայրերից երթևեկություն են մղում դեպի ձեր կայք: Դրանք այն բաները չեն, որոնք դուք կարող եք անել ձեր կայքում այն օպտիմալացնելու համար, այլ արտաքին գործոններ են, որոնք ազդում են ձեր վարկանիշի վրա:

Տեխնիկական SEO

Տեխնիկական SEO-ն կայքի հետամասի օպտիմիզացման գործընթաց է, որն ուղղված է կայքի ճիշտ աշխատանքն ապահովելուն և դրա տեսանելիությունը որոնողական համակարգերում բարելավելուն: Ի տարբերություն բովանդակության կամ օն-փեյջ SEO-ի, տեխնիկական SEO-ն կենտրոնանում է կայքի տեխնիկական ասպեկտների վրա, որոնք անհրաժեշտ են որոնողական համակարգերի ալգորիթմների համար կայքը ճիշտ «կարդալու» և ինդեքսավորման համար: Նույնիսկ եթե կայքը գեղեցիկ է և ունի հրաշալի բովանդակություն, տեխնիկական խնդիրները կարող են խոչընդոտել դրա արդյունավետությունը:

1. Էջի Արագություն

Էջի արագությունը տեխնիկական SEO-ի ամենակարևոր ասպեկտներից մեկն է: Օգտվողները ակնկալում են, որ կայքը կբեռնվի 2-3 վայրկյանից ավելի արագ: Եթե էջը դանդաղ է բեռնվում, դա կարող է հանգեցնել օգտվողների անհամբերության և կայքից հեռանալուն, ինչը բացասաբար է ազդում SEO-ի վրա:

Ինչպե՞ս բարելավել էջի արագությունը.

1. Սեղմել պատկերները.

- Օգտագործիր պատկերների սեղմման գործիքներ, ինչպիսիք են TinyPNG կամ ImageOptim, պատկերների չափը նվազեցնելու համար՝ առանց որակի կորստի:
- Օգտագործիր WebP ֆորմատը, որը ապահովում է ավելի բարձր սեղմման հարաբերակցություն:

2. Օգտագործիր արագ սերվեր.

- Ընտրիր արագ և հուսալի հոսթինգ (օրինակ՝ Cloudflare, AWS, Google Cloud):

3. Մինիմալիզացրու CSS և JavaScript ֆայլերը.

- Հեռացրու ավելորդ կոդը և միավորիր CSS և JavaScript ֆայլերը:

4. Օգտագործիր քեշավորում (Caching).

- Քեշավորումը թույլ է տալիս պահել կայքի տարրերը օգտվողի բրաուզերում, ինչը նվազեցնում է բեռնման ժամանակը:

5. Ստուգիր արագությունը Google PageSpeed Insights-ի միջոցով.

- Այս գործիքը կտրամադրի մանրամասն վերլուծություն և առաջարկություններ էջի արագությունը բարելավելու համար:

2. Բջջային Հարմարվողականություն (Mobile-Friendliness)

Բջջային սարքերի միջոցով ինտերնետ օգտագործողների թիվը արագ աճում է, և Google-ը առաջնահերթություն է տալիս բջջային օպտիմիզացված կայքերին: Եթե կայքը հարմարեցված չէ բջջային սարքերի համար, դա կարող է հանգեցնել որոնման

արդյունքներում ցածր դիրքերի:

Ինչպե՞ս ապահովել բջջային հարմարվողականություն.

1. Օգտագործիր ռեսպոնսիվ դիզայն.
 - Ռեսպոնսիվ դիզայնը ապահովում է, որ կայքը ավտոմատ կերպով հարմարվի օգտագործողի սարքի էկրանի չափին (դեսկտոպ, պլանշետ, սմարթֆոն):
2. Ստուգիր բջջային հարմարվողականությունը Google Mobile-Friendly Test-ի միջոցով.
 - Այս գործիքը կտրամադրի մանրամասն տեղեկատվություն կայքի բջջային օպտիմիզացման վերաբերյալ:
3. Մինիմալիզացրու բջջային էջերի բեռնման ժամանակը.
 - Հեռացրու ավելորդ տարրերը, ինչպիսիք են մեծ պատկերները կամ անհարկի սկրիպտները:

3. Կայքի Կառուցվածք և URL-ներ

Կայքի կառուցվածքը և URL-ները պետք է լինեն պարզ և հասկանալի: Սա օգնում է որոնողական համակարգերին ավելի հեշտությամբ ինդեքսավորել կայքը:

Ինչպե՞ս օպտիմիզացնել կայքի կառուցվածքը.

1. Օգտագործիր հիերարխիկ կառուցվածք.
 - Կայքը պետք է ունենա պարզ և տրամաբանական կառուցվածք՝ գլխավոր էջից մինչև ենթաէջեր:
 - Օրինակ՝
 - Գլխավոր էջ. «Կանացի հագուստ»
 - Ենթաէջ. «Կանացի կոշիկներ»
 - Ենթաէջ. «Գնել կանացի կոշիկներ Երևանում»
2. Ստեղծիր հստակ և հիմնաբառերով հարուստ URL-ներ.
 - URL-ները պետք է լինեն կարճ, հասկանալի և ներառեն հիմնաբառեր:
 - Օրինակ՝ www.yourwebsite.com/kanatsi-koshikner:

4. Մխեմաների Նշում (Schema Markup)

Մխեմաների նշումը կայքի կոդում ավելացվող հատուկ նշում է, որը օգնում է որոնողական համակարգերին ավելի լավ հասկանալ կայքի բովանդակությունը: Սա կարող է բարելավել կայքի տեսանելիությունը որոնման արդյունքներում:

Օրինակներ սխեմաների նշման.

1. Ապրանքի նշում.
 - Օգտագործիր Product սխեման՝ ապրանքի մասին տեղեկատվություն ավելացնելու համար:
 - Օրինակ՝ ապրանքի անվանում, գին, նկարագրություն:

2. Հարցումների նշում.

- Օգտագործիր FAQ սխեման՝ հաճախակի տրվող հարցերի պատասխանները ավելացնելու համար:

5. Կայքի Անվտանգություն (HTTPS)

Կայքի անվտանգությունը կարևոր է ն՝ օգտվողների համար, ն՝ որոնողական համակարգերի համար: Google-ը առաջնահերթություն է տալիս HTTPS պրոտոկոլով կայքերին:

Ինչպե՞ս ապահովել անվտանգություն.

1. Տեղադրիր SSL/TLS վկայական.
 - SSL/TLS վկայականը ապահովում է կայքի և օգտվողի միջև տվյալների գաղտնագրում:
2. Ստուգիր կայքի անվտանգությունը.
 - Օգտագործիր գործիքներ, ինչպիսիք են SSL Labs, կայքի անվտանգությունը ստուգելու համար:

6. Կայքի Ինդեքսավորում և XML Sitemap

Կայքի ինդեքսավորումը կարևոր է, որպեսզի որոնողական համակարգերը կարողանան գտնել և ավելացնել կայքի էջերը իրենց ինդեքսում:

Ինչպե՞ս ապահովել ինդեքսավորում.

1. Ստեղծիր XML Sitemap.
 - XML Sitemap-ը ֆայլ է, որը պարունակում է կայքի բոլոր էջերի ցանկը: Այն օգնում է որոնողական համակարգերին ավելի արագ ինդեքսավորել կայքը:
2. Ներկայացրու Sitemap-ը Google Search Console-ում.
 - Google Search Console-ը թույլ է տալիս վերահսկել և կառավարել կայքի ինդեքսավորումը:

Սև գլխարկ SEO

Սև գլխարկ SEO-ի տեխնիկան օգտագործում է որոնման ալգորիթմները՝ ավելի բարձր վարկանիշ ստանալու համար:

Որոշ ձեռնարկություններ օգտագործում են սև գլխարկի SEO տեխնիկան, քանի որ դրանք արագ արդյունքներ են բերում: Մարդիկ ցանկանում են անհապաղ բավարարվածություն՝ առանց ավելորդ աշխատանք դնելու: Անկումը, սակայն, այն է, որ սև գլխարկ SEO-ն ավարտվում է երկարաժամկետ հեռանկարում բիզնեսին վնասելու համար:

Կան բազմաթիվ մարտավարություններ, որոնք ընկնում են սև գլխարկի տակ.

- Հղման սպամ
- Հիմնաբառի լցում
- Քոդարկում
- Թաքնված տեքստ/հղումներ
- Օգտագործելով անհամապատասխան, հայտնի հիմնաբառեր
- HTML վերնագրերի օպտիմալացում
- Բովանդակության պատճենում
- Հղումներ գնելը

Սրանք ընդամենը մի քանի մարտավարություն են, որոնք ընկերությունները կօգտագործեն իրենց կայքը օպտիմալացնելու համար: Թեև այս մարտավարությունները բարելավում են վարկանիշը, այն չի տևում: Google-ը բռնում է այս տեխնիկան, և եթե ձեզ բռնեն, ձեր կայքը խիստ կտուժի: Որոնման համակարգերը կպատժեն ձեր բիզնեսին և նույնիսկ կարգելեն ձեր կայքը հայտնվել համապատասխան որոնման արդյունքներում: Քանի որ Google-ը ցանկանում է արտադրել միայն համապատասխան որոնման արդյունքներ, այն կարգելվի մարդկանց, ովքեր կարճ ճանապարհով փորձում են բարձրանալ վարկանիշային աղյուսակում:

Մոխրագույն գլխարկ SEO

Մոխրագույն գլխարկ SEO-ն սպիտակ և սև գլխարկների SEO-ի միջին հիմքն է: Այս տեխնիկան այնքան էլ լավ չէ ձեր կայքի համար, բայց այն նշված չէ որպես վատ: Հետևաբար, ինչու է սա «գորշ» գլխարկի SEO մարտավարություն, քանի որ այն ընկնում է գորշ տարածքի մեջ:

SEO-ի այս մեթոդներն արգելված չեն Google-ի կողմից, բայց դրանք չեն համարվում առաջատար դիրքեր ձեռք բերելու ընդունելի միջոց: Մոխրագույն գլխարկների SEO տեխնիկան ռիսկային է, բայց շուկայավարվում է որպես «մատչելի լուծում»: Ընկերությունները, որոնք առաջարկում են գորշ SEO տեխնիկա, նվազեցնում են իրենց ծախսերը, քանի որ նրանք օգտագործում են կասկածելի մեթոդներ:

Ահա գորշ SEO տեխնիկայի մի քանի օրինակ.

- Clickbait հոդվածներ
- Spun բովանդակություն
- Հղումների փոխանակումներ
- Վճարովի ակնարկներ

Սրանք մարտավարություններ են, որոնք ձեռնարկությունները կօգտագործեն իրենց կայքի աշխատանքը որոնման արդյունքներում բարելավելու համար: Մոխրագույն գլխարկների SEO-ի տեխնիկան այնքան էլ արգելված չէ Google-ի կողմից, բայց դրանք ձեր կայքի աճին օգնելու լավագույն միջոցը չեն: Դա կարող է հանգեցնել բացասական հետևանքների, որոնք ավելի շուտ կվնասեն ձեր բիզնեսին, քան օգնեն նրան:

Բացասական SEO

Բացասական SEO-ն եզակի իրավիճակ է ձեր բիզնեսի համար: Դա ձեր բիզնեսի կողմից չէ, այլ ավելի շուտ՝ ուրիշի կողմից: Դուք կտեսնեք բացասական SEO սցենարներ բիզնեսի և նրանց մրցակիցների միջև: Երբ տեսնում եք, որ բիզնեսը օգտագործում է բացասական SEO, նրանք փորձում են վնասել իրենց մրցակիցների հեղինակությանը: Ընկերությունները օգտագործում են մոխրագույն և սև գլխարկների տեխնիկան՝ մրցակիցների բիզնեսին վնաս պատճառելու

համար:

Նրանք իրականացնում են այս տեխնիկան՝ նվազեցնելու բիզնեսի վարկանիշը, որպեսզի նրանք կարողանան բարելավել իրենց սեփականը:

Կան մի քանի ռազմավարություններ, որոնք ընկերությունները կօգտագործեն բացասական SEO-ով.

- Անբնական հղումներ ստեղծելով մրցակցի կայքում
- Բացասական ակնարկների տեղադրում
- Կայքերի կոտրում՝ բովանդակությունը փոփոխելու համար

Բացասական SEO-ի նպատակն է վնասել այլ կայքեր անձնական շահի համար: Դժվար է վերահսկել դա, քանի որ այլ ընկերություններ վնասում են ձեր կայքը: Երբ դուք վարում եք ձեր SEO արշավը, դուք պետք է անընդհատ վերահսկեք այն, որպեսզի համոզվեք, որ գործունեության կամ կատարողականի մեջ որևէ արտասովոր կամ անսպասելի աճ/անկում չկա:

1. 2. 2 SEO-ի տեսակներ

Այս դասակարգումը հիմնված է այն բանի վրա՝ SEO-ն կայքի որ հատվածում է կիրառվում:

Վերնագիր	Բացատրություն
Ներքին SEO (On-page SEO)	Կիրառվում է կայքի ներսում՝ բովանդակության, բանալի բառերի, meta թեգերի օպտիմալացում
Արտաքին SEO (Off-page SEO)	Կիրառվում է կայքից դուրս՝ հղումների ստեղծում, սոցցանցերի ակտիվություն
Տեխնիկական SEO (Technical SEO)	Կայքի տեխնիկական կառուցվածքի և ինդեքսավորման բարելավում՝ sitemap, mobile-friendly, schema
Տեղային SEO (Local SEO)	Վարկանիշի բարձրացում տեղական որոնման արդյունքներում՝ հիմնականում բիզնեսների համար
Բովանդակային SEO (Content SEO)	Բարձրորակ, արժեքավոր և բանալի բառերով հարուստ բովանդակության ստեղծում

On-Page SEO տեխնիկաներ

Ներքին SEO-ն վերաբերում է կայքի էջերի օպտիմալացման գործընթացին՝ այնպես, որ դրանք ավելի լավ արտացոլեն օգտատերերի որոնումներն ու ավելի հեշտությամբ գտնվեն որոնման համակարգերում: Այն ներառում է ինչպես բովանդակությունը, այնպես էլ տեխնիկական ձևավորման ասպեկտները, որոնք ուղղված են կայքի էջերի օպտիմալացմանը:

Ներքին SEO-ի հիմնական տարրերը

1. Հիմնաբառերի օգտագործում (Keyword Usage)

- **Հիմնաբառերը** պետք է օգտագործվեն կայքի հիմնական էջերում՝ հիմնական հոդվերում, ենթավերնագրերում, բովանդակության մեջ և մետա տեգերում (meta tags):
- Այս հիմնաբառերը պետք է ընտրվեն ըստ օգտատերերի հարցումների և համապատասխանեն որոնողական intent-ի (օգտագործողի մտադրություն):

 Օրինակ՝ եթե դու վաճառում ես «մարմնամարզական շապիկներ», ապա պետք է դրանք օգտագործես վերնագրերում և տեքստում:

2. Վերնագրերի կառուցվածք (Heading Structure)

- Սահուն և հստակ կառուցվածք ունեցող էջերը կարևոր են՝ ընթերցողներին հեշտությամբ տրամադրելով ինֆորմացիա:
- Ստորև ներկայացվում է լավ վերնագրերի կառուցվածքը՝
 - **H1** – գլխավոր վերնագիր (միայն մեկ էջում),
 - **H2** – ենթավերնագրեր,
 - **H3** – փոքր ենթաբաժիններ և այլն:

✚

H1:	«Համակարգված տղամարդու շապիկներ մեր խանութում»
H2:	«Տարբեր տեսակի շապիկներ»,
H3:	«Սպորտային շապիկներ»

3. Մետա տեգեր (Meta Tags)

- Մետա տեգերը կարևոր են որոնման արդյունքների համար:
 - **Meta Title** – էջի վերնագիր, որը ցուցադրվում է որոնման արդյունքներում:
 - **Meta Description** – էջի նկարագրություն, որն ավելացնում է կարճ բացատրություն՝ ինչ է կայքում:

✚

Meta Title:	«Ալկոհոլային խմիչքներ՝ առցանց գնել»
Meta Description:	«Գտիր մեծ ընտրություն ալկոհոլային խմիչքների մեր խանութում՝ լավագույն գներով և արագ առաքմամբ:»

4. Նկարների օպտիմալացում (Image Optimization)

- Կայքում օգտագործվող նկարները պետք է ունենան **alt տեգ** (Alternative Text), որն օգնում է ոչ միայն որոնման համակարգերին, այլ նաև անձանց, ովքեր օգտագործում են էկրանային կարդալիչներ:
- Կարճ նկարագրությամբ նկարները նաև ավելացնում են օգտատիրոջ փորձը:

✚

	Օրինակ՝
---	---------

5. Ներքին հղումներ (Internal Links)

- Ներքին հղումները (այդ թվում՝ կոդավորումը) օգնում են կայքի էջերին կապվել միմյանց հետ, ինչը նպաստում է կայքի սողալման և ինդեքսավորման գործընթացին:
- Պետք է ուշադրություն դարձնել, որպեսզի ներքին հղումները բնական տեսք ունենան և օգտատիրոջ համար հարմար լինեն:

✚

«Մեր նոր տղամարդու շապիկների ընտրանին կարող եք տեսնել այստեղ»:	Օրինակ՝
--	---------

6. Բովանդակության որակ (Content Quality)

- Տեսահոլովակներ, նկարներ, բլոգներ կամ արտադրանքի նկարագրություններ պետք է լինեն արժեքավոր ու տեղին՝ ապահովելով օգտատերերին տեղեկություններ և հարցերին պատասխաններ:
- Որպես օրինակ՝ եթե դու ունես օնլայն խանութ, արտադրանքի նկարագրությունները պետք է լինեն մանրամասն՝ ներառելով չափսերը, գույները, նյութերը և այլն:

7. Բովանդակության թարմացում (Content Freshness)

- Թարմ բովանդակությունը շատ կարևոր է՝ դա ուղղակիորեն ազդում է էջի վարկանիշի վրա:
- Google-ը գնահատում է նոր ու արդիական բովանդակությունը, և օգտատերերը հավանաբար ավելի շատ կգրադվեն թարմ տվյալներով:

✓ Ներքին SEO-ի առավելությունները

- Բարձրացնում է կայքի տեսանելիությունը
- Օգնում է բարձրացնել որոնողական համակարգերի արդյունքներում դիրքերը
- Բերում է թիրախավորված այցելուներ, ովքեր ճշգրիտ փնտրում են քո առաջարկվող ծառայություններն ու ապրանքները
- Պարզեցնում է կայքի կառուցվածքն ու բարելավում օգտատիրոջ փորձը

✦ Ամփոփում

Ներքին SEO-ն շատ կարևոր է կայքի օպտիմալացման գործում, քանի որ այն ապահովում է, որ բովանդակությունը ճիշտ կառուցված է՝ չնայած որոնման համակարգերի պահանջներին: Ներքին SEO-ի ճիշտ կիրառում կօգնի բարձրացնել վարկանիշը և բերվող այցելուների որակը:

Արտաքին SEO (Off-page SEO)

Արտաքին SEO-ն իրենից ներկայացնում է այն բոլոր գործողությունները, որոնք

կատարվում են կայքից դուրս՝ նպատակ ունենալով բարձրացնել կայքի հեղինակությունը, վստահելիությունը և վարկանիշը որոնողական համակարգերում:

Այս մեթոդներն օգնում են որոնման համակարգերին (օրինակ՝ Google-ին) հասկանալ, որ կայքը արժեքավոր է, վստահելի և համապատասխան է օգտատերերի որոնումներին, նույնիսկ եթե այդ կայքում այդ պահին ակտիվություն չկա:

Արտաքին SEO-ի հիմնական տարրերը

1. Հետադարձ հղումներ (Backlinks)

- Սա արտաքին SEO-ի ամենակարևոր գործիքն է:
- Երբ այլ կայքեր հղում են անում քո կայքին, որոնողական համակարգերը սա ընկալում են որպես վստահության նշան:
- Որքան բարձր է հղում տվող կայքի հեղինակությունը, այնքան ավելի արժեքավոր է այդ backlink-ը:

Օրինակ՝

Եթե մի հայտնի բլոգ կամ լրատվական կայք հղում է անում քո արտադրանքին կամ հոդվածին, դա բարձրացնում է քո կայքի վարկանիշը:

2. Սոցիալական մեդիա ազդակներ

- Քո կայքի բովանդակության տարածումը սոցցանցերում (Facebook, Instagram, Twitter, LinkedIn և այլն) նույնպես կարող է օգնել SEO-ին:
 - Թեև սոցմեդիայի հղումները սովորաբար “nofollow” են, այնուամենայնիվ, դրանք ստեղծում են վստահելիության ազդակներ:
-

3. Օգտագործողների ակնարկներ (Reviews) և վստահելի հարթակներում ներկայություն

- Google Business էջում կամ այլ հարթակներում (օր. TripAdvisor, Trustpilot) քո բիզնեսի վերաբերյալ դրական կարծիքները նույնպես նպաստում են արտաքին SEO-ին:
- Որքան շատ և որակյալ ակնարկներ ունես, այնքան որոնման համակարգերն ավելի վստահում են քո բիզնեսին:

4. Բլոգերում կամ ֆորումներում հիշատակումներ

- Երբ օգտատերերը ֆորումներում կամ բլոգներում խոսում են քո կայքի մասին (նույնիսկ առանց հղման), դա ևս ազդում է որոնողական ընկալման վրա:

5. Գեսթ փոստինգ (Guest Posting)

- Դու կարող ես հոդված գրել այլ բլոգի համար և դրանում հղում անել քո կայքին: Սա երկկողմանի օգուտ է՝ թե՛ SEO-ի, թե՛ ճանաչելիության համար:

✅ Արտաքին SEO-ի առավելությունները

- Բարձրացնում է կայքի հեղինակությունը
- Օգնում է հայտնվել որոնման արդյունքների վերին հատվածում
- Տարածում է բրենդը նոր լսարանների շրջանում
- Բերում է օրգանական այցելուներ նույնիսկ առանց վճարովի գովազդի

⚠ Կարևոր նշում

Արտաքին SEO-ի դեպքում կարևոր է **բնական և որակյալ հղումների** ստացումը: Եթե օգտագործվեն սև գլխարկ SEO մեթոդներ՝ օրինակ՝ հղումների զանգվածային գնում անորակ կայքերից, դա կարող է վնասել կայքի վարկանիշին:

Տեխնիկական SEO (Technical SEO)

Տեխնիկական SEO-ն վերաբերում է կայքի տեխնիկական և կառուցվածքային ասպեկտների օպտիմալացմանը, որոնք օգնում են որոնողական համակարգերին (օրինակ՝ Google, Bing) հեշտությամբ մուտք գործել, սողալ (crawl), հասկանալ և ինդեքսավորել կայքի բովանդակությունը:

Այն ապահովում է, որ կայքը աշխատի սահուն՝ լինի հասանելի, արագ, անվտանգ և հեշտությամբ ընթերցելի ինչպես օգտատերերի, այնպես էլ որոնման ռոբոտների համար:

Տեխնիկական SEO-ի հիմնական տարրերը

1. Կայքի արագություն (Page Speed)

- Որքան արագ է բեռնվում կայքը, այնքան ավելի լավ:
- Արագ կայքերը տալիս են ավելի լավ օգտատերերի փորձ և դրական ազդում են վարկանիշի վրա:

Օրինակներ՝

- Նկարների սեղմում (image compression)
- CSS/JS ֆայլերի օպտիմալացում
- Կեշավորում (caching)

2. Mobile-Friendly / Հարմարեցվածություն շարժական սարքերին

- Google-ն օգտագործում է “Mobile-First Indexing”, այսինքն՝ առաջին հերթին հաշվի է առնում կայքի մոբայլ տարբերակը:
- Կայքը պետք է ճիշտ ցուցադրվի հեռախոսներով և պլանշետներով:

Օրինակ՝ Responsive դիզայն՝ CSS Media Queries-ի միջոցով:

3. SSL սերտիֆիկատ և HTTPS

- Կայքը պետք է աշխատի անվտանգ կապով՝ `https://`
- Որոնողական համակարգերը նախապատվություն են տալիս անվտանգ կայքերին:

✦ Օրինակ՝ "<http://example.com>" → "<https://example.com>"

4. Sitemap.xml և Robots.txt

- `sitemap.xml` ֆայլը որոնման համակարգերին ներկայացնում է կայքի բոլոր հիմնական էջերը:
- `robots.txt` ֆայլը թույլ է տալիս սահմանել՝ որ էջերն են հասանելի որոնման ռոբոտների համար:

✦ Օրինակ՝ `sitemap`-ը օգնում է Google-ին արագ գտնել քո նոր էջերը:

5. Canonical թեգեր

- Օգտագործվում են կրկնվող բովանդակությունից խուսափելու համար:
- Ցույց է տալիս՝ որ տարբերակը պետք է համարվի հիմնական (canonical) էջ:

✦ Օրինակ՝ եթե ունես միևնույն էջի մի քանի URL՝ `example.com/shirt?id=1` և `example.com/red-shirt`, դու կարող ես նշել՝ որն է հիմնականը:

6. Structured Data (Schema Markup)

- Օգտագործելով JSON-LD կամ Microdata՝ կարող ես որոնման համակարգերին տալ լրացուցիչ ինֆո քո էջի մասին՝ արտադրանքի գին, գնահատական, իրադարձության օր և այլն:

✦ Օրինակ՝ որոնման արդյունքներում քո արտադրանքն ունի աստղանիշներ, գին, և նկար:

7. 404 սխալների կառավարում և վերաուղղումներ (Redirects)

- Կայքում չպետք է լինեն կոտրված հղումներ (broken links) կամ 404 էջեր:
 - Հին կամ փոփոխված URL-ները պետք է ճիշտ վերաուղղվեն (301 redirect) դեպի նոր էջեր:
-

8. Բովանդակության ինդեքսավորման վերահսկում

- Կարող ես որոշել, թե որ էջերը պետք է ինդեքսավորվեն, իսկ որոնք՝ ոչ: Դրա համար օգտագործվում են noindex, nofollow թեգեր:
-

✓ Տեխնիկական SEO-ի առավելությունները

- Բարձրացնում է կայքի արագությունը և հասանելիությունը
- Օգնում է որոնման ռոբոտներին սողալ և ինդեքսավորել կայքը
- Նվազեցնում է տեխնիկական սխալների պատճառով վարկանիշի անկման ռիսկը
- Բարձրացնում է օգտատերերի ընդհանուր փորձը (UX)

Ամփոփում

Տեխնիկական SEO-ն նույնքան կարևոր է, որքան բովանդակությունը: Այն ստեղծում է կայուն հիմք, որի վրա կարելի է կառուցել մնացած SEO աշխատանքները: Առանց տեխնիկական SEO-ի, նույնիսկ լավագույն բովանդակությունը կարող է չհայտնվել որոնման արդյունքներում:

Տեղային SEO (Local SEO)

Տեղային SEO-ն որոնողական համակարգերի օպտիմալացման մի ճյուղ է, որը նպատակ ունի բարելավել քո կայքի տեսանելիությունը **տեղային որոնումների** ժամանակ, այսինքն՝ այն հարցումների, որոնք կատարվում են որևէ կոնկրետ աշխարհագրական վայրի վերաբերյալ:

Օրինակ՝ երբ օգտատերը որոնում է՝ «*շապիկների խանութ Երևանում*», որոնողական համակարգը կփորձի ցուցադրել տվյալ վայրում գործող և օպտիմալացված բիզնեսներ:

Տեղային SEO-ի հիմնական տարրերը

1. Google My Business-ի օպտիմալացում

- Ստեղծիր և հաստատիր քո բիզնեսը Google My Business-ում:
- Լրացրու բոլոր դաշտերը՝ անուն, հասցե, հեռախոս, աշխատանքային ժամեր, կայքի հղում, նկարներ:
- Սա օգնում է, որ քո բիզնեսը հայտնվի Google-ի քարտեզում և տեղային արդյունքներում:

 Օրինակ՝ եթե ունես խանութ Երևանում, օգտատերը քարտեզում կտեսնի քո խանութի դիրքը, գնահատականները, նկարները և ուղղություն ստանալու կոճակը:

2. Հասցեի և կոնտակտային ինֆորմացիայի հետևողականություն (NAP consistency)

- Քո բիզնեսի անունը (**Name**), հասցեն (**Address**) և հեռախոսահամարը (**Phone number**) պետք է նույն ձևով գրվեն բոլոր հարթակներում՝ կայքում, քարտեզներում, կատալոգներում և այլն:
-

3. Տեղային ակնարկներ և գնահատականներ

- Հավաքիր դրական ակնարկներ (reviews) Google-ում և այլ տեղային հարթակներում:
 - Այս ակնարկները ոչ միայն ազդում են վարկանիշի վրա, այլ նաև վստահություն են ստեղծում օգտատերերի մոտ:
-

4. Տեղային հիմնաքառերի օգտագործում

- Օգտագործիր բովանդակության մեջ տեղային հիմնաքառեր՝ ինչպես օրինակ՝ "Էրեբունիում շապիկների խանութ", "ձեռագործ զարդեր Գյումրիում" և այլն:
-

5. Տեղային backlink-ների ստեղծում

- Ստեղծիր հղումներ քո կայքին տեղային բլոգներից, լրատվամիջոցներից կամ բիզնես կատալոգներից (օր. spyur.am, armindex.am և այլն):
-

6. Կայքում տեղային էջեր ունենալը

- Եթե քո բիզնեսը ներկայացված է տարբեր վայրերում, յուրաքանչյուր վայրի համար կարող ես ստեղծել առանձին էջ՝ նկարագրությամբ, հասցեով, նկարներով և այլն:
-

✔ Տեղային SEO-ի առավելությունները

- Բարձրացնում է տեսանելիությունը տեղային որոնման արդյունքներում
- Բերում է հաճախորդներ այն մարդկանցից, ովքեր իրականում փնտրում են քո վայրում
- Օգնում է մրցակցել նույնիսկ խոշոր բրենդների հետ՝ տեղային մակարդակում
- Ընդլայնում է հավատարմությունը՝ դրական ակնարկների շնորհիվ

📌 Ամփոփում

Տեղային SEO-ն հատկապես կարևոր է փոքր և միջին բիզնեսների համար, որոնք աշխատում են կոնկրետ վայրում կամ քաղաքում: Եթե քո մինի օնլայն խանութը նաև ունի ֆիզիկական առաքում որևէ վայրում (օրինակ՝ Երևանում), այս տեսակը կարող է շատ օգտակար լինել քեզ համար:

Բովանդակային SEO (Content SEO)

Բովանդակային SEO-ն որոնողական համակարգերի օպտիմալացման այն տեսակը կամ ուղղությունն է, որը կենտրոնանում է կայքի **բովանդակության** որակի, կառուցվածքի և հիմնաբառերի ճիշտ օգտագործման վրա:

Այն ուղղված է նրան, որ կայքը հայտնվի որոնման արդյունքներում՝ իր արժեքավոր, տեղեկատվական և օգտակար բովանդակության շնորհիվ:

📌 Ինչու է կարևոր բովանդակային SEO-ն

Բովանդակությունն է այն, ինչի հիման վրա Google-ը կամ այլ որոնողական համակարգեր որոշում են, թե ինչի մասին է քո կայքը և արդյոք այն արժե ցուցադրել որոնման արդյունքներում:

Ճիշտ գրված և օպտիմալացված բովանդակությունը կարող է զգալիորեն բարձրացնել կայքի այցելությունները:

Բովանդակային SEO-ի հիմնական տարրերը

1. Հիմնաբառերի (Keywords) ուսումնասիրություն և օգտագործում

- Նախքան բովանդակություն գրելը, պետք է հասկանալ, թե ինչ են մարդիկ փնտրում:
- Հիմնաբառերը պետք է տեղադրվեն՝ վերնագրում, մետա նկարագրության մեջ, առաջին պարբերությունում, ենթավերնագրերում և հիմնական տեքստում:

 Օրինակ՝ «Ժամանակակից տղամարդու սպորտային շապիկ» կարող է լինել հիմնաբառ, եթե վաճառվում են նման ապրանքներ:

2. Որակյալ և արժեքավոր բովանդակություն

- Բովանդակությունը պետք է լինի եզակի, օգտակար և օգտատիրոջ հարցին պատասխանող:
 - Չպետք է լինի պատճենված կամ չափից դուրս կարճ (thin content):
-

3. Վերնագրերի կառուցվածք (H1, H2, H3...)

- Բովանդակությունը պետք է ունենա հստակ կառուցվածք՝ ճիշտ վերնագրերով:
 - H1 – գլխավոր վերնագիր (ընդամենը մեկը էջում),
H2 – ենթավերնագրեր,
H3 – ներքին բաժիններ:
-

4. Մետա տեղեր (Meta title & Meta description)

- Թեև դրանք ուղղակիորեն չեն երևում էջում, որոնման արդյունքներում ցուցադրվում են և մեծ դեր են խաղում այցելությունների քանակի վրա:

✦ Օրինակ՝ Meta Title: «Կանացի շապիկներ առցանց | Dav's Store»

Meta Description: «Գտիր բարձրորակ և ժամանակակից կանացի շապիկներ մեր օնլայն խանութում՝ հարմար գներով և արագ առաքմամբ»:

5. Նկարների օպտիմալացում

- Նկարներն ու մեդիա ֆայլերը պետք է ունենան “alt” հատկանիշ, որը նկարագրում է նկարի բովանդակությունը:
- Սա օգնում է ինչպես որոնման համակարգերին, այնպես էլ մատչելիության համար:

✦ Օրինակ՝

6. Ներքին հղումներ (Internal Linking)

- Մի էջից մյուս էջի վրա հղում անելը օգնում է որոնման ռոբոտներին ման գալ կայքում և օգտատիրոջը՝ ավելի երկար մնալ կայքում:

7. Թարմացում և արդիականացում

- Հին բովանդակության վերանայում և թարմացում նոր տվյալներով:
- Google-ը գնահատում է թարմ և արդիական բովանդակությունը:

✓ Բովանդակային SEO-ի առավելությունները

- Բերում է թիրախավորված այցելուներ՝ հիմնված կոնկրետ որոնումների վրա

- Օգնում է կառուցել վստահելիություն և հեղինակություն քո ոլորտում
- Բարձրացնում է կայքի վարկանիշը և այցելությունների քանակը
- Խթանում է փոխակերպումները (Conversions), օրինակ՝ գնումները

Ամփոփում

Բովանդակային SEO-ն կայքի “սիրտն” է: Առանց որակյալ բովանդակության հնարավոր չէ հասնել կայուն և բարձր որոնողական դիրքերի:

Եթե ունես օնլայն խանութ, ուրեմն բովանդակության SEO-ն կարող է ներառել՝

- ապրանքների նկարագրություններ,
- բլոգ գրառումներ,
- հաճախ տրվող հարցերի բաժին (FAQ),
- ակնարկներ և օգտատերերի կարծիքներ:

Գլուխ 2 Front-End

Front-End-ը (կամ հաճախ կոչվում է նաև **կայքի արտաքին ինտերֆեյս** կամ **օգտագործողի ինտերֆեյսի հատված**) վեբ ծրագրավորման այն մասն է, որը վերաբերում է այն ամենին, ինչ տեսնում է և ինչի հետ փոխազդում է օգտատերը կայք այցելելիս: Այսինքն՝ դա կայքի **տեսանելի և ինտերակտիվ մասն է՝** պատկերներ, տեքստեր, կոճակներ, մենյուներ, ձևեր (Forms) և այլն:

Front-End-ի հիմնական նպատակները

- Ստեղծել կայքի **տեսանելի կառուցվածք**՝ օգտագործելով HTML:
- Ապահովել **դիզայն** և գեղեցիկ տեսք՝ CSS-ի միջոցով:
- Իրականացնել **ինտերակտիվություն**՝ JavaScript-ի օգնությամբ:
- Համոզվել, որ կայքը **հարմար է բոլոր սարքերում**՝ համակարգիչ, պլանշետ, հեռախոս (Responsive Design):

Front-End-ի հիմնական տեխնոլոգիաները

1. HTML (HyperText Markup Language)

Կայքի կմախքն է: Դրանով գրվում է էջի բովանդակությունը՝ վերնագրեր, պարբերություններ, նկարներ, հղումներ և այլն:

2. CSS (Cascading Style Sheets)

Պատասխանատու է կայքի արտաքին տեսքի համար: Օգնում է ձեւավորել HTML տարրերը՝ գույներ, տառատեսակներ, դասավորություն, անիմացիաներ և այլն:

3. JavaScript

Ապահովում է կայքի դինամիկությունն ու ինտերակտիվությունը: Օրինակ՝ մենյուի բացում-փակում, սլայդերներ, վավերացման ստուգում՝ մինչ տվյալների ուղարկում:

Front-End ծրագրավորողն ի՞նչ է անում

Front-End ծրագրավորողը (կամ վեբ դիզայներ/մակետավորող) ստեղծում է այն մասերը, որոնք

- օգտատերը տեսնում է կայքում,
- և որոնց հետ կարող է փոխազդել:

Նա համագործակցում է **Back-End** ծրագրավորողների հետ, որոնք ստեղծում են կայքի ներքին տրամաբանությունը՝ տվյալների պահպանումը, գրանցումը, մուտքը և այլն:

2. 1. HTML (HyperText Markup Language)

2. 1. 1. Ի՞նչ է HTML-ը

HTML ([անգլ.](#)՝ HyperText Markup Language, գերտեքստի նշագրման լեզու), նշագրման լեզու, որն օգտագործվում է [վեբ-էջերի](#) մեծամասնության նշագրման համար: HTML լեզուն վերամշակվում է [դիտարկչի](#) կողմից և ցուցադրվում է մարդու կողմից հեշտ ընթերցման փաստաթղթի տեսքով:

HTML-ի օգնությամբ հեշտությամբ կարելի է ստեղծել համեմատաբար պարզ, սակայն գեղեցիկ ձևավորված փաստաթուղթ: Ի հավելումն փաստաթղթի կառուցվածքի պարզեցմանը՝ HTML-ում աջակցվում էր հիպերտեքստը: Մուլտիմեդիայի հնարավորությունները ավելացվեցին քիչ ավելի ուշ: Ի սկզբանե HTML լեզուն ստեղծվել է որպես փաստաթղթերի կառուցման և ձևավորման միջոց՝ անկախ օգտագործվող արտածման միջոցից (էկրան, տպիչ և այլն) ցուցադրելու միջոց: Նախնական HTML ձևաչափով տեքստը պետք է առանց ոճային և կառուցվածքային ձևափոխման արտածվեր սարքավորման վրա՝ տարբեր տեխնիկական համալրմամբ (ժամանակակից համակարգչի գունավոր էկրանը, օրգանայգերի մոնիթորմային էկրան, բջջային հեռախոսների կամ տեքստերի ձայնային վերարտադրման ծրագրերի կամ սարքավորումների սահմանափակ չափերով էկրան): Սակայն HTML-ի ժամանակակից կիրառումը հեռու է սկզբնական նշանակությունից:

Տեքստային փաստաթղթերը, որոնք պարունակում են HTML կոդ (այդ փաստաթղթերը հիմնականում ունեն .html կամ .htm ընդլայնում), մշակվում են հատուկ ծրագրերի միջոցով, որոնք ցուցադրում են փաստաթուղթը իր ձևավորման համաձայն: Այդ ծրագրերը, որոնք անվանում են [դիտարկիչներ](#) ([անգլ.](#)՝ browser), սովորաբար իրենցից ներկայացնում են հարմար միջավայր վեբ էջերի հայցման, դրանց դիտման և անհրաժեշտության դեպքում նաև դեպի սերվեր տվյալների փոխանցման համար: Այժմ աշխարհում առավել գործածվող դիտարկիչներն են՝ [Մոզիլա Ֆայրֆոքս](#), [Գուգլ Գրոմ](#), [Ինտերնետ Էքսպլորեր](#), [Օպերա](#) և [Սաֆարի](#):

1990-ական թվականի կեսերին առաջ եկավ հետևյալ երևույթը. դիտարկիչների հիմնական արտադրող ընկերությունները՝ [Նեթսքեյփ](#) (անգլ.՝ Netscape) և [Մայքրոսոֆթ](#) ((անգլ.)Microsoft), սկսեցին ներառել HTML-ը տարրերի սեպական հավաքածուներով: Տարբեր դիտարկիչների անհամատեղելիության պատճառով ստեղծվեց խառնաշփոթ, առավելապես բարդություններ էին առաջանում նույն փաստաթուղթը տարբեր դիտարկիչներում նույն տեսքով ստանալու հարցում: Վեր մասնագետները ստիպված էին միևնույն էջը մի քանի տարբերակով ստեղծել: 2000-ականների սկզբին, մի քանի տարով այդ խնդիրը փոքր-ինչ կորցրեց իր հրատապությունը՝ կապված Ինտերնետ էքսպլորերի տարածման և զարգացմամբ հետ:

Տարբերակներ

- [RFC 1866](#) — HTML 2.0, ստանդարտը հաստատվել է [1995 թվականի սեպտեմբերի 22-ին](#);
- HTML 3.2^[3] — [1997 թվականի հունվարի 14](#);
- HTML 4.0^[4] — 1997 թվականի [դեկտեմբերի 18](#);
- HTML 4.01^[5] (փոփոխություններ, որը ավելի կարևոր է, քան թվում է առաջին հայացքից) — [1999 թվականի դեկտեմբերի 24](#);
- ISO/IEC 15445:2000^[6] (այսպես կոչված ISO HTML վրա հիմնված HTML 4.01 Strict-ը) — [2000 թվականի մայիսի 15](#);
- [HTML 5](#)^[7] — մշակման մեջ է: Մշակման վերջը նախատեսվում է մինչև 2014 թվականը:
- HTML 5.1-ը սկսել է մշակել 2012 թվականի դեկտեմբերի 19-ից^[8]:

HTML 1.0-ի պաշտոնական սետիֆիկացիան չկա: 1995 թվականից կային բազմաթիվ HTML-ի ոչ պաշտոնական ստանդարտներ:

2.1.2.HTML փաստաթղթի կառուցվածքը

HTML լեզվով գրված ցանկացած փաստաթուղթ իրենից ներկայացնում է իրար մեջ ներդրված տարրերի հավաքածու (անգլ.՝ *HTML element*), ընդ որում բոլոր տարրերի սկիզբը և վերջը նշվում է հատուկ նշիչներով՝ թեգերով (անգլ.՝ *tag*): Տարրերը կարող են լինել դատարկ, այսինքն՝ չպարունակեն ոչ մի տեքստ կամ տվյալ: Օրինակ `<br />` թեգը ջարդում է տողը՝ հաջորդող տեքստը սկսելով նոր տողից (բայց ոչ նոր պարբերությունից): Այս դեպքում սովորաբար չի նշվում փակող թեգ: Բացի այդ, տարրերը կարող են պարունակել հատկանիշներ (անգլ.՝ HTML ատրիբուտներ), որոնք որոշում են նրանց որոշակի հատկանիշներ, օրինակ տառաչափը *font* տարրի համար: Ատրիբուտները նշվում են բացող թեգի մեջ: HTML փաստաթուղթը սկսվում է `<html>` թեգով և ավարտվում է `</html>` թեգով: Փաստաթուղթը բաղկացած է երկու հիմնական մասերից՝ ծանուցման բաժին և փաստաթղթի մարմին: Ծանուցման բաժինը ներառվում է `<head></head>` թեգերի մեջ: Այս բաժնում ներառվում են նաև տեխնիկական այլ բնութագրեր: Էջի վերնագիրը տրվում է `<title>..</title>` մեկնարկի և ավարտի թեգերի միջև: `html`-փաստաթղթի մարմինը ներառվում է `<body>` և `</body>` թեգերի մեջ: Գրանցումները, որտեղ նշվել են տարրի անվանումը կամ ատրիբուտները, կարևոր չեն (բացի XHTML -ից): Տարրերը կարող են լինել ներդրված: Օրինակ, ինչպես հետևյալ կողը.

```
1. <!DOCTYPE html>
2. <html>
3.   <head>
4.     <meta http-equiv="Content-Type" content="text/html; charset=utf-8" />
5.     <title>HTML Document</title>
6.   </head>
7.   <body>
8.     <p>
9.       <b>
10.         Այս տեքստը թավ է,
11.         <i>հսկ սա՝ նաև ավելի շեղ</i>
12.       </b>
13.     </p>
14.   </body>
15. </html>
```

Կտա հետևյալ արդյունքը՝

Այս տեքստը թավ է, *հսկ սա՝ նաև ավելի շեղ*:

Ստորև բերվում են հատվածներ HTML փաստաթղթից.

```
1. <strong>Բացող և փակող պիտակների միջև ընկած տեքստ: Այն թավատառ է:</strong>
```

```
1. <a href="http://www.example.com">Այս պիտակը պարունակում է href հատկանիշը: Սա հղում է, որը տանում է եջի http://www.example.com էջ</a>
```

Անհրաժեշտ է տարբերել և , <i> և թեզերը. թեև երկուսն էլ նույն տեսքն ունեն, սակայն և թեզերը [որոնորակաև համակարգերի](#) համար համարվում են առավել կարևոր:

Իսկ այստեղ բերվում է դատարկ թեզի օրինակ.

```
1. <b>
2.     Այս տեքստը կլինի թավ,
3.     <i>իսկ սա՝ նաև շեղ</i>
4. </b>
```

Դիտարկչի էկրանին կարտածվի հետևյալ կերպ՝ **Այս տեքստը կլինի հաստ, *իսկ սա՝ նաև շեղ***:

Բացի էլեմենտներից, HTML փաստաթղթում օգտագործվում են նաև հատուկ նշաններ, որոնք նշվում են &անուն ձևով, օրինակ. ©-ն կարտածի հեղինակային իրավունքի նշանը՝ ©:

Յուրաքանչյուր HTML փաստաթուղթ, որ համապատասխանում է HTML ստանդարտի որևէ տարբերակի, պետք է սկսվի <!DOCTYPE...>-ով, որն հայտարարում է HTML-ի տարբերակը և ունի այս տեսքը՝

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
```

Եթե այս տողը նշված չէ, ապա դիտարկչում փաստաթղթի ճիշտ արտածման հասնել դժվար կլինի: Այնուհետև պետք է գրվեն <html><head><body> թեզերը:

HTML փաստաթղթերը կարող են ձևափոխվել CSS համակարգչային լեզվի միջոցով, որի համար պետք է փաստաթղթի մեջ ընդգրկել <style></style> կողը, որոնց միջև գրվում է հատուկ կոդագրություն: Օրինակ՝

```
1. <style>
2. body{
3.     background-color: #D91C3F;
4.     font-family: Segoe UI;
5. }
6. </style>
```

Այս կողը էջին ստիպում է, որ նրա ֆոնի գույնը փոխվի և դառնա կարմրագույն: #(և այլն) գունային կոդի ֆորմատը կոչվում է HEX: Նաև խմբագրողը կարող է պարզապես օգտագործել գույնի անունը, բայց առանց երանգների: Կամ, կարելի է օգտագործել [RGB](#) կոդային համակարգը:

Նաև, HTML կոդային համակարգը արհամարհում է ազատ տառերը:

HTML փաստաթղթի մեջ մի նոր գիծ ստանալու համար պետք է գրել հետևյալ կողը՝

1.

2. , իսկ մի գիծ ավելացնելու համար`
3. <hr>

HTML փաստաթղթեր ստեղծելու համար օգտագործվում են [HTML մասնիկներ](#), որոնք իրենցից ներկայացնում են HTML-ի թեզեր պարփակված ուղղանկյուն փակագծերի մեջ: Օրինակ՝ <p>:

Դեպքերի մեծ մասում մասնիկները հանդիպում են զույգ թեզերով, որոնք են "բացման թեզը" <p> և "փակման թեզը" </p>, որի մեջ դրվում է "/" նշանը: Ներկայացվող ինֆորմացիան գրվում է այդ երկու թեզերի արանքում:

Որոշ դեպքերում բացման թեզերը իրենց մեջ պարունակում են լրացուցիչ ատրիբուտներ, որոնք օգտագործվում են հավելյալ ինֆորմացիա ներկայացնելու համար: Հիմնական դեպքերում դրանք օգտագործվում են HTML փաստաթղթին այլ ֆորմատի նյութեր կցելու համար նախատեսված թեզերի հետ կամ մասնիկի մեջ ներկայացված ինֆորմացիայի կոորդինատները նշելու համար:

Օրինակ՝ :

Առանձին մասնիկներ, ինչպիսին է տողընդհատը
 , իրենց մեջ չեն պարունակում տեքստ կամ այլ թեզեր և պարունակում են միայն մեկ դատարկ բացման թեզ՝ առանց փակման թեզի:

Մասնիկների մի մասը կամավոր օգտագործման են և նրանց օգտագործումը փաստաթղթի մեջ ոչինչ չի փոխում: Այդպիսին է օրինակ պարբերության մասնիկը <p>: Այդ մասնիկները օգտագործվում են հին բրաուզերների համար ստեղծվող կայքերի մեջ:

Այնուամենայնիվ, HTML մասնիկի հիմնական տեսքն է <թեզ ատրիբուտ1="արժեք1" ատրիբուտ2="արժեք2">պարունակություն</թեզ>-ը: Իսկ դատարկ մասնիկների դեպքում, ինչպիսին են
 և մասնիկները <թեզ ատրիբուտ1="արժեք1" ատրիբուտ2="արժեք2">-ը: Դատարկ մասնիկների մեջ փակման թեզի օգտագործումը ո՛չ անհրաժեշտ է, ո՛չ էլ թույլատրելի: Իսկ, եթե ատրիբուտին արժեք տրված չէ, օգտագործվում են հիմնական արժեքները:

2.1.3 Օգտագործված HTML թեգերի նկարագրությունը

Կուրսային աշխատանքի շրջանակներում օգտագործվել են HTML-ի մի շարք հիմնական և կարևոր թեգեր, որոնց միջոցով ձևավորվել է կայքի կառուցվածքը և բովանդակությունը: Ստորև ներկայացված են ամենակարևոր թեգերը և նրանց նշանակությունը:

1. <html>

Այս թեգը պարունակում է ամբողջ HTML փաստաթուղթը: Այն վերնագրված է <!DOCTYPE html> հայտարարությամբ, որը տեղեկացնում է բրաուզերին, որ սա HTML5 փաստաթուղթ է:

2. <head>

Պարունակում է մետատվյալներ՝ որոնք ուղղակիորեն չեն երևում կայքի էջում, սակայն կարևոր են՝ օրինակ էջի վերնագիր, տառատեսակների միացում, SEO-ի պարամետրեր և այլն:

3. <title>

Ներկայացնում է էջի վերնագիրը, որը երևում է բրաուզերի վերնամասում (tab-ի վրա):

4. <body>

Պարունակում է էջի տեսանելի բովանդակությունը՝ վերնագրեր, տեքստեր, նկարներ, ձևեր և այլն:

5. <header>

Օգտագործվել է կայքի վերնամասը ներկայացնելու համար: Սովորաբար այստեղ են տեղադրվում լոգոն, մենյուն կամ ողջույնի հաղորդագրությունը:

6. <nav>

Նշանակված է նավիգացիոն հատվածի համար: Այստեղ տեղադրվում են հղումներ դեպի կայքի տարբեր էջեր կամ բաժիններ:

7. <main>

Ներկայացնում է էջի հիմնական բովանդակությունը: Դա այն հատվածն է, որը հիմնականում տարբերվում է մյուս էջերից:

8. <section>

Օգտագործվել է բովանդակությունը բաժանելու համար՝ ըստ թեմաների կամ տրամաբանական հատվածների:

9. <footer>

Ներկայացնում է էջի ներքևի հատվածը: Այստեղ սովորաբար տեղադրվում են հեղինակային տվյալներ, կոնտակտային ինֆորմացիա կամ լրացուցիչ հղումներ:

10. <h1> - <h6>

Վերնագրերի թեզեր են՝ օգտագործվել են էջի տարբեր բաժինների համար: <h1> ամենակարևորն է, իսկ <h6>՝ ամենափոքր մակարդակի վերնագիրը:

11. <p>

Օգտագործվել է պարբերություններ ստեղծելու համար:

12. <a>

Ստեղծում է հղում դեպի այլ էջ կամ կայք:

**13. **

Օգտագործվել է կայքում պատկերներ ցուցադրելու համար:

**14. , , **

Ստեղծում են չղասակարգված () և դասակարգված () ցուցակներ, իսկ -ն՝ ցուցակի տարրերը:

15. <form>, <input>, <button>

Օգտագործվել են ձևեր ստեղծելու և տվյալների մուտք ապահովելու համար (օրինակ՝ կապի ձև, որոնման դաշտ):

2.1.4.Semantic HTML-ի կիրառությունը

Semantic HTML-ը ենթադրում է այնպիսի թեգերի օգտագործում, որոնք ոչ միայն ցուցադրում են բովանդակություն, այլ նաև իմաստային նշանակություն են հաղորդում այդ բովանդակությանը: Այսպիսի թեգերը՝ ինչպես օրինակ <header>, <nav>, <main>, <section>, <article>, <footer>, հստակ ցույց են տալիս, թե ինչ նպատակ է ծառայում տվյալ հատվածը էջում:

Սեմանտիկ թեգերի կիրառումը վեբ կայքերում ունի մի շարք առավելություններ.

- Բարձրացնում է կայքի **ընթերցելիությունը** որոնման համակարգերի և օգնության տեխնոլոգիաների (օրինակ՝ ձայնային ընթերցիչներ) համար:
- Նպաստում է **SEO-ի** արդյունավետությանը՝ թույլ տալով որոնման համակարգերին ճիշտ հասկանալ էջի կառուցվածքը:
- Ապահովում է **տրամաբանական և մաքուր կոդ**, որը հեշտ է խնամել ու զարգացնել:

Օրինակներ իմ նախագծից

Իմ կուրսային նախագծում ակտիվորեն կիրառվել են սեմանտիկ HTML թեգեր.

- <header> թեգը օգտագործվել է կայքի վերնամասը սահմանելու համար, որտեղ տեղադրվել է կայքի անունը և հիմնական մենյուն:
- <nav> թեգը կիրառվել է նավիգացիոն հղումները պարունակելու համար:
- <main> թեգը պարունակում է էջի հիմնական բովանդակությունը:
- <section> թեգը բաժանել է տարբեր բովանդակային հատվածները՝ օրինակ՝ ապրանքների ցուցադրություն, տեղեկատվական բաժիններ:
- <footer> թեգը ներառել է էջի վերջնամասը, որտեղ նշված են հեղինակային իրավունքները և կոնտակտային տվյալները:

Սեմանտիկ HTML-ի շնորհիվ կայքի կառուցվածքը դարձել է ավելի հասկանալի և պրոֆեսիոնալ՝ ինչպես տեխնիկական, այնպես էլ օգտատիրական տեսանկյունից:

2.1.5.HTML և SEO-ի կապը

SEO (Search Engine Optimization) կամ որոնման համակարգերի օպտիմալացում, վեբ կայքի կառուցման կարևոր փուլերից մեկն է, որի նպատակն է կայքը դարձնել ավելի հասանելի և տեսանելի որոնման համակարգերում (օրինակ՝ Google, Bing): Այս գործընթացում մեծ դեր ունի **HTML-ը**, քանի որ հենց HTML թեգերի միջոցով որոնման համակարգերը հասկանում են կայքի կառուցվածքն ու բովանդակությունը:

Ինչպե՞ս է HTML-ը ազդում SEO-ի վրա

1. Վերնագրերի ճիշտ օգտագործում (<h1> - <h6>)

Հիմնական վերնագիրը (<h1>) պետք է լինի եզակի ամեն էջի համար և հստակ փոխանցի էջի բովանդակությունը: Մնացած վերնագրերը (<h2>, <h3> և այլն) պետք է օգտագործվեն բովանդակությունը կառուցվածքային մակարդակով բաժանելու համար: Դա օգնում է ինչպես օգտատիրոջ, այնպես էլ որոնման համակարգի համար:

2. Meta թեգեր (<meta>) head բաժնում

Օգտագործվում են էջի նկարագրությունը, բանալի բառերը և կողավորումը փոխանցելու համար: Դրանք կարևոր են որոնման արդյունքներում կայքի ճիշտ ներկայացման համար:

Օրինակ՝

```
1. <meta name="description" content="Ապրանքների բարձր որակ, մատչելի գներով:">
```

3. Semantic թեգերի օգտագործում (<main>, <article>, <section>, <nav>, <footer>)

Սեմանտիկ թեգերը որոնման համակարգերին օգնում են հասկանալ էջի որ հատվածն ինչ նպատակ ունի: Դա մեծացնում է էջի SEO-ն:

4. ALT ատրիբուտը պատկերների համար ()

Երբ պատկերները ունեն բացատրող alt ատրիբուտ, որոնման համակարգերը կարող են պատկերները ճիշտ դասակարգել և ցույց տալ որոնման մեջ: Բացի այդ, դա բարելավում է նաև կայքի մատչելիությունը:

Օրինակ՝

```
1. 
```

5. Հղումների ճիշտ կառուցվածք ()

SEO-ի համար կարևոր է ներքին և արտաքին հղումների ճիշտ տեղադրումը: Բանալի բառերի վրա հղումներ տալը օգնում է որոնման համակարգերին հասկանալ էջի կապերը:

6. Ոչ կրկնվող վերնագրեր և URL-ներ

Յուրաքանչյուր էջի վերնագիր (<title>) պետք է լինի եզակի և համապատասխան բովանդակությանը:

Իմ նախագծում

Իմ կայքում օգտագործվել են բոլոր վերոնշյալ սկզբունքները՝

- Յուրաքանչյուր էջ ունի եզակի <title> և <meta description>:
- Պատկերները ունեն համապատասխան alt բացատրություններ:
- Կառուցվածքն իրականացված է սեմանտիկ HTML թեգերով՝ ինչը հեշտացնում է բովանդակության ընկալումը:
- Վերնագրերը դասավորված են տրամաբանական հերթականությամբ՝ սկսած <h1>-ից:

Այս ամենը նպաստում է կայքի SEO ցուցանիշների բարելավմանը և որոնման արդյունքներում ավելի բարձր դիրք գրավելուն:

2. 2. **CSS** (Cascading Style Sheets)

2.2.1 Ի՞նչ է CSS-ը և ինչու է այն կարևոր

CSS (Cascading Style Sheets — Կասկադային ոճաթերթեր) վեբ տեխնոլոգիա է, որը կիրառվում է վեբ էջերի արտաքին տեսքի ձևավորման համար: Այն աշխատում է համատեղ **HTML**-ի հետ՝ ապահովելով վեբ կայքի գեղագիտական և ինտերակտիվ կառուցվածքը: Եթե HTML-ը կայքի կմախքն է, ապա CSS-ը նրա հագուստն է:

CSS-ի օգնությամբ հնարավոր է վերահսկել, թե ինչպես պետք է ներկայացվի HTML տարրերի տեսքը՝ ներառյալ գույները, տառատեսակները, չափերը, դիրքը, հեռավորությունները, սահմանագծերը և նույնիսկ անիմացիաները:

CSS-ի կարևորությունը կայքի մեջ

1. Արտաքին տեսքի կառավարում մեկ վայրից

CSS-ի միջոցով կարելի է միևնույն ոճերը կիրառել տարբեր էջերի վրա՝ առանց HTML կոդը կրկնելու: Սա մեծապես հեշտացնում է կայքի սպասարկումը:

2. Բարելավում է օգտագործողի փորձը (UX)

Գեղեցիկ և լավ դասավորված ինտերֆեյսը ավելի հարմար և գրավիչ է օգտատիրոջ համար: CSS-ը օգնում է ստեղծել այդ փորձը:

3. Անջատում է կառուցվածքն ու ձևավորումը

HTML-ը ապահովում է բովանդակությունը, իսկ CSS-ը՝ դրա տեսքը: Այս բաժանումը ապահովում է մաքուր և կազմակերպված կոդ:

4. Ապահովում է պատասխանիչ կայք (Responsive Design)

CSS-ի մեդիա հարցումների (media queries) միջոցով կայքը կարող է հարմարվել տարբեր էկրանների չափերին՝ հեռախոսներ, պլանշետներ, համակարգիչներ:

5. Անիմացիաներ և անցումներ

CSS-ը հնարավորություն է տալիս ստեղծել հարթ անիմացիաներ՝ առանց JavaScript-ի օգտագործման՝ օրինակ, կոճակի վրա սահուն ֆոնային փոփոխություն:

2.2.2. CSS սինտաքսը և կառուցվածքը

CSS-ը բաղկացած է կանոններից (rules), որոնք կիրառվում են HTML տարրերի վրա՝ ձևավորելու համար դրանց տեսքը: Ամեն CSS կանոն բաղկացած է երեք հիմնական մասից՝

- **Ընտրիչ (selector)** — ասում է, թե որ HTML էլեմենտին է կիրառվում ոճը,
- **Բնութագրիչ (property)** — այն հատկանիշը, որը ցանկանում ենք փոխել (օրինակ՝ գույն, չափ),
- **Արժեք (value)** — այդ հատկանիշի նշանակությունը:

♦ Օրինակ՝

```
1. p {  
2.   color: red;  
3.   font-size: 16px;  
4. }
```

Այս կանոնը ընտրում է բոլոր <p> տարրերը և դարձնում է նրանց տեքստը կարմիր, իսկ տառաչափը՝ 16px:

2.2.3. CSS-ի միացման եղանակները

CSS-ը կարելի է միացնել HTML-ին երեք տարբեր ձևով՝

1. **Inline CSS**

Ոճերը գրվում են ուղիղ HTML տարրի մեջ style ատրիբուտով:

Օրինակ՝

```
1. <p style="color: blue;">Բարև աշխարհ</p>
```

2. **Internal CSS**

Ոճերը գրվում են <style> թեգի մեջ՝ HTML էջի <head> հատվածում:

Օրինակ՝

```
1. <head>
2.   <style>
3.     h1 {
4.       color: green;
5.     }
6.   </style>
7. </head>
8.
```

3. **External CSS**

Ոճերը գրվում են առանձին .css ֆայլում, որը միացվում է HTML-ին <link> թեգով:

Օրինակ՝

```
1. <link rel="stylesheet" href="styles.css" />
```

styles.css ֆայլում կարող է լինել՝

```
1. body {
2.   background-color: #f9f9f9;
3. }
```

♦ External մեթոդը համարվում է ամենալավը՝ վեբօգտագործելիության և մաքրության տեսանկյունից:

2.2.4. Ընտրիչներ (Selectors) և դրանց տեսակները

CSS-ում ընտրիչները թույլ են տալիս թիրախավորել կոնկրետ HTML տարրեր: Ահա հիմնական տեսակները՝

1. Տարրային ընտրիչ (Element selector)

Օրինակ՝ p, h1, div

2. Դասային ընտրիչ (Class selector)

Օգտագործում է կետ . նշանը: Օրինակ՝ .container

```
1. .title {  
2.   color: orange;  
3. }
```

3. Իդենտիֆիկատոր ընտրիչ (ID selector)

Օգտագործում է հեշ # նշանը: Օրինակ՝ #header

```
1. #main {  
2.   background-color: lightblue;  
3. }
```

4. Խմբային ընտրիչներ

Միաժամանակ մի քանի տարրերի վրա նույն ոճի կիրառում: Օրինակ՝

```
1. h1, h2, h3 {  
2.   color: darkblue;  
3. }
```

5. Երկրորդային ընտրիչներ (descendant, child, sibling)

Օրինակ՝ div p, ul > li, h2 + p

2.2.5. Տեքստի ձևավորում CSS-ով

CSS-ը թույլ է տալիս մանրամասն վերահսկել տեքստի տեսքը՝ դարձնելով էջը ավելի ընթեռնելի և գեղագիտական:

♦ Տեքստի հիմնական հատկությունները՝

Հատկանիշ (Property) Նկարագրություն		Օրինակ
color	Տեքստի գույնը	color: red;
font-size	Տառաչափը	font-size: 20px;
font-family	Տառատեսակը	font-family: Arial, sans-serif;
font-weight	Տառերի հաստությունը	font-weight: bold;
text-align	Տեքստի հորիզոնական դիրքը	text-align: center;
line-height	Տողերի բարձրությունը	line-height: 1.5;
text-decoration	Ստորագծում, զծանցում	text-decoration: underline;

Օրինակ՝

```
1. p {  
2.   color: #333;  
3.   font-size: 18px;  
4.   font-family: 'Verdana', sans-serif;  
5.   line-height: 1.6;}
```

2.2.6.Տեքստի ձևավորում CSS-ով (գույն, չափ, տառատեսակ և այլն)

CSS-ը թույլ է տալիս վերահսկել տեքստի տեսքը՝ դարձրելով այն ընթերցելի և գրավիչ: Հիմնական հատկություններ՝

- `color` — տեքստի գույնը, օրինակ՝ `color: #333333;`
- `font-size` — տառաչափի չափը, օրինակ՝ `font-size: 16px;`
- `font-family` — տառատեսակը, օրինակ՝ `font-family: Arial, sans-serif;`
- `font-weight` — տառերի հաստությունը, օրինակ՝ `font-weight: bold;`
- `text-align` — տեքստի ուղղությունը, օրինակ՝ `text-align: center;`
- `line-height` — տողերի բարձրությունը, օրինակ՝ `line-height: 1.5;`

Օրինակ՝

```
1. p {  
2.   color: darkblue;  
3.   font-size: 18px;  
4.   font-family: 'Verdana', sans-serif;  
5.   line-height: 1.6;  
6.   text-align: justify;  
7. }  
8.
```

2.2.7.Էլեմենտների չափսեր և տեղակայում (width, height, margin, padding)

- `width` և `height` սահմանում են էլեմենտի լայնությունը և բարձրությունը:
- `margin` էկրանին էլեմենտի շուրջը ազատ տարածություն է տալիս՝ տարրերը իրարից հեռացնելու համար:
- `padding` էլեմենտի ներսում տարածություն է ստեղծում նրա պարունակության և եզրերի միջև:

Օրինակ՝

```
1. div {  
2.   width: 300px;  
3.   height: 150px;  
4.   margin: 20px;  
5.   padding: 10px;  
6.   background-color: lightgray;  
7. }
```

2.2.8.Box Model-ի բացատրությունը

Box Model-ը վեբ էլեմենտների դիզայնի հիմքն է: Ամեն HTML տարր ներկայացվում է որպես քառակուսի (կամ ուղղանկյուն) տուփ, որը բաղկացած է չորս մասից՝

1. **Content** — տարրի պարունակությունը (տեքստ, պատկեր և այլն)
2. **Padding** — պարունակության և սահմանագծի միջև տարածությունը
3. **Border** — տարրի արտաքին եզրը
4. **Margin** — տարրի և հարևան տարրերի միջև բաց տարածությունը

2.2.9.Գույների օգտագործում՝ HEX, RGB, HSL

- **HEX** — գույնը ներկայացվում է 6-ճան (կամ 3-նշան) հեքսադեցիմալ ձևով, օրինակ՝ #FF5733
- **RGB** — գույնը ներկայացվում է կարմիր (Red), կանաչ (Green), և կապույտ (Blue) մասերի թվերով, օրինակ՝ rgb(255, 87, 51)
- **HSL** — գույնը ներկայացվում է երանգով (Hue), հագեցվածությամբ (Saturation), և պայծառությամբ (Lightness), օրինակ՝ hsl(14, 100%, 60%)

2.2.9. Դասավորության մոդելներ (Display, Position)

- **display** սահմանում է, թե ինչպես է էլեմենտը ցուցադրվում՝
 - block (բլոկային տարրեր, սկսվում են նոր տողի վրա),
 - inline (տեքստային տարրեր, շարքի մեջ),
 - flex (հարմար է բլոկ տարրերի տեղադրման համար),
 - grid (ցանցային դասավորության համար):
 - **position** վերահսկում է տարրի դիրքը էջում՝
 - static (ստանդարտ՝ ըստ հոսքի),
 - relative (տեղափոխվում է իր սովորական դիրքից),
 - absolute (տեղադրվում է սուբյեկտի նկատմամբ),
 - fixed (նույն դիրքում է էկրանին սքրոլելու ժամանակ):
-

2.2.10. Flexbox-ի կիրառումը

Flexbox (Flexible Box Layout)՝ CSS-ի հզոր գործիք է, որը օգնում է դասավորել տարրերը տողերով կամ սյուներով՝ առանց բարդ հաշվարկների:

Հիմնական հատկություններ՝

- `display: flex;` — ակտիվացնում է Flexbox-ը տարրի համար:
- `justify-content` — հորիզոնական հարմարեցում (առաջ, կենտրոն, վերջ, տարածված):
- `align-items` — ուղղահայաց հարմարեցում:
- `flex-direction` — շարքի ուղղությունը (row, column):

Օրինակ՝

```
1. .container {  
2.   display: flex;  
3.   justify-content: center;  
4.   align-items: center;  
5.   height: 200px;  
6. }
```

2.2.11.CSS Grid Layout-ի կիրառումը

CSS Grid թույլ է տալիս ստեղծել երկուաչափ ցանցային դասավորություններ՝ հստակ վերահսկելով շարքերն ու սյուները:

Հիմնական հատկանիշներ՝

- `display: grid;` — ակտիվացնում է ցանցը:
- `grid-template-columns` և `grid-template-rows` — սահմանում է սյուների և շարքերի չափերը:
- `gap` — դուրի/տարածության բացը սյուների և շարքերի միջև:

Օրինակ՝

```
1. .grid-container {  
2.   display: grid;  
3.   grid-template-columns: repeat(3, 1fr);  
4.   gap: 10px;  
5. }
```

2.2.12.CSS փոփոխականներ (Custom Properties / Variables)

CSS փոփոխականները (կոչվում են նաև Custom Properties) թույլ են տալիս պահպանել և վերագրագործել արժեքներ մի վայրում, ինչը շատ հարմար է կայքի ոճերի միատարրություն պահպանելու և փոփոխություններ անելու ժամանակ:

- CSS փոփոխականները սահմանվում են երկու փոքրակետով և հիֆենով՝ օրինակ՝ `--main-color`:
- Փոփոխականները սովորաբար դրվում են արմատային տարրի վրա (`:root`), որպեսզի հասանելի լինեն ամբողջ էջում:

Օրինակ՝

```
1. :root {
2.   --main-color: #3498db;
3.   --font-size: 16px;
4. }
5. body {
6.   color: var(--main-color);
7.   font-size: var(--font-size);
8. }
9. button {
10.  background-color: var(--main-color);
11.  font-size: var(--font-size);
12. }
```

Արժեքների փոխելը մեկ տեղում կփոխի դրանք ամբողջում, ինչը խնայում է ժամանակը և հեշտացնում մաքրման և փոփոխությունների գործընթացը:

2.2.13. Մաքուր և վերագրագործելի CSS գրելու խորհուրդներ

1. **Օգտագործիր CSS փոփոխականներ**՝ հարմարեցվող և միատարր գույների, չափերի և այլ հատկությունների համար:
2. **Գործածի անուններ (naming conventions)**, օրինակ՝ BEM մեթոդաբանություն (block__element--modifier), որը օգնում է դասավորել դասերն ըստ կառուցվածքի և նպատակների:
3. **Համակարգիչի ոճերը՝ ըստ տրամաբանության**՝ օրինակ՝ բոլոր տեքստային ոճերը միասին, դասավորության ոճերը միասին:
4. **Կրճատի կրկնությունները**՝ օգտագործիր համատեղող ընտրիչներ և ընդհանուր կանոններ:
5. **Օգտագործիր կոմպոնենտների ոճավորում**՝ ստեղծիր սովորական դասեր, որոնք վերագրագործում են տարբեր վայրերում, օրինակ՝ .btn, .card:
6. **Հեռացրու անպետք ոճերը և առանձին ֆայլեր բաժանիր** (օրինակ՝ reset.css, typography.css, layout.css):
7. **Համատեղելիություն և արձագանքող դիզայն** հաշվի առիր, օգտագործիր Media Queries, ոճավորիչ տարբեր էկրանների համար:
8. **Փորձիր CSS ֆրեյմվորքեր կամ պրեպրոցեսորներ** (Sass, LESS) երբ ավելի բարդ նախագծեր են գրում, բայց եթե սովորական աշխատանք է, բավական է մաքուր CSS:

2.3. JS (JavaScript)

2.3.1. JavaScript-ը ինչ է և ինչու է կարևոր

JavaScript-ը — վեբի հիմնական ծրագրավորման լեզուն է, որը թույլ է տալիս ինտերակտիվություն ավելացնել վեբէջերին:

- Այն աշխատում է հաճախ հենց բրաուզերում և փոխադրում է HTML և CSS-ի հետ՝ կենդանացնելու էջը:
- JS-ը կարող է փոփոխել էջի պարունակությունը՝ առանց վերաթափելու (DOM մանիպուլյացիա):
- Հիմնականում օգտագործվում է օգտատերերի հետ փոխգործակցության, ձևերի վավերացման, անիմացիաների, խաղերի, դինամիկ տվյալների բեռնման և շատ ուրիշ առաջադրանքների համար:

ՋավաՍկրիպտ ([անգլ.](#)՝ JavaScript), ինտերպրետացվող ծրագրավորման լեզու: Այն համացանցում ամենահանրահայտ սկրիպտային լեզուն է և աշխատում է բոլոր հանրահայտ [բրաուզերներում](#), որոնց թվում են [Google Chrome](#), [Mozilla Firefox](#), [Օպերա](#) և [Սաֆարի](#):

Այս լեզուն առաջին անգամ կիրառվել է Netscape Communications-ի կողմից Netscape Navigator 2 beta (1995) բրաուզերում: ՋավաՍկրիպտ լեզուն ստեղծվել է [Բրենդան Այկի](#) կողմից [1995](#) թվականին:

Ի՞նչ է ՋավաՍկրիպտը

- ՋավաՍկրիպտը նախագծվել է կայքերի [HTML](#) էջերին դինամիկություն հաղորդելու համար
- ՋավաՍկրիպտը սկրիպտային լեզու է
- ՋավաՍկրիպտը սովորաբար ներառվում է ուղղակիորեն [HTML](#) ֆայլերի մեջ
- ՋավաՍկրիպտը ինտերպրետացվող լեզու է

ԶավաՍկրիպտ

Նախքան [Բրենդան Այկի](#) ներգրավումը [Netscape](#) ընկերություն (1995 թվականի ապրիլի 4), խնդիր էր դրված ընդլայնել ծրագրավորման սխեման՝ [Scheme](#) կամ Netscape զննարկիչում փոփոխության ենթարկել որոշ կարևորագույն տարրեր: Քանի-որ պահանջը դրված էր խիստ նպատակային, Էյիային ներգրավեցին ընկերություն, որպես սերվերային նյութերի պատասխանատու: Նա այս աշխատանքը կատարեց շուրջ մեկ ամիս, զբաղվեց [HTTP](#)-ի պրոտոկոլի բարելավման աշխատանքներով: Այստեղ նա ծավալեց նաև նոր ծրագրավորման աշխատանքներ: Մայիսի 8-ին ծրագրավորողը ետ վերադարձավ, զբաղվելով հաճախորդների սպասարկման աշխատանքներով (զննարկիչում), որտեղ էլ սկսեց վաստակել նոր ծրագրավորման լեզվի առաջխաղացման հաշվին: Բացի Թոմ Պակինայից (Tom Paquin), Պիկա Շելլան (Rick Schell) ևս համոզված էր, որ Netscape-ը պետք է աջակցի ծրագրավորման լեզվին, որը գրված է HTML կոդով:



Մարկ Անդրեսեն



Բիլ Ջոյ

Բրենդան Էյիսի հետ համագործակցում էին հիմնադիր ([Netscape Communications](#)) [Մարկ Անդրեսեն](#) և համահեղինակ ([Sun Microsystems](#)) [Բիլ Ջոյ](#): Որպեսզի հաջողությամբ ավարտեին աշխատանքը, ընկերությունը համագործակցության պայմանագիր ստորագրեց: Նրանք նպատակ էին դրել՝ ստեղծել այնպիսի ծրագրավորման լեզու, որը բազային ծրագրավորման մաս կկազմեր, հարմար կլիներ և՛ ոճաբանի, և՛ ցմ ծրագրավորողի համար, չէր պահանջի լրացուցիչ ծավալուն և բարդ գործողություններ կատարելու անհրաժեշտություն: Առաջին ծրագիրը կոչվում էր Mocha, այնուհետև անվանափոխվեց LiveScript և նախատեսված է թե՛ որպես ծրագրավորում՝ հաճախորդի համար, և թե՛ որպես ծրագիր՝ սերվերի համար (այնտեղ այն պետք է ունենա LiveWire

անվանում) : Որպես սինտակսներ հանդես եկան [Սի](#) և [Java](#) լեզուները, ու, քանի որ այդ ժամանակ Java-ն համարվում էր խիստ ժամանակակից [ժամանակակից բան](#), 1995 թ. դեկտեմբերի 4-ին LiveScript լեզուն վերանվանեցին JavaScript, ստանալով արտոնագիր [Sun](#)-ից: JavaScript անունը Netscape-ի և Sun -ի համար հանդիսանում է [բետա-տարբերակի](#) (Netscape Navigator) երկրորդ թողարկումը:

[1996 թվականին](#) [Microsoft](#) ընկերությունը թողարկեց JavaScript լեզուն, [JScript](#) անվանումով: Այն առաջադրվել էր 1996 թվականի հուլիսի 18-ին: Առաջին զննարկիչը, որ աշխատեց այս ծրագրի հետ, [Internet Explorer](#) 3.0 էր:

Netscape -ը ներկայացված է որպես [ECMA](#) ասոցիացիայի լեզու: Ստանդարտ տարբերակն ունի [ECMAScript](#) անվանումը, նկարագրվում է [ECMA-262](#) ստանդարտով: Առաջին տարբերակը հանդիսացավ JavaScript 1.1 տարբերակը, ինչպես նաև JScript և [ScriptEasy](#):

Ինչ կարող է անել JavaScript-ը

[[խմբագրել](#) | [խմբագրել կոդը](#)]

- JavaScript-ը [HTML](#) դիզայներին տալիս է ծրագրավորման գործիք
- JavaScript-ը կարող է դինամիկ տեքստ ներդնել HTML էջի մեջ
- JavaScript-ը կարող է արձագանքել իրավիճակներին(events), այսինքն՝ JavaScript -ը կարող է գործել, երբ ինչ որ բան է պատահում, օրինակ երբ էջը ավարտում է բեռնավորումը, կամ երբ օգտագործողը մատնահարում է HTML էլեմենտի վրա
- JavaScript-ը կարող է կարդալ և փոխել HTML էլեմենտի պարունակությունը
- JavaScript-ը կարող է օգտագործվել տվյալը վավերացնելու համար մինչ վերջինիս սպասարկիչ ուղարկվելը: Սա կարող է սպասարկչին զերծ պահել ավելորդ գործընթացներից և ծանրաբեռնվածությունից
- JavaScript-ը կարող է օգտագործվել կայքից օգտվողի զննարկիչը ճանաչելու համար, և կախված զննարկիչից՝ բեռնել տվյալ բրաուզերին սպեցիֆիկ էջը

- JavaScript-ը կարող է օգտագործվել [cookie](#)-ներ ստեղծելու համար

JavaScript-ի կառուցվածքը կարելի է բաժանել հետևյալ 3 մասերի

1. [ECMAScript](#)
2. բրաուզերի օբյեկտավորված մոդել
3. դոկումենտի օբյեկտավորված մոդել

JavaScript-ի դյուրըմբռնելիությունը կայանում է նրա օբյեկտների կառուցվածքի մեջ:

JavaScript-ը բավականին նման է [C](#) ծրագրավորման լեզվին: JavaScript-ի օբյեկտները պարունակում են՝

1. մեթոդներ (ֆունկցիաներ)
2. հատկանիշներ՝ (տվյալներ կամ այլ օբյեկտներ) նկարագրում է օբյեկտների հատկանշական գծերը
3. իրադարձություններ ([անգլ.](#)՝ events), որոնք փոխկապակցված են օբյեկտների հետ
4. [AngularJS](#) - JavaScript- շրջանակ՝ բաց կոդով:

2.3.2.DOM-ը (Document Object Model)

1. Ինչ է DOM-ը

DOM-ը (Document Object Model) ներկայացնում է HTML (կամ XML) փաստաթղթի կառուցվածքը որպես **ընտրելի հանգույցների ծառ** (tree of nodes):

- Յուրաքանչյուր HTML տարր (թեգ) DOM-ում ներկայացված է որպես **HTMLElement node**:
- Տեքստը, որը վկայակոչվում է տարրերի մեջ, ներկայացվում է որպես **Text node**:
- DOM-ը մի միջանկյալ շերտ է, որը թույլ է տալիս JavaScript-ին՝ աշխատել էջի հետ՝ առանց HTML-ի ուղղակի փոփոխության:

2. DOM-ի հիմնական տարրերը (Nodes types)

Տարրերակ	Օրինակ	Նկարագրություն
Element Node	<div>, <p>, <a>	HTML տարրերը (թեգերը)
Text Node	Տեքստը տարրի մեջ	HTML-ի տեքստային բովանդակությունը
Attribute Node	class="button"	Տարրերի հատկանիշները (attributes)
Comment Node	<!-- մեկնաբանություն -->	HTML-ի մեկնաբանություններ

3. DOM ծառի կառուցվածք

Օրինակ՝

```
1. <body>
2.   <h1 id="title">Բարև աշխարհ</h1>
3.   <p>Սա օրինակ է</p>
4. </body>
```

DOM-ը կլինի ծառ, որտեղ

- body — արմատային տարրը, ունի երկու երեխա՝
 - h1 (և ներսում՝ տեքստային հանգույց՝ "Բարև աշխարհ")
 - p (և ներսում՝ տեքստային հանգույց՝ "Սա օրինակ է")

4. DOM-ի հետ աշխատելու հիմնական մեթոդները

Տարր գտնել

- document.getElementById('id') — գտնում է տարր id-ի միջոցով:
- document.getElementsByClassName('className') — գտնում է բոլոր տարրերը, որոնք ունեն տվյալ դասը:
- document.getElementsByTagName('tagName') — գտնում է բոլոր տվյալ տեգի տարրերը:

- `document.querySelector(selector)` — գտնում է առաջին տարրը, որը համապատասխանում է CSS սելեկտորին:
 - `document.querySelectorAll(selector)` — գտնում է բոլոր տարրերը, որոնք համապատասխանում են CSS սելեկտորին:
-

Տեքստը փոփոխել

- `element.textContent = 'Նոր տեքստ';` — փոխում է տարրի ներքին տեքստը (text only):
 - `element.innerHTML = '<b>Հաջորդ տեքստ</b>'` — փոխում է ներքին HTML բովանդակությունը (կարող է ավելացնել նոր տարրեր):
-

Ավելացնել նոր տարրեր

- `document.createElement('tagName')` — ստեղծում է նոր տարր, որը դեռ ոչ մի տեղ չէ տեղադրված:
 - `parent.appendChild(child)` — parent տարրի վերջում ավելացնում է child տարրը:
 - `parent.insertBefore(newNode, existingNode)` — ավելացնում է նոր տարր՝ որոշակի տեղում:
-

Հեռացնել տարր

- `parent.removeChild(child)` — ջնջում է child տարրը parent-ից:
 - Եթե ունեն ուղիղ տարրը՝ `element.remove()`-ով կա կարելի է ջնջել:
-

Attributes (հատկանիշներ) կարդալ և փոփոխել

- `element.getAttribute('attributeName')` — կարդում է հատկանիշի արժեքը:
 - `element.setAttribute('attributeName', 'value')` — ավելացնում կամ փոփոխում է հատկանիշը:
 - Կարող ես օգտագործել նաև կարճ ձևեր՝ `element.id`, `element.className`:
-

5. Տարրերակներ DOM հետ աշխատելու համար

- Մեծամասամբ աշխատում են այդ մեթոդների հետ՝ ստանում տարր, ապա փոխում, ավելացնում կամ ջնջում:
- Երբեմն աշխատում են նաև իրադարձությունների (events) հետ, որ էջը դառնա ինտերակտիվ:

6. DOM հետ աշխատելու պարզ օրինակ՝ ամբողջ գործընթաց

```

1. <!DOCTYPE html>
2. <html lang="hy">
3. <head>
4.   <meta charset="UTF-8" />
5.   <title>DOM Օրինակ</title>
6. </head>
7. <body>
8.   <h1 id="title">Յին վերնագիր</h1>
9.   <button id="changeBtn">Փոխել վերնագիրը</button>
10.
11.   <script>
12.     const btn = document.getElementById('changeBtn');
13.     const title = document.getElementById('title');
14.
15.     btn.addEventListener('click', () => {
16.       // Փոխում ենք տեքստը
17.       title.textContent = 'Նոր վերնագիր՝ DOM-ի միջոցով!';
18.
19.       // Ավելացնում ենք նոր պարբերություն
20.       const p = document.createElement('p');
21.       p.textContent = 'Այս պարբերությունը ավելացվել է կոճակի սեղմման ժամանակ:';
22.       document.body.appendChild(p);
23.     });
24.   </script>
25. </body>
26. </html>

```

2.3.3.JS իրադարձություններ (Events) և դրանց կառավարում

1. Ի՞նչ են իրադարձությունները (Events)

Իրադարձությունը (event) նշանակում է օգտատիրոջ կամ բրաուզերի կողմից կատարված գործողություն, օրինակ՝

- սեղմել կոճակը (click),
- մուտքագրել տեքստ (input),
- ընտրել էլեմենտ (change),
- ձևի ուղարկում (submit) և այլն:

JS-ում իրադարձությունները թույլ են տալիս հետևել այս գործողություններին և արձագանքել դրանց՝ կատարելով որոշակի ֆունկցիա (callback):

2. Ինլիներ (Inline) իրադարձությունների հայտարարություն

HTML-ում կարող ենք ուղիղ գրել՝ ինչ պետք է կատարվի, երբ իրադարձություն տեղի ունենա, օրինակ՝

```
1. <button onclick="alert('Կոճակը սեղմվեց')">Սեղմիր ինձ</button>
```

Սակայն այս մոտեցումը չի համարվում լավագույն պրակտիկա, քանի որ խառնվում է վեբեջի և կոդի լոգիկան:

3. JavaScript-ի իրադարձությունների կառավարում՝ addEventListener

Առավել հաճախ օգտագործվող և առաջարկվող մեթոդը՝

```
1. element.addEventListener('eventName', functionToRun);  
2.
```

Օրինակ՝

```
1. const btn = document.querySelector('button');  
2. btn.addEventListener('click', () => {  
3.   alert('Կոճակը սեղմվեց');  
4. });
```

4. Հիմնական իրադարձությունների տեսակներ

Իրադարձություն	Որ դեպքերում է օգտագործվում
----------------	-----------------------------

click	Օգտատերը սեղմում է որևէ տարր
input	Օգտատերը տեքստ է մուտքագրում input կամ textarea դաշտում
change	Էլեմենտի արժեքը փոխվում է (select, input)
submit	Form-ը ուղարկվում է
focus	Էլեմենտը կենտրոնացված է (օրինակ input)
blur	Էլեմենտը կորցնում է կենտրոնացումը

5. addEventListener մեթոդի առանձնահատկություններ

- Կարող ես մի տարրին մի քանի անգամ ավելացնել տարբեր իրադարձություններ կամ նույն իրադարձության բազմակի հսկողություններ:
- Տարբերակ է՝ կարող ես իրադարձությանը նաև լրացուցիչ պարամետրեր տալ (օրինակ, bubbling, capturing):

6. Իրադարձությունների պրոպագացիա (Event Propagation)

Իրադարձությունները տրավերստ են DOM ծառում երկու հիմնական ուղղություններով.

- **Capturing phase** (ներխուժում՝ վերնից ներքևին)
- **Bubbling phase** (փչող՝ ներքևից վերևին)

Օրինակ, եթե սեղմում ես մի button, իրադարձությունը սկսվում է դոկի արմատից (document) անցնում է տարրերից ներքև (capturing), ապա՝ հակառակ ուղղությամբ վերև (bubbling):

7. Իրադարձությունների դելեգացիա (Event Delegation)

Դելեգացիան տեխնիկա է, երբ իրադարձությունները ավելացնում ես ոչ թե անմիջապես նպատակակետ տարրին, այլ նրա ծնող տարրին, և դրանց վրա՝ ըստ նպատակակետի տարբերակների (event.target) անում էիր համապատասխան գործողություն:

Օրինակ, եթե ունես մեծ ցանկ՝

```
1. <ul id="list">
2.   <li>Առաջին</li>
3.   <li>Երկրորդ</li>
4.   <li>Երրորդ</li>
5. </ul>
6.
```

Դու կարող ես իրադարձություն ավելացնել **գրանցմանը** (սև տարրին), և ստուգել, թե որ li-ն սեղմվեց՝

```
1. const list = document.getElementById('list');
2. list.addEventListener('click', (event) => {
3.   if(event.target && event.target.nodeName === 'LI') {
4.     alert('Սեղմված է՝ ' + event.target.textContent);
5.   }
6. });
```

Դա արդյունավետ է, երբ տարրերը դինամիկ են ավելացվում կամ շատ են:

8. Օրինակ՝ input և change իրադարձություններ

```
1. <input type="text" id="nameInput" placeholder="Գրիր անունը" />
2. <p id="output"></p>
3. <script>
4.   const input = document.getElementById('nameInput');
5.   const output = document.getElementById('output');
6.   // input - երբ փոփոխում ես տեքստը՝ իմանում եմք փոփոխության պահին
7.   input.addEventListener('input', () => {
8.     output.textContent = 'Ղու գրում ես՝ ' + input.value;
9.   });
10.
11.  // change - երբ ֆոկուսն կորցնում է դաշտը կամ սեղմում ես Enter
12.  input.addEventListener('change', () => {
13.    output.textContent = 'Փոփոխվել է, վերջնական արժեք՝ ' + input.value;
14.  });
15. </script>
```

2.3.4. JavaScript Հիմնական Թեմաներ

1. Փոփոխականներ (Variables), **let**, **const**, **var**

- **var** — հին ձև, ունի ֆունկցիոնալ սքոպ, կարող է փոփոխվել, ունի hoisting (բարձրացում):
- **let** — բլոկային սքոպ, կարող է փոփոխվել:
- **const** — բլոկային սքոպ, չփոխվող արժեք (բայց եթե օբյեկտ է, օբյեկտի սեփականությունները կարող են փոփոխվել):

```
1. let age = 25;  
2. const name = "David";  
3. var isActive = true;
```

2. Տվյալների Տիպեր

- **String** — տողեր, օրինակ՝ "Hello"
 - **Number** — թվեր, օրինակ՝ 100, 3.14
 - **Boolean** — ճշմարիտ կամ կեղծ, true / false
 - **Array** — զանգվածներ, օրինակ՝ [1, 2, 3]
 - **Object** — օբյեկտներ, օրինակ՝ {name: "David", age: 25}
 - **null, undefined** — հատուկ արժեքներ
-

3. Ֆունկցիաներ (Functions)

- Սովորական ֆունկցիաներ`

```
1. function greet(name) {  
2.   return `Hello, ${name}!`;  
3. }  
4. console.log(greet('Dav'));  
5.
```

- Arrow ֆունկցիաներ (յամդա ֆունկցիաներ)`

```
1. const greet = (name) => `Hello, ${name}!`;  
2. console.log(greet('Dav'));
```

4. Մասիվներ (Arrays) և Օբյեկտներ (Objects)

- Ստեղծում, մուտք, փոփոխում

```
1. const arr = [1, 2, 3];  
2. arr.push(4); // ավելացնել տարր  
3. arr.pop(); // հանել վերջինը  
4.  
5. const obj = {name: "Dav", age: 25};  
6. obj.age = 26; // փոփոխել  
7. obj.city = "Yerevan"; // ավելացնել նոր հատկություն  
8.
```

- Հիմնական մեթոդներ`

```
1. arr.map(x => x * 2);  
2. arr.filter(x => x > 2);  
3. arr.reduce((sum, x) => sum + x, 0);  
4.
```

5. Պայմաններ և ցիկլեր

- if / else, switch

```
1. if(age > 18) {  
2.   console.log("Մեծահասակ");  
3. } else {  
4.   console.log("Մնչափահաս");  
5. }  
6.  
7. switch(day) {  
8.   case 1: console.log("Երկուշաբթի"); break;  
9.   default: console.log("Այլ օր");  
10. }
```

- ցիկլեր`

```
1. for(let i = 0; i < arr.length; i++) { ... }
2. while(condition) { ... }
3. arr.forEach(item => console.log(item));
```

6. Ասինքրոն JS, Promise-ներ, async/await

Promise — սա օբյեկտ է, որը ներկայացնում է ապագա արժեք, որը կիրականանա (fulfilled) կամ մերժվի (rejected)՝ ասինքրոն գործողությունից հետո:

Այլ խոսքով՝ Promise-ը թույլ է տալիս աշխատել այն արդյունքի հետ, որը դեռ չկա, բայց որը կստացվի ապագայում (օրինակ՝ տվյալներ բեռնել սերվերից կամ սպասել ժամանակի անցնելուն):

Promise-ի երեք վիճակներ

1. Pending (կատարման մեջ է) — Promise-ը դեռ ավարտված չէ, սպասում ենք արդյունքին:
2. Fulfilled (կատարված է) — Promise-ը հաջողությամբ ավարտվել է և ունի արժեք:
3. Rejected (մերժված է) — Promise-ը ավարտվել է սխալով:

- Promise օրինակ

```
1. const promise = new Promise((resolve, reject) => {
2.   setTimeout(() => resolve('Դիմումը հաջողվեց'), 1000);
3. });
4. promise.then(result => console.log(result));
```

- async/await

```
1. async function fetchData() {
2.   const result = await promise;
3.   console.log(result);
4. }
5. fetchData();
```

7. JavaScript և LocalStorage, SessionStorage

- Տվյալների պահպանում

```
1. localStorage.setItem('name', 'Dav');
2. const name = localStorage.getItem('name');
3. sessionStorage.setItem('token', 'abc123');
4.
```

8. Form Validation — ձևերի ստուգում JS-ով

```
1. const form = document.getElementById('myForm');
2. form.addEventListener('submit', function(e) {
3.   const email = form.email.value;
4.   if(!email.includes('@')) {
5.     e.preventDefault(); // չհղորդել ձևը
6.     alert('Սխալ էլ. հասցե');
7.   }
8. });
9.
```

9. ES6 Մոդուլներ (Modules)

- export և import

```
1. // math.js
2. export const sum = (a, b) => a + b;
3.
4. // main.js
5. import { sum } from './math.js';
6. console.log(sum(2,3));
7.
```

10. JS ֆրեյմվորքեր / լայբրերներ

- React, Vue, Angular՝ օգտագործվում են front-end նախագծերում, UI կառուցելու համար: Եթե հետաքրքրում է՝ կարող եք խորանալ:

Գլուխ 3 Back-End

Back-End-ը (հայերեն՝ բեք-ենդ) ծրագրավորման մեջ նշանակում է այն մասի զարգացումը, որը թաքնված է տեսանելի ինտերֆեյսից՝ այսինքն օգտագործողը չի տեսնում: Դա վեբկայքի կամ հավելվածի «ներսի» կողմն է՝ որտեղ իրականացվում է տվյալների մշակում, պահում և լոգիկա:

Բեք-ենդը զբաղվում է՝

1. **Սերվերային ծրագրավորմամբ** – հարցումների ընդունում և պատասխանի վերադարձրում:
 2. **Տվյալների բազայի հետ աշխատանքով** – օրինակ՝ MySQL, PostgreSQL, MongoDB:
 3. **API-ների ստեղծմամբ** – Front-End-ը (օգտագործողի մաս) կապվում է Back-End-ի հետ API-ների միջոցով:
 4. **Բիզնես տրամաբանությամբ** – օրինակ՝ օգտվողի գրանցում, մուտք, պատվերի մշակման ընթացք և այլն:
-

Օգտագործվող տեխնոլոգիաներ

- **Ծրագրավորման լեզուներ՝**
 - JavaScript (Node.js),
 - PHP,
 - Python (Django, Flask),
 - Java,
 - Ruby,
 - Go և այլն:
- **Տվյալների բազաներ՝**
 - MySQL,

- PostgreSQL,
- MongoDB,
- SQLite և այլն:

Օրինակ

Երբ օգտվողը կայքում լրացնում է մուտքի ձևը և սեղմում "Մուտք գործել"

- **Front-End-ը** ուղարկում է տվյալները (էլ. հասցե, գաղտնաբառ) դեպի **Back-End**:
- **Back-End-ը** ստուգում է տվյալները սերվերում՝ համեմատելով տվյալների բազայի հետ:
- Եթե ամեն ինչ ճիշտ է՝ վերադառնում է հաջող մուտքի պատասխանը, օրինակ՝ token:

3.1. Օգտագործողների կառավարում PHP-ով (User Authentication & Authorization)

Այս համակարգը վեբ հավելվածների հիմնական բաղադրիչներից է, որը ապահովում է.

1. Գրանցում (Registration)

- Օգտատերերը մուտքագրում են իրենց տվյալները (անուն, էլ. փոստ, գաղտնաբառ)
- Համակարգը ստուգում է տվյալների վավերականությունը
- Գաղտնաբառը հեշավորվում է password_hash() ֆունկցիայով
- Օգտատերերի տվյալները պահպանվում են տվյալների բազայում

// Գրանցման օրինակ

```
1. $hashed_password = password_hash($password, PASSWORD_DEFAULT);
2. $stmt = $pdo->prepare("INSERT INTO users (username, email, password) VALUES (?, ?, ?)");
3. $stmt->execute([$username, $email, $hashed_password]);
```

2. Մուտք (Login)

- Օգտատերը մուտքագրում է մուտքանուն/էլ. փոստ և գաղտնաբառ
- Համակարգը ստուգում է տվյալները տվյալների բազայի հետ
- Գաղտնաբառի համեմատումը կատարվում է password_verify() ֆունկցիայով
- Հաջող մուտքի դեպքում ստեղծվում է սեսիա

// Մուտքի օրինակ

```
1. $stmt = $pdo->prepare("SELECT * FROM users WHERE username = ?");
2. $stmt->execute([$username]);
3. $user = $stmt->fetch();
4.
5. if ($user && password_verify($password, $user['password'])) {
6.     $_SESSION['user_id'] = $user['id'];
7.     $_SESSION['username'] = $user['username'];
8.     $_SESSION['role'] = $user['role'];
9. }
```

3. Ելք (Logout)

- Սեսիայի տվյալների մաքրում
- Սեսիայի ավարտ

```
1. // Ելքի օրինակ
2. session_unset();
3. session_destroy();
4. header('Location: login.php');
5. exit();
6.
```

4. Session-ների կառավարում

- Սեսիաները պահպանում են օգտատիրոջ վիճակը բազմաթիվ էջերի միջև
- Սկսվում է session_start() ֆունկցիայով յուրաքանչյուր էջի սկզբում
- Տվյալները պահվում են \$_SESSION գլոբալ զանգվածում

5. Password հեշավորում

- password_hash() - ստեղծում է անվտանգ հեշ
- password_verify() - ստուգում է գաղտնաբառի համապատասխանությունը հեշին
- Անվտանգության համար երբեք չպահպանեք գաղտնաբառերը պարզ տեքստով

6. Role-based Access Control

- Տարբերակում է օգտատերերի իրավունքները (օր. user vs admin)
- Կարող է իրականացվել տվյալների բազայում role սյունակով
- Ստուգվում է մուտքի ժամանակ և պահպանվում սեսիայում

```
1. // Կարգավիճակի ստուգման օրինակ
2. function isAdmin() {
3.     return isset($_SESSION['role']) && $_SESSION['role'] === 'admin';
4. }
```

```

1. // Պաշտպանված էջի օրինակ
2. if (!isset($_SESSION['user_id'])) {
3.     header('Location: login.php');
4.     exit();
5. }
6.
7. if (!isAdmin()) {
8.     die('Մուտքը արգելված է');
9. }
10.

```

Այս համակարգը հնարավորություն է տալիս.

- Անվտանգ կառավարել օգտատերերի հաշիվները
- Սահմանել տարբեր մակարդակի մուտք
- Պաշտպանել հավելվածի ռեսուրսները

3.2. CRUD գործողություններ PHP-ում (Տեսական հատված)

1. CRUD-ի հայեցակարգը

CRUD-ը (Create, Read, Update, Delete) տվյալների բազայի հետ աշխատելու չորս հիմնական գործողությունն է, որն օգտագործվում է վեբ-ծրագրավորման մեջ: Այն հանդիսանում է ցանկացած դինամիկ վեբ կայքի հիմքը, օրինակ՝

- **Բլոգային համակարգեր** (գրառումների ստեղծում, խմբագրում, դիտում, ջնջում)
- **Էլեկտրոնային առևտրի կայքեր** (ապրանքների կատալոգի կառավարում)
- **Օգտատերերի կառավարման համակարգեր**

2. CRUD-ի հիմնական գործողությունները

Գործողություն	Նկարագրություն	SQL հրաման	PHP մեթոդ
Create	Տվյալների ավելացում	INSERT INTO	\$pdo->prepare(), \$stmt->execute()
Read	Տվյալների կարդալը	SELECT	fetch(), fetchAll()

Գործողություն	Նկարագրություն	SQL հրաման	PHP մեթոդ
Update	Տվյալների խմբագրում	UPDATE	\$pdo->prepare(), \$stmt->execute()
Delete	Տվյալների ջնջում	DELETE	\$pdo->prepare(), \$stmt->execute()

3. MySQL/MariaDB-ի հետ աշխատանք (PDO)

PHP-ում տվյալների բազայի հետ աշխատելու համար առավել անվտանգ է **PDO (PHP Data Objects)**-ի կամ **MySQLi**-ի օգտագործումը: PDO-ն աջակցում է բազմաթիվ տվյալների բազաների և ապահովում է պարամետրացված հարցումներ՝ SQL ինժեկցիաներից պաշտպանվելու համար:

PDO կապի ստեղծում

```

1. $host = 'localhost';
2. $dbname = 'database_name';
3. $user = 'username';
4. $pass = 'password';
5.
6. try {
7.     $pdo = new PDO("mysql:host=$host;dbname=$dbname", $user, $pass);
8.     $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
9. } catch (PDOException $e) {
10.     die("Տվյալների բազայի կապը ձախողվեց: " . $e->getMessage());
11. }

```

4. CRUD-ի իրականացում (Բլոգի գրառումների օրինակով)

Աղյուսակի ստեղծում (posts)

```

1. CREATE TABLE posts (
2.     id INT AUTO_INCREMENT PRIMARY KEY,
3.     title VARCHAR(255) NOT NULL,
4.     content TEXT NOT NULL,
5.     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
6. );

```

1. Create (Ստեղծել)

```
1. $title = "Բլոգի վերնագիր";
2. $content = "Բլոգի բովանդակություն...";
3. $stmt = $pdo->prepare("INSERT INTO posts (title, content) VALUES (?, ?)");
4. $stmt->execute([$title, $content]);
5. echo "Գրառումը հաջողությամբ ավելացվեց!";
```

2. Read (Կարդալ)

Մեկ գրառման ստացում

```
1. $stmt = $pdo->prepare("SELECT * FROM posts WHERE id = ?");
2. $stmt->execute([1]);
3. $post = $stmt->fetch(PDO::FETCH_ASSOC);
4. echo "<h1>{$post['title']}</h1>";
5. echo "<p>{$post['content']}</p>";
```

Բոլոր գրառումների ստացում

```
1. $stmt = $pdo->query("SELECT * FROM posts");
2. $posts = $stmt->fetchAll(PDO::FETCH_ASSOC);
3. foreach ($posts as $post) {
4.     echo "<h2>{$post['title']}</h2>";
5.     echo "<p>{$post['content']}</p>";
6. }
```

3. Update (Թարմացնել)

```
$newTitle = "Նոր վերնագիր";
$newContent = "Թարմացված բովանդակություն...";
$postId = 1;
$stmt = $pdo->prepare("UPDATE posts SET title = ?, content = ? WHERE id = ?");
$stmt->execute([$newTitle, $newContent, $postId]);
echo "Գրառումը թարմացվեց!";
```

4. Delete (Ջնջել)

```
1. $postId = 1;
2. $stmt = $pdo->prepare("DELETE FROM posts WHERE id = ?");
3. $stmt->execute([$postId]);
4. echo "Գրառումը ջնջվեց!";
```

5. Անվտանգություն CRUD-ում

- Պարամետրացված հարցումներ (Prepared Statements) – Կանխում է SQL ինժեկցիաները:
- Տվյալների վավերացում – Ստուգել, որ օգտատերը մուտքագրել է ճիշտ տվյալներ:
- CSRF պաշտպանություն – Օգտագործել token-ներ ձևաթղթերում:

```
1. // Պարամետրացված հարցման օրինակ
2. $stmt = $pdo->prepare("SELECT * FROM users WHERE username = ?");
3. $stmt->execute([$username]);
```

6. Եզրափակում

CRUD գործողությունները հանդիսանում են ցանկացած վեբ-հավելվածի հիմքը:

Օգտագործելով PDO կամ MySQLi, կարելի է ապահով կերպով կատարել տվյալների բազայի գործողություններ՝ առանց անվտանգության ռիսկերի: Բլոգային համակարգերում, ապրանքների կատալոգներում և օգտատերերի կառավարման համակարգերում CRUD-ի ճիշտ իրականացումը կարող է ապահովել կայքի կայուն աշխատանքը:

3.3. REST API Մշակում PHP-ով

REST API-ի Հիմնական Սկզբունքներ

REST (Representational State Transfer) API-ն հանդիսանում է վեբ ծառայությունների ճարտարապետական ոճ, որն օգտագործում է HTTP պրոտոկոլի հնարավորությունները տվյալների փոխանակման համար:

Հիմնական Բնութագրերը

- **Client-Server արքիտեկտուրա** - առանձնացնում է օգտագործողի ինտերֆեյսը և տվյալների պահեստավորումը
- **Անպետքական վիճակ (Stateless)** - յուրաքանչյուր հարցում պարունակում է բոլոր անհրաժեշտ տեղեկությունները
- **Քեշավորում** - բարելավում է արտադրողականությունը
- **Միասնական ինտերֆեյս** - պարզեցված և ստանդարտացված փոխգործակցություն

JSON-ով Տվյալների Փոխանակում

JSON (JavaScript Object Notation) հանդիսանում է տվյալների փոխանակման թեթև ֆորմատ:

PHP-ում JSON-ի մշակում

```

1. // Տվյալների կոդավորում JSON-ի
2. $data = [
3.     'id' => 101,
4.     'name' => 'Արամ Արամյան',
5.     'email' => 'aram@example.com'
6. ];
7.
8. header('Content-Type: application/json');
9. echo json_encode($data);
10.
11. // JSON-ի ապակոդավորում
12. $json = '{"id":101,"name":"Արամ Արամյան","email":"aram@example.com"}';
13. $data = json_decode($json, true);
14.

```

JWT (JSON Web Tokens) Վավերացում

JWT-ն օգտագործվում է օգտատերերի վավերացման համար և բաղկացած է 3 մասից՝

1. Header (ալգորիթմ և տոկենի տեսակ)
2. Payload (տվյալներ)
3. Signature (ստորագրություն)

JWT Իրականացում PHP-ում

```

1. use Firebase\JWT\JWT;
2. use Firebase\JWT\Key;
3.
4. // Գաղտնի բանալի
5. $secret_key = 'your_secret_key';
6.
7. // Տոկենի ստեղծում
8. $payload = [
9.     'user_id' => 123,
10.    'username' => 'aram',
11.    'exp' => time() + 3600 // Տոկենի ժամկետը (1 ժամ)
12. ];
13.
14. $jwt = JWT::encode($payload, $secret_key, 'HS256');
15.
16. // Տոկենի վավերացում
17. try {
18.     $decoded = JWT::decode($jwt, new Key($secret_key, 'HS256'));
19.     print_r($decoded);
20. } catch (Exception $e) {
21.     http_response_code(401);
22.     echo json_encode(['message' => 'Վավերացումը ձախողվեց']);
23. }
24.

```

Fetch API/AJAX-ի Միջոցով Front-end-ի հետ Կապ

Fetch API Օրինակ (JavaScript)

```
1. // GET հարցում
2. fetch('api/posts.php')
3.   .then(response => response.json())
4.   .then(data => console.log(data))
5.   .catch(error => console.error('Միսալ:', error));
6.
7. // POST հարցում JWT տոկենով
8. fetch('api/create_post.php', {
9.   method: 'POST',
10.  headers: {
11.    'Content-Type': 'application/json',
12.    'Authorization': 'Bearer ' + token
13.  },
14.  body: JSON.stringify({
15.    title: 'Նոր գրառում',
16.    content: 'Բովանդակություն...'
17.  })
18. })
19. .then(response => response.json())
20. .then(data => console.log(data));
21.
```

PHP Back-end-ի Օրինակ

```
1. // posts.php - GET հարցումների մշակում
2. header("Access-Control-Allow-Origin: *");
3. header("Content-Type: application/json; charset=UTF-8");
4.
5. require_once 'config.php';
6.
7. // Ստուգում JWT տոկենը
8. $headers = getallheaders();
9. if (!isset($headers['Authorization'])) {
10.   http_response_code(401);
11.   echo json_encode(['message' => 'Չի տրամադրվել տոկեն']);
12.   exit;
13. }
14.
15. $token = str_replace('Bearer ', '', $headers['Authorization']);
16.
17. try {
18.   $decoded = JWT::decode($token, new Key($secret_key, 'HS256'));
19.
20.   // Վերադարձնել բոլոր գրառումները
21.   $stmt = $pdo->query("SELECT * FROM posts");
22.   $posts = $stmt->fetchAll(PDO::FETCH_ASSOC);
23.
24.   http_response_code(200);
25.   echo json_encode($posts);
26. } catch (Exception $e) {
27.   http_response_code(401);
28.   echo json_encode(['message' => 'Անվավեր տոկեն']);
29. }
30.
```

REST API Վերջնակետերի Օրինակ

GET /api/posts - Բոլոր գրառումների ստացում

GET /api/posts/{id} - Մեկ գրառման ստացում

POST /api/posts - Նոր գրառման ստեղծում

PUT /api/posts/{id} - Գրառման թարմացում

DELETE /api/posts/{id} - Գրառման ջնջում

POST /api/auth/register - Գրանցում

POST /api/auth/login - Մուտք

Անվտանգության Միջոցառումներ

1. **HTTPS** - Բոլոր հաղորդակցությունը պետք է լինի գաղտնագրված
2. **JWT ժամկետ** - Տոկենները պետք է ունենան սահմանափակ ժամկետ
3. **CORS կառավարում** - Սահմանել թույլատրելի դոմեններ
4. **Մուտքի սահմանափակում** - Կիրառել rate limiting

// CORS կարգավորումներ

```
1. header("Access-Control-Allow-Origin: https://yourdomain.com");  
2. header("Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS");  
3. header("Access-Control-Allow-Headers: Content-Type, Authorization");
```

Այս մոտեցումը թույլ է տալիս ստեղծել ապահով, մասշտաբային և պահպանելի REST API լուծումներ՝ օգտագործելով PHP, JWT և ժամանակակից Front-end տեխնոլոգիաներ:

Գլուխ 4 Database (MySQL)

Database (տվյալների բազա) էլեկտրոնային համակարգ է, որը նախագծված է տվյալների կազմակերպված պահպանման, կառավարման և արագ մուտքի համար: Այն թույլ է տալիս.

- Կառուցվածքային կերպով պահել մեծ քանակությամբ տվյալներ
- Արագ որոնում և վերականգնում կատարել
- Ապահովել տվյալների անվտանգություն և ամբողջականություն
- Հնարավորություն տալ բազմաթիվ օգտատերերի միաժամանակյա աշխատանքին

MySQL-ի մասին

MySQL-ը աշխարհում ամենատարածված օպեն սորս ռելյացիոն տվյալների բազաներից մեկն է (RDBMS):

Հիմնական բնութագրերը

1. **Ռելյացիոն մոդել** - տվյալները պահվում են աղյուսակների (tables) տեսքով, որոնք կապված են միմյանց հետ
2. **Օպեն սորս** - անվճար օգտագործման համար (GPL լիցենզիայի ներքո)
3. **Բարձր արտադրողականություն** - արագ աշխատանք նույնիսկ մեծ ծավալի տվյալների դեպքում
4. **Հուսալիություն** - ապահովում է տվյալների ամբողջականություն
5. **Հեշտ օգտագործում** - համեմատաբար պարզ է սովորել և օգտագործել

Հիմնական հասկացություններ

- **Աղյուսակներ (Tables)** - տվյալների հիմնական կառուցվածքային միավորը
- **Սյուններ (Columns)** - աղյուսակի առանձին դաշտեր

- **Տողեր (Rows)** - աղյուսակի առանձին գրառումներ
- **Հիմնական բանալի (Primary Key)** - յուրահատուկ նույնականացուցիչ աղյուսակի յուրաքանչյուր տողի համար
- **Արտաքին բանալի (Foreign Key)** - մի աղյուսակի և մյուսի միջև կապ հաստատող դաշտ

Օրինակ SQL հարցումներ MySQL-ում

```
-- Աղյուսակի ստեղծում
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Տվյալների ավելացում
INSERT INTO users (username, email) VALUES ('john_doe', 'john@example.com');

-- Տվյալների ընթերցում
SELECT * FROM users WHERE username = 'john_doe';

-- Տվյալների թարմացում
UPDATE users SET email = 'new_email@example.com' WHERE id = 1;

-- Տվյալների հեռացում
DELETE FROM users WHERE id = 1;
```

MySQL-ը լայնորեն օգտագործվում է վեբ հավելվածներում և հանդիսանում է այնպիսի հայտնի համակարգերի բաղադրիչ, ինչպիսիք են WordPress, Drupal, Joomla և շատ ուրիշներ: Այն հատկապես հարմար է փոքր և միջին մասշտաբի ծրագրերի համար:

MySQL (արտ. [maɪˌɛskjuːˈɛl]), բաց կոդով տվյալների բազաների կառավարման համակարգ: MySQL-ի մշակումն ու աջակցումն իրականացվում է [Օրաքլ](#) կորպորացիայի կողմից: Վերջինս MySQL-ի նկատմամբ իրավունքները ստացել է [Sun Microsystems](#) ընկերության գնումից հետո, որն էլ, նախքան այդ, իր հերթին գնել էր [MySQL AB](#) ընկերությունը: MySQL-ն տարածվում է [GNU GPL](#), ինչպես նաև սեփական կոմերցիոն արտոնագրի ներքո, չնայած մշակողներն նոր ֆունկցիաներ են ավելացնում արտոնագրված օգտագործողների պատվերներով:

MySQL-ը հիմնականում օգտագործվում է փոքր և միջին ծանրաբեռնվածության ծրագրերի համար: Այն ներառված է մի շարք վեբ-ծրագրավորման համար նախատեսված ծրագրերի փաթեթների մեջ, ինչպիսիք են [WAMP](#)-ը, [AppServ](#)-ը, [LAMP](#)-ը, [XAMPP](#)-ը և [Open Server](#)-ը:

MySQL բավականին ճկուն է: Այն աջակցում է աղյուսակների այնպիսի տեսակներ, ինչպիսիք են [MyISAM](#)-ը կամ [InnoDB](#)-ն: Բացի այդ, այն տեղադրվում է հատուկ **EXAMPLE** անվանմամբ աղյուսակի տեսակով, որն ցուցադրում է նոր աղյուսակի տիպեր ստեղծելու սկզբունքները: GPL արտոնագրի շնորհիվ, MySQL-ն անընդհատ հարստանում է նոր աղյուսակի տեսակներով:

2008 թ. փետրվարի 26-ին [Sun Microsystems](#) ընկերությունը գնեց [MySQL AB](#)-ն \$1 մլրդ.-ով, իսկ 2010 թ. հունվարի 27-ին [Sun Microsystems](#)-ը գնվեց [Օրաքլ](#) ընկերության կողմից, որն զբաղվում է տվյալների բազաների կառավարման համակարգերի ստեղծմամբ ու զարգացմամբ:

Գործնական

Գլուխ 1 SEO

1.1. Կայքի Հիմնաքառեր

1.2. Մեթա տեգերի օպտիմալացում (Meta Tags Optimization)

Մեթա տեգերի օպտիմալացումը (Meta Tags Optimization) կարևոր քայլ է SEO-ի համար, քանի որ այն ազդում է կայքի տեսանելիության վրա որոնման համակարգերում: Մեթա տեգերը տեղեկություններ են տրամադրում Google-ին (և այլ որոնիչներին) այն մասին, թե ինչ է ներկայացնում մեր էջը: Դրանք պետք է լինեն հստակ, գրավիչ և ներառեն հիմնաքառեր, որոնք լավագույնս արտացոլում են մեր կայքի բովանդակությունը:

Եվ քանի որ մեր բրենդը **AVERIST** է, որը մասնագիտանում է հագուստի վաճառքում (մայկաներ, շապիկներ և այլն), այսպես կարող են օպտիմալացնել մեթա տեգերը՝ հաշվի առնելով այս տեղեկությունները:

1. Meta Title (Վերնագիր)

Meta Title-ը էջի վերնագիրն է, որը երևում է որոնման արդյունքներում: Այն պետք է լինի կարճ (մաքսիմալ 60 նիշ) և պետք է ներառի հիմնական հիմնաքառերը:

Օրինակներ՝

- **Meta Title 1:** "AVERIST: Fashionable T-Shirts & Tank Tops | Online Store in Armenia"
- **Meta Title 2:** "Shop Stylish T-Shirts Online | AVERIST - Affordable Clothing in Armenia"
- **Meta Title 3:** "Buy Trendy T-Shirts & Tank Tops Online | AVERIST Clothing Store"

2. Meta Description (Նկարագրություն)

Meta Description-ը հակիրճ նկարագրություն է, որը երևում է էջի տակ որոնման արդյունքներում: Այն պետք է գրավիչ լինի, որպեսզի մարդիկ ցանկանան մտնել կայք, և պետք է ներառի հիմնաբառեր: Ստուգենք, որ նկարագրությունը չի գերազանցում 160 նիշ:

Օրինակներ՝

- **Meta Description 1:** "Shop the latest stylish t-shirts and tank tops from AVERIST. High-quality, affordable clothing for men and women. Fast shipping in Armenia."
- **Meta Description 2:** "Find the best t-shirts and tank tops online at AVERIST. Affordable fashion with trendy designs. Explore our collection now!"
- **Meta Description 3:** "AVERIST offers a wide range of high-quality t-shirts and tank tops. Perfect for every occasion. Shop now for free delivery in Armenia."

Խորհուրդ: Նկարագրությունը պետք է ներառի գործնական հիմնաբառեր, ինչպիսիք են "buy t-shirts", "fashion", "affordable", "shipping":

3. Meta Keywords (Մեթա բանալիներ) — Google-ը չի օգտագործում այս տեղը SEO-ի համար, բայց կարող է օգտակար լինել որոշ ուրիշ որոնիչների համար:

Ներդնենք թարմ հիմնաբառեր, որոնք կապված են մեր կայքի ապրանքների հետ:

Օրինակ՝

- **Meta Keywords:** "AVERIST t-shirts, affordable tank tops, buy fashion online, stylish clothing Armenia, trendy t-shirts, online clothing store"

4. Open Graph Tags (Open Graph Tagging) — սոցիալական մեդիայի համար:

Open Graph-ը կարևոր է սոցիալական մեդիայի կիսելու համար, որովհետև այն օգնում է կարգավորել այն, ինչ երևում է սոցիալական կայքերում, օրինակ՝ Facebook-ում կամ Twitter-ում: Դրանք պետք է ներառեն մեզ հատուկ հիմնաբառեր և նկարագրություններ:

Օրինակ՝

- **og:title:** "Shop Fashionable T-Shirts & Tank Tops Online | AVERIST"

- **og:description:** "Shop for stylish and affordable t-shirts and tank tops at AVERIST. Perfect for every occasion. Free shipping within Armenia."
- **og:image:** (Նկարագրությունն ապրանքի լուսանկարի URL, որին վերաբերում է վերնագիրը)

5. Twitter Card Tags (Twitter-ի համար)

Twitter Card-ը թարմացնում է այն, ինչ մարդիկ տեսնում են, երբ կիսում են մեր կայքը Twitter-ում: Դա օգնում է գրավելու այցելուներին Twitter-ում:

Օրինակ՝

- **twitter:title:** "Shop Stylish T-Shirts and Tank Tops | AVERIST"
- **twitter:description:** "Find trendy t-shirts and tank tops at AVERIST. Affordable, stylish clothing with fast shipping in Armenia."
- **twitter:image:** (Նկարագրությունն ապրանքի լուսանկարի URL, որը կցված կլինի Twitter-ի հղմանը)

6. Canonical Tags

Canonical tag-ը ցույց է տալիս, թե որ էջը պետք է համարվի հիմնականը, եթե կան մի քանի էջեր նույն բովանդակությամբ: Այսպես օգնում ես որոնիչներին հասկանալ, որ նույն բովանդակության տարբեր էջերը պետք է համարվեն մեկ, և դա կարող է օգնել SEO-ին:

1. <code><link rel="canonical" href="https://www.averist.com/t-shirts"></code>
--

7. Image Alt Text (Լուսանկարների Alt տեքստ)

Լուսանկարների համար պետք է ավելացնել Alt Text, որը բեռնում է այն պատկերները, եթե ինչ-որ բան սխալ է լինում, և այն նաև կարևոր է SEO-ի համար: Անվտանգ է, երբ ներառում ես հիմնաբառեր լուսանկարում:

Օրինակ՝

- **Alt Text:** "Fashionable t-shirts for men and women at AVERIST"
- **Alt Text:** "High-quality tank tops online | AVERIST Armenia"

8. URL Structure (Հասցեի կառուցվածք)

Մենք պետք է նաև ուշադրություն դարձնենք մեր կայքի URL հասցեներին: Քանի որ դրանք կարող են ազդել որոնման արդյունքների վրա, նրանք պետք է լինեն կարճ և ներառեն հիմնաբառեր:

Օրինակ՝

- **URL:** <https://www.averist.com/t-shirts>
- **URL:** <https://www.averist.com/shop/affordable-tank-tops>

9. Schema Markup (Միսեմա նշումներ)

Միսեմա նշումները հնարավորություն են տալիս որոնիչներին հասկանալ տվյալների կոնտեքստը՝ օրինակ՝ ապրանքների, սպառողների կամ առցանց խանութների մասին: Ապրանքների սխեմաների նշումները կարող են պարունակել արժեքներ, նկարագրություններ և գներ:

```

"@context": "https://schema.org",
"@type": "Product",
"name": "Fashionable T-Shirt",
"image": "https://www.averist.com/images/tshirt.jpg",
"description": "Shop the latest stylish t-shirts at AVERIST. Affordable
and high-quality clothing.",
"brand": "AVERIST",
"offers": {
  "@type": "Offer",
  "priceCurrency": "AMD",
  "price": "3500",
  "availability": "https://schema.org/InStock"
}
}

```

1.3. Մրցակիցների SEO վերլուծություն

Մրցակիցների SEO վերլուծությունը հանդիսանում է կարևոր գործիք, որը թույլ է տալիս հասկանալ, թե ինչպես են գործարկվել SEO ռազմավարությունները այլ կայքերում, ինչ հիմնաբառեր են նրանք օգտագործում, և ինչպիսին է նրանց կայքի բովանդակությունը: Այս ուսումնասիրությունը օգնում է ստեղծել ավելի մրցունակ ռազմավարություն և բարելավել կայքի վարկանիշը որոնիչներում:

1.3.1 Մրցակից 1 SEO վերլուծություն – AZAT MARD

1. Ընդհանուր Տեղեկություններ

AZAT MARD-ը հանդիսանում է հագուստի ոլորտում ակտիվ brend, որը հատկապես կենտրոնանում է թրենդային և ժամանակակից հագուստի գույների վրա: Նրանք փորձում են ներառել հաճախ օգտագործվող հիմնաբառերը և ճիշտ օպտիմալիզել իրենց կայքի բովանդակությունը, որպեսզի որոնիչներում բարձր դիրք ստանան:

2. Հիմնաբառերի (Keywords) Օգտագործում

- **Հիմնաբառերի ընտրություն:** AZAT MARD-ը ընտրում է այն հիմնաբառերը, որոնք կապ են ունենում իրենց արտադրանքի բնույթին, օրինակ՝ "fashionable t-shirts", "modern clothing", "affordable style", "trendy outfits", և այլն:
 - Այս ուղով նրանք փորձում են ներկայացնել իրենց brend-ը որպես թրենդային, մատչելի և ժամանակակից:
- **Օգտագործման վայրեր:**
 - **Meta Title**-ում՝ նրանք ներառում են հիմնաբառեր, օրինակ՝ "AZAT MARD | Fashionable T-Shirts & Modern Clothing"
 - **Meta Description**-ում՝ առաջարկում են հստակ նկարագրություն՝ "Discover trendy and affordable clothing at AZAT MARD. Shop our stylish collection of t-shirts, dresses, and more."
 - **Բովանդակության մեջ**՝ վերնագրերում, ենթավերնագրերում և հոդվածներում միշտ ընդգրկում են վերաբերող հիմնաբառեր:

3. Meta Tag-ների Օպտիմալացում

- **Meta Title:** AZAT MARD-ի Meta Title-ը արտահայտում է brend-ի հիմնական առաջարկը, օրինակ՝ "AZAT MARD | Trendy & Affordable Fashion for Modern Lifestyles":
 - Այս անունն ու ուղեգրում ընդգրկում են հիմնական հիմնաբառերը, որոնք կապ ունեն ինչպես արտադրանքի բնույթին, այնպես էլ target լսարանին:
- **Meta Description:** AZAT MARD-ի Meta Description-ը ընդգրկում է կարճ, գրավիչ և պարզ նկարագրություն՝ առաջարկում է brend-ի սկզբնական արժեքները, օրինակ՝ "Explore the latest collection of trendy t-shirts, dresses, and more at AZAT MARD. High-quality fashion for every modern individual."
 - Այստեղ էլ ակնկալվում է, որ գլխավոր հիմնաբառերը ներառված են, ինչը նպաստում է որոնիչների համար ավելի հեշտ ընկնելու նպատակին:

4. Բովանդակության Օպտիմալացում

- **Բովանդակության որակ:** AZAT MARD-ը ուշադիր է դարձնում իրենց կայքի բովանդակության որակը՝ գրելով մանրամասն, թարմ և օգտակար նյութեր, որոնք կենտրոնանում են ժամանակակից հազուստի թեմայի շուրջ:
 - Օրինակ՝ նրանք հնարավորություն են տալիս ընթերցողներին ստանալ տվյալներ ապրանքի նկարագրության, չափսերի, մատուցման հնարավորություն և հաճախակի կարդալի հոդվածներ հազուստի նոր թրենդերի մասին:
- **Վերնագրերի և ենթավերնագրերի կառուցվածք:** Օգտագործում են համապատասխան H1, H2, H3 թեգեր, որտեղ H1-ը գլխավոր վերնագիրը է, իսկ H2, H3-ը օգնում են կազմավորել մոդուլներ, որոնք ներկայացնում են brend-ի առաջարկները:

5. Գործնական SEO Գործիքների Օգտագործում

- **Google Analytics** և **Google Search Console:** AZAT MARD-ը հետևում է իրենց կայքի կատարմանը՝ ստուգում, թե որ հիմնական հիմնաբառերով գալիս են այցելուները, և ինչպես են կոնկրետ էջերը դիրքավորվում որոնիչների արդյունքներում:
- **SEMrush** կամ **Ahrefs:** Դրանք օգտագործվում են մրցակիցների հետ համեմատելու համար, գտնելու brend-ի շակաթները, ինչպես նաև վերահսկելու, թե որոնք հիմնական հիմնաբառերն են ամենաբարձիկները:

6. Եզրակացություններ

AZAT MARD-ի SEO ռազմավարությունն աջակցվում է կայքի բոլոր մակարդակներում՝

- Հիմնաբառերի խելամիտ ընտրությամբ, որոնք օգտագործվում են Meta Tag-ներում, վերնագրերում և բովանդակության մեջ:

- Բովանդակության օպտիմալացումով՝ տրամադրված է բարձրորակ, թարմ և օգտակար տեղեկատվություն:
- Գործնական SEO գործիքների օգտագործմամբ՝ կարգավորվում է brend-ի առցանց ներկայացում և արդյունքները, ինչը թույլ է տալիս վերելավել մրցակցային դիրքը:

Page SEO audit

Page analysis

URL	✓	26 characters — optimal. azatmardofficial.com/hy-am	Tips ▾
Canonical	✓	A canonical tag is set for this page and the link is working fine. https://azatmardofficial.com/hy-am	Tips ▾
Title	⚠	102 characters — not good. Try to shorten title to 70 characters. AZAT MARD - ժամանակակից տղամարդու հագուստ / Շքերություն Տղամարդկանց հագուստ - ԱՉԱՏ ՏՐԱՄԱՐԴ - Azat Mard	Tips ▾
Meta description	✓	194 characters — optimal. Շքերություն Տղամարդկանց հագուստ - AZAT MARD-ը տղամարդկանց հագուստի ժամանակակից բրենդ է, որը մասնագիտացած է շքեղ հագուստի և վերնազգեստի մեջ նորաձևության հեղինակ, հզոր ժամանակակից տղամարդու համար:	Tips ▾
Headings	✓	Good! Your page implements HTML headings (<H1> through <H3>) and only one <H1> heading is used. H1 (1), H2 (11), H3 (48), H4 (0), H5 (0), H6 (0) <H1>: View others ▾	Tips ▾
Images	🔊	32 images without ALT.	Tips ▾
Text/HTML ratio	🔊	26.98% — not bad. We suggest adding more text to your website.	Tips ▾
Frames	🔊	We see that your website has IFRAMEs. Usually this is not a problem for SEO if they are used properly.	Tips ▾
Flash	✓	Not detected	Tips ▾
Microformats	🔊	This page does not utilize Microformat markup. You can validate your markup with the Google Structured Data Testing Tool .	Tips ▾
Schema.org	✓	Great! We have found several schema.org types. You can validate your markup with the Google Structured Data Testing Tool . Organization, WebSite	Tips ▾
The Open Graph	✓	Great! We have found several Open Graph objects. You can validate your markup with the Facebook Object Debugger tool . website	Tips ▾
Twitter Card	✓	Nice! We have found Twitter Cards on your page. You can validate your cards with the Twitter Card validator tool . summary_large_image	Tips ▾

Mobile compliance

Meta viewport	✓	Good! This page has a typical viewport meta tag value compatible with most of mobile browsers. <code>width=device-width,initial-scale=1</code>	Tips ▼
---------------	---	---	--------

Site compliance

Robots.txt	✓	Good, it looks like your website has a robots.txt file. https://azatmardofficial.com/robots.txt	Tips ▼
XML Sitemaps	✓	Good! Your website contains an XML sitemap. https://azatmardofficial.com/sitemap.xml	Tips ▼
Language	✓	Great! You have specified your websites language. <code>hy</code>	Tips ▼
Doctype	✓	Great! You have specified the doctype. <code>HTML5</code>	Tips ▼
Encoding	✓	Declaring a character/language encoding will significantly strengthen your SEO. It also prevents complication when rendering page. <code>utf-8</code>	Tips ▼
Google™ Analytics	✓	Google™ Analytics is monitoring your website. This is a great tool and should be taking advantage of.	Tips ▼
Favicon	✓	It's good that you have a favicon image. https://azatmardofficial.com/cdn/shop/files/AM.jpg?crop=center&height=32&v=1613058910&width=32	Tips ▼

1.4.Robots.txt

Robots.txt ֆայլը այն պարզ տեքստային ֆայլն է, որը գտնվում է ձեր կայքի հիմնական մենյուի մեջ և նախատեսված է որոնողական համակարգերի (ինչպես Google, Bing և այլն) ռոբոտների կամ "ծանրաբերան" սկաներների համար: Ֆայլը հուշում է, թե որ էջերը կամ հատվածները որոնողական համակարգերը կարող են ինդեքսավորել, իսկ որոնք՝ ոչ:

Robots.txt-ի հիմնական նպատակներն են.

1. **Վերահսկել կայքի ինդեքսավորումը.** Կայքի բոլոր էջերը չեն պահանջում որոնողական համակարգերում հայտնվել: Օրինակ, անձնական կամ ադմինիստրատիվ էջեր:
2. **Կայքի ռեսուրսների խնայողություն.** Որոշ դեպքերում կայքի որոշ էջեր կամ պանակներ կարող են չափազանց ծանր լինել կամ անհրաժեշտ չեն ինդեքսավորման համար: Այդ դեպքում, robots.txt ֆայլով կարելի է խոչընդոտել դրանց սկանավորումը:
3. **Ձեր SEO ռազմավարության օպտիմալացում.** Robots.txt ֆայլի ճիշտ օգտագործումը կարող է օգնել որոնողական համակարգերի ռոբոտներին կենտրոնանալ այն էջերի վրա, որոնք կարևոր են SEO-ի համար:

Robots.txt-ի հիմնական կոդերը

- **User-agent** – Փորձարկվող ռոբոտի անունը (այնը, որ որոնողական համակարգը օգտագործում է էջերի սկանավորման համար):
- **Disallow** – Ինչ տարածքներ կամ էջեր պետք է արգելվեն սկանավորումից:
- **Allow** – Որոշ հատվածներ, որոնք սովորաբար արգելված են, բայց դուք ցանկանում եք թույլ տալ դրանք:
- **Sitemap** – Որպեսզի որոնողական համակարգերը ճիշտ գտնեն ձեր կայքի բոլոր կարևոր էջերը, կարող եք նշել նաև Sitemap-ի URL-ը:

Օրինակ Robots.txt համար AVERIST կայքի:

```
1. User-agent: *
2. Disallow: /admin/
3. Disallow: /login/
4. Disallow: /cart/
5. Disallow: /checkout/
6. Disallow: /private/
7. Allow: /
8.
9. Sitemap: https://www.averist.com/sitemap.xml
```

Բացատրություն:

- **User-agent: *** – Այսօրինակ կոդը նշանակում է, որ այն վերաբերում է բոլոր ռոբոտներին (Google, Bing և այլն):
- **Disallow: /admin/, /login/, /cart/, /checkout/, /private/** – Այս մասերում նշված URL-ները չեն կարող ինդեքսավորվել որոնողական համակարգերի կողմից:
- **Allow: /** – Այս կոդը թույլ է տալիս բոլոր մյուս էջերը:
- **Sitemap** – Գործընթացը ցույց է տալիս sitemap-ի դիրքորոշումը, ինչը օգնում է ռոբոտներին գտնել ձեր բոլոր ցանկալի էջերը՝ այդպիսով բարձրացնելով ձեր կայքի SEO-ի տեսանելիությունը:

Ինչպե՞ս օգտագործել Robots.txt ֆայլը:

1. **Պատրաստեք Robots.txt ֆայլը:** Մա սովորաբար պարզ տեքստային ֆայլ է՝ robots.txt անվանումով:
2. **Պահեք այն կայքի հիմնական պանակում** (օրինակ՝ <https://www.averist.com/robots.txt>):
3. **Կապեք այն Sitemap ֆայլին:** Դա օգնում է որոնողական համակարգերին ավելի արագ հասկանալ, թե որոնք են ձեր կայքի կարևոր էջերը:

Այսպիսով, **robots.txt** ֆայլի միջոցով դուք կարող եք ուղղորդել որոնողական համակարգերը, որպեսզի նրանք կենտրոնանան միայն այն էջերի վրա, որոնք կարիք ունեն ձեր SEO արդյունքների բարելավման համար, և խուսափեն բոլոր այն հատվածներից, որոնք ոչինչ չեն ավելացնում որոնման արդյունքներում:

1. 5. Sitemap.xml

Sitemap.xml ֆայլը XML ձևաչափով փաստաթուղթ է, որը պարունակում է ձեր կայքի բոլոր կարևոր էջերի ցանկը, որպեսզի որոնողական համակարգերը (օրինակ՝ Google, Bing և այլն) կարողանան հեշտությամբ գտնել և ինդեքսավորել ձեր կայքի ամբողջ բովանդակությունը: Sitemap-ի միջոցով դուք ապահովում եք, որ որոնողական համակարգերը գտնում են բոլոր էջերը, որոնք ուզում եք, որ լինեն տեսանելի որոնման արդյունքներում:

Sitemap.xml-ի հիմնական նպատակները:

1. **Ուշադիր ինդեքսավորում:** Մայթմափը օգնում է որոնողական համակարգերին հասկանալ՝ որ էջերը պետք է ինդեքսավորվեն և որոնք են կայքի կարևորագույն էջերը, ինչպիսիք են արտադրանքի էջերը, ծառայությունների մանրամասները և այլն:
2. **Կայքի կառուցվածքի ցույց:** Sitemap-ն ապահովում է որոնողական համակարգերի համար կայքի էջերի կառուցվածքը, ինչը օգնում է նրանց ավելի արագ գտնել և ինդեքսավորել նոր կամ թարմացված էջերը:
3. **Դասակարգում:** Եթե ձեր կայքում կա մեծ թվով էջեր կամ բովանդակություն, որոնք դժվար է գտնել, sitemap.xml-ն օգնում է կենտրոնացնել ինդեքսավորման գործընթացը՝ ապահովելով, որ ձեր կարևոր էջերը գտնվեն արագ:
4. **Sitemap-ում հայտնված բոլոր էջերը կարող են արագ ինդեքսավորվել:** Եթե նոր էջը ավելացրել եք ձեր կայքում, ու եթե այն գտնվում է Sitemap-ում, որոնողական համակարգը այն ավելի արագ կգտնի:

Sitemap.xml-ի կազմվածքը:

Sitemap.xml ֆայլը պետք է լինի **XML** ձևաչափով: Այս ֆայլում յուրաքանչյուր էջ ներկայացված է <url> տարրով, որտեղ ներկայացվում է այդ էջի URL-ը, վերջին անգամ փոփոխված ամսաթիվը (կամ տարբերակը) և էջի առաջնահերթությունը (priority):

Sitemap.xml-ի հիմնական տարրերը:

- **<loc>** – Մատնանշում է էջի URL-ը:
- **<lastmod>** – Սկսվում է այն ամսաթիվը, երբ վերջին անգամ էջը փոփոխվել է: Մաօգտակար է այն դեպքում, երբ դուք թարմացնում եք էջերը, և ցանկանում եք, որ որոնողական համակարգերը թարմացնեն այն:
- **<changefreq>** – Ներկայացնում է այն հաճախությունը, որով էջը հավանական է փոփոխություն ապրի: Օրինակ՝ եթե մի էջը հաճախ թարմացվում է, կարող եք նշել «daily» (օրական) կամ «weekly» (շաբաթական):
- **<priority>** – Արժեք, որը ցույց է տալիս էջի առաջնահերթությունը (0.0-ից 1.0): 1.0-ն նշանակում է առավելագույն առաջնահերթություն, իսկ 0.0-ն՝ նվազագույն:

Sitemap.xml-ի օրինակ (AVERIST կայքի համար):

xml

```

1. <?xml version="1.0" encoding="UTF-8"?>
2. <urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">
3.   <!-- Հիմնական էջ -->
4.   <url>
5.     <loc>https://www.averist.com/</loc>
6.     <lastmod>2025-04-07</lastmod>
7.     <changefreq>daily</changefreq>
8.     <priority>1.0</priority>
9.   </url>
10.
11.  <!-- Շապիկների էջ -->
12.  <url>
13.    <loc>https://www.averist.com/t-shirts</loc>
14.    <lastmod>2025-04-07</lastmod>
15.    <changefreq>weekly</changefreq>
16.    <priority>0.8</priority>
17.  </url>
18.
19.  <!-- Մայկաների (tank tops) էջ -->
20.  <url>
21.    <loc>https://www.averist.com/tank-tops</loc>
22.    <lastmod>2025-04-07</lastmod>
23.    <changefreq>weekly</changefreq>
24.    <priority>0.8</priority>
25.  </url>

```

```

26.
27. <!-- Ապրանքների էջ (Products) -->
28. <url>
29.   <loc>https://www.averist.com/products</loc>
30.   <lastmod>2025-04-07</lastmod>
31.   <changefreq>monthly</changefreq>
32.   <priority>0.7</priority>
33. </url>
34.
35. <!-- Կապի էջ -->
36. <url>
37.   <loc>https://www.averist.com/contact</loc>
38.   <lastmod>2025-04-07</lastmod>
39.   <changefreq>monthly</changefreq>
40.   <priority>0.5</priority>
41. </url>
42.
43. <!-- Այլ էջեր (օրինակ՝ քաղաքականություն) -->
44. <url>
45.   <loc>https://www.averist.com/privacy-policy</loc>
46.   <lastmod>2025-04-07</lastmod>
47.   <changefreq>yearly</changefreq>
48.   <priority>0.3</priority>
49. </url>
50. </urlset>
51.

```

Բացատրություն:

1. **<loc>**: Ապրանքի էջի, գլխավոր էջի կամ մյուս կարևոր էջերի URL-ներ:
2. **<lastmod>**: Սա ցույց է տալիս, երբ տվյալ էջը վերջին անգամ փոփոխվել է: Դա կարևոր է այն դեպքերում, երբ կայքը մշտապես թարմանում է:
3. **<changefreq>**: Իհարկե, այս արժեքը պետք է փոխվի՝ կախված էջի թարմացման հաճախականությունից: Եթե էջը հաճախ չի թարմացվում (օրինակ՝ գլխավոր էջը), ապա նշեք "monthly" կամ "yearly":
4. **<priority>**: Այս արժեքը նշում է, թե որքան կարևոր է տվյալ էջը մյուսների համեմատ: Գլխավոր էջը սովորաբար ունի 1.0 առաջնահերթություն, իսկ երկրորդական կամ նվազ կարևոր էջերն ունեն ավելի ցածր առաջնահերթություն (օրինակ՝ 0.5 կամ 0.3):

Ինչպես օգտագործել Sitemap.xml-ը:

1. **Ստեղծեք ֆայլը:** Ստեղծեք XML ֆայլը՝ վերոհիշյալ ձևաչափով:
2. **Պահեք այն կայքի հիմնական պանակում:** Քո sitemap.xml ֆայլը պետք է տեղադրվի ձեր կայքի հիմնական փաթեթում՝ <https://www.averist.com/sitemap.xml>:
3. **Կապեք robots.txt ֆայլի հետ:** Robots.txt ֆայլում ավելացրեք հետևյալ տողը, որպեսզի որոնողական համակարգերը կարողանան գտնել ձեր sitemap-ը:

arduino

1. **Sitemap:** <https://www.averist.com/sitemap.xml>

Սա կօգնի որոնողական համակարգերին ավելի արագ գտնել և ինդեքսավորել ձեր կայքի բոլոր էջերը:

1. 6. Internal Linking

Internal linking-ը (նույն կայքում ներքին հղումները) շատ կարևոր է SEO-ի համար, քանի որ այն օգնում է կապել ձեր կայքի տարբեր էջերը, բարելավելով կայքի կառույցը և օգտագործողի փորձը: Ներքին հղումները նաև օգնում են որոնողական համակարգերին հասկանալ, թե որ էջերն են կարևոր և ինչպես պետք է ինդեքսավորվեն:

Ահա որոշ օրինակներ, թե ինչպես կարող ես ներքին հղումներ ավելացնել ձեր AVERIST կայքում:

Internal Linking օրինակներ AVERIST կայքի համար:

1. Գլխավոր էջ -> Հագուստի տեսականիներ:

- **Հագուստներ:** Ձեր կայքի գլխավոր էջում կարող եք ներքին հղումներ անել դեպի կտրոնների կամ կատեգորիաների էջեր, ինչպիսիք են **T-shirts, Tank tops** և այլն:

1. `<a href="https://www.averist.com/t-shirts">Բացահայտեք մեր նորագույն շապիկները</a>`

2. Ապրանքների էջ -> Կոնկրետ ապրանքի էջ:

- Ամեն մի ապրանքի նկարագրության մեջ կարող եք ներառել ներքին հղումներ, որոնք կտան օգտվողներին արագ հասանելիություն դեպի կոնկրետ ապրանք:

1. `<a href="https://www.averist.com/product/12345">Հասանելի է մեր նորագույն 100% բամբուկե շապիկը</a>`

3. Կապի էջ -> Կայքի հիմնական էջեր:

- Կապի էջում (contact page) կարող եք ներառել հղումներ դեպի այլ կարևոր էջեր, օրինակ՝ «Մեր մասին» կամ «Պայմանները և գաղտնիությունը» էջեր:

1. Գտեք ավելին մեր մասին այստեղ
2.

4. Հաճախակի հարցեր (FAQ) -> Կայքի կարևոր բաժիններ:

- FAQ էջում կարող եք ներառել հղումներ դեպի կայքի հիմնական բաժինները, որոնք կօգնեն օգտվողներին գտնել ավելին, օրինակ՝ վերադասավորված առաքումներ, վճարման մեթոդներ:

1. Նաև կարող եք կարդալ մեր առաքման պայմանները

5. Ակցիաներ և հատուկ առաջարկներ -> Ապրանքների էջեր:

- Օրինակ, երբ դուք ունենաք հատուկ գեղչ կամ հատուկ առաջարկներ, ապա կարող եք ներքին հղումներ անել այդ էջերին:

1. Մեր հատուկ գեղչերը հիմա՝ 30% գեղչ!
2.

6. Blog -> Հազուստի համար խորհուրդներ:

- Եթե ունեք բլոգ, որտեղ ներկայացնում եք նոր հազուստի որոշ տենդենցներ կամ խորհուրդներ, կարող եք ներքին հղումներ անել դեպի համապատասխան արտադրանքի էջեր:

1. Այս գարունին ընտրեք մեր նոր T-shirts նորածնություն
2.

Ներքին հղումների օգտակարություն SEO-ի համար:

- **Ապրանքի տեսանելիություն:** Ներքին հղումները օգնում են օգտվողներին գտնել ձեր արտադրանքը կամ ծառայությունները: Սա բարձրացնում է դրանց տեսանելիությունը և կարող է բարելավել կոնվերտացիաները (վաճառքները):
- **Նպաստում է կայքի կառուցվածքին:** Որպեսզի որոնողական համակարգերը կարողանան լավ հասկանալ ձեր կայքի կառուցվածքը, պետք է ճիշտ և խելացի ներքին հղումներ լինեն:
- **Քաջի քաջիտում:** Ներքին հղումները օգնում են «SEO քաջը» փոխանցել ավելի կարևոր էջերին: Եթե ինչ-որ էջ հաճախ է հղում ուրիշ էջերի, դա որոնողական համակարգերին կօգնի ճանաչել այն որպես ավելի կարևոր:

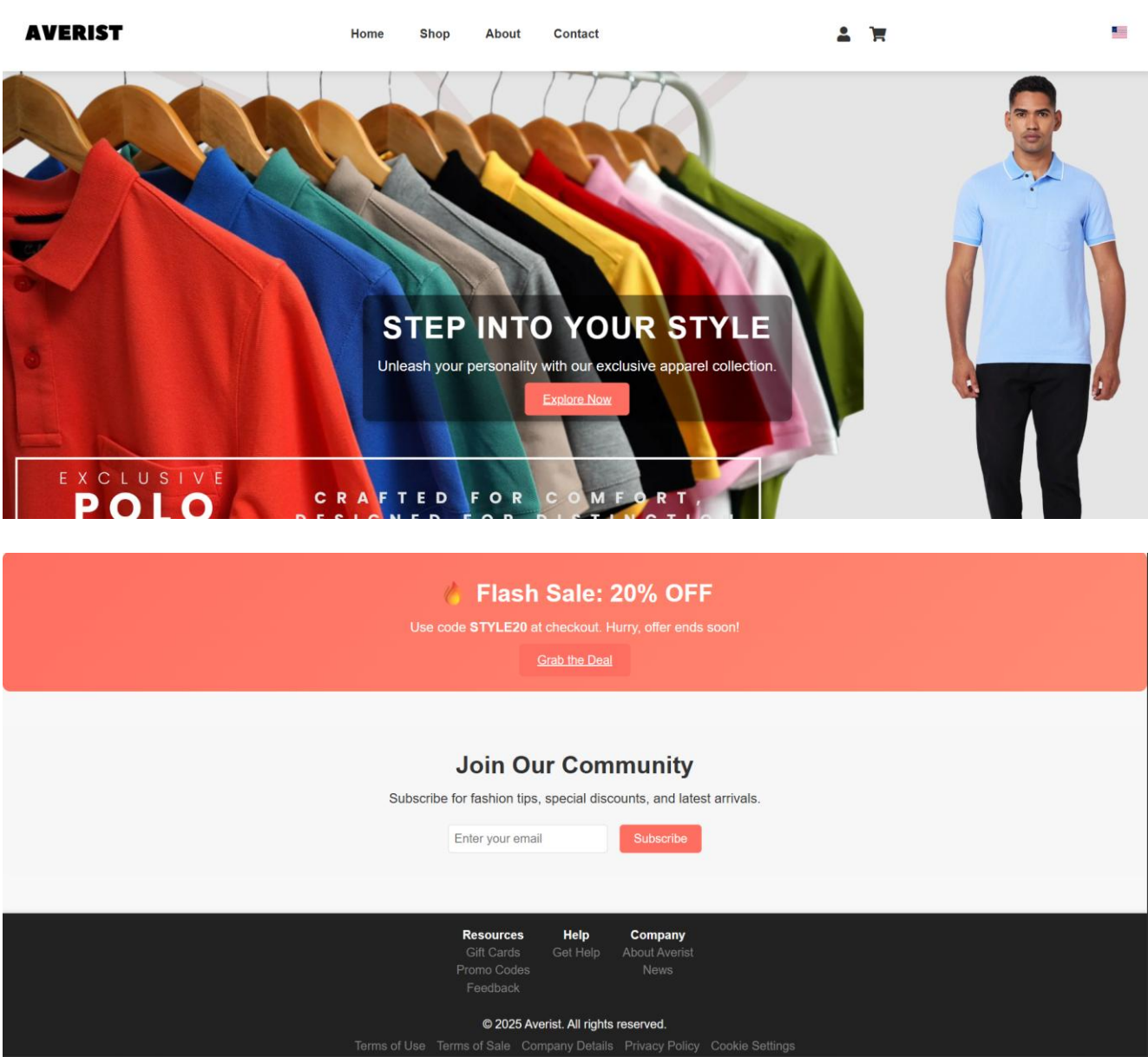
- **Օգտագործողի փորձ:** Ներքին հղումները բարելավում են օգտվողների կայքում շրջայցի փորձը, ինչը կարող է օգնել ավելի երկար ժամանակ մնալ կայքում:

Խորհուրդներ:

- Որպեսզի ներքին հղումները արդյունավետ լինեն, պետք է դրանք տեղադրել բնական ձևով՝ առանց չափազանցնելու:
- Չափավոր օգտագործեք հղումները, բայց դրանք պետք է լինեն գործնական և հեշտ գտանելի:
- Դրանք պետք է գրավիչ լինեն և տեղադրված լինեն այնպիսի տեղերում, որոնք օգտատերերը կհետևեն, ինչպես արտադրանքի էջերի ներքևում կամ էջերի մեջտեղում:

Ներքին հղումների ճիշտ կիրառումը բարելավում է կայքի SEO-ն և օգտատերերի փորձը, ինչը անհրաժեշտ է ձեր AVERIST կայքի հաջողության համար:

Quiz 2 Front-End–



Home

Shop

About

Contact

All Products

T-Shirts

Hoodies &
Sweatshirts

Accessories

Kids

t-shirt Collection

Newest First

Showing 1-2 of 5 products



Black Cotton T-Shirt

A premium cotton t-shirt with a classic black color

\$15.99



White Printed T-Shirt

Soft white t-shirt with stylish print

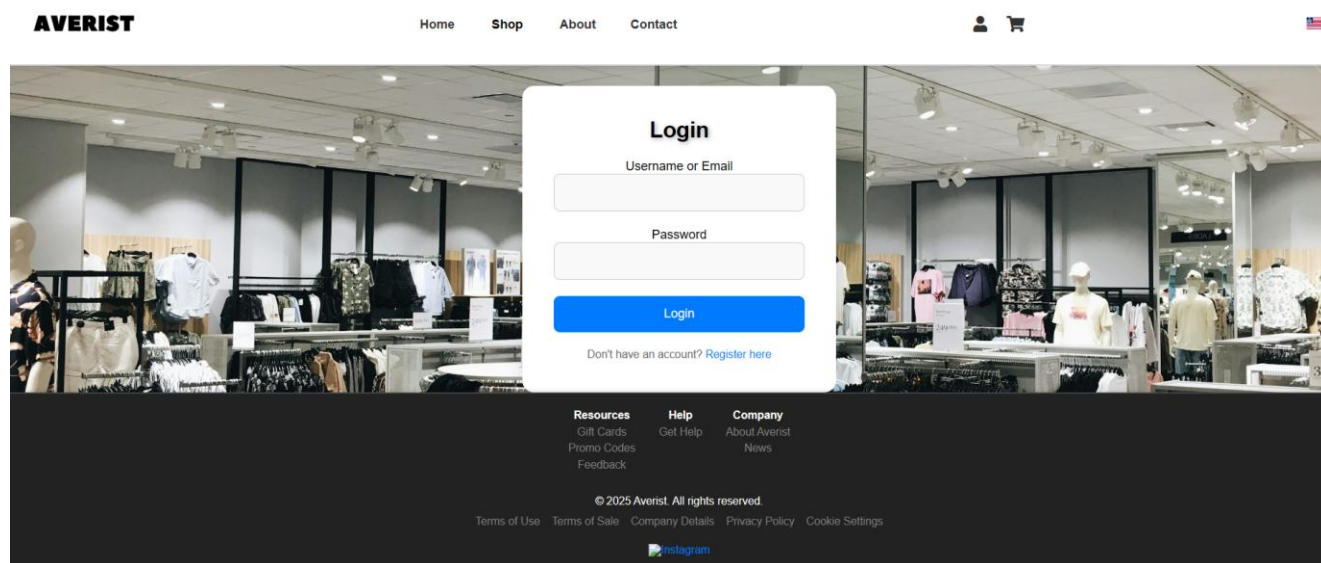
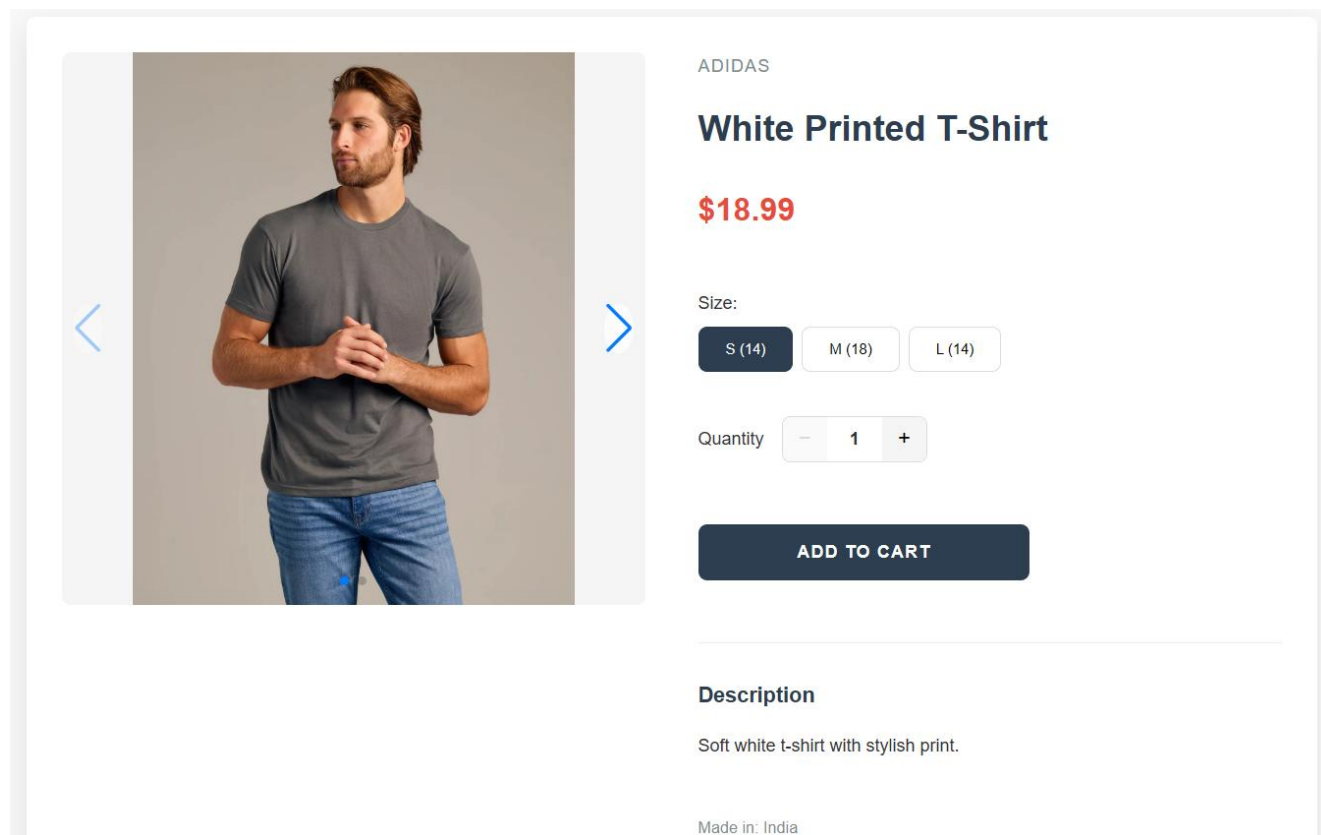
\$18.99

1

2

3

Next »



Register

Username

Email

Password

Confirm Password

Register

Already have an account? [Login here](#)

Welcome, Davit Hovhannisyan!

- Username: admin
- Email: dav.hovhannisyan.0102@gmail.com

Your Order History

Order #18

Apr 8, 2025

COMPLETED

White Printed T-Shirt

\$18.99

Size: S | Qty: 4

Black Cotton T-Shirt

\$15.99

Size: S | Qty: 3

Total: \$123.93

Payment: Card

Order #17

Apr 6, 2025

COMPLETED

White Printed T-Shirt

\$18.99

Size: L | Qty: 1

Total: \$18.99

Payment: Card

Order #16

Apr 6, 2025

COMPLETED

White Printed T-Shirt

\$18.99

Size: S | Qty: 1

Total: \$18.99

Payment: Card

Order #3

Apr 6, 2025

COMPLETED

Graphic Oversized T-Shirt

\$20.99

Size: M | Qty: 3

Total: \$62.97

Payment: Card

Order #2

Apr 6, 2025

COMPLETED

Black Cotton T-Shirt

\$15.99

Size: S | Qty: 25

Total: \$399.75

Payment: Cod

Order #2

Apr 6, 2025

COMPLETED

Black Cotton T-Shirt

\$15.99

Size: S | Qty: 25

Total: \$399.75

Payment: Cod

[→ Logout](#)

Our Story

Crafting exceptional shopping experiences since 2023

The Mini_Store Ethos

Founded in 2023, Mini_Store began as a passion project between friends who believed shopping should be more than just transactions. We envisioned a space where every purchase tells a story, where quality meets personality, and where customers become part of our growing family.

What started as a modest online boutique has blossomed into a curated marketplace known for its distinctive selection and exceptional service. At Mini_Store, we don't just sell products—we cultivate experiences.

Our Philosophy

"

To redefine retail by combining exceptional craftsmanship with heartfelt service, creating moments of delight in every package.

"

We meticulously source each item in our collection, prioritizing quality, sustainability, and design. Our team personally tests every product to ensure it meets our exacting standards before it earns a place in our store.

Why Choose Mini_Store



Curated Selection

Each item is handpicked by our team of experts, ensuring only the finest products make it to your doorstep.



Sustainable Practices

We partner with eco-conscious brands and use environmentally responsible packaging.



Personal Touch

Every order includes a handwritten note and carefully selected gift wrapping at no extra cost.

Meet Our Family

The heart of Mini_Store lies in our dedicated team who pour their passion into every aspect of our business.



Alexandra Chen

FOUNDER & CREATIVE
DIRECTOR

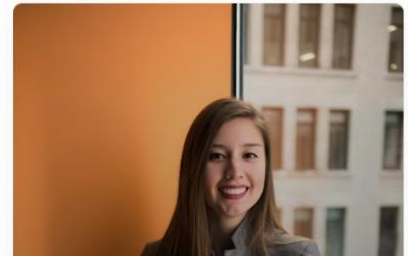
"Beauty lies in the details—that's what we bring to every product."



James Wilson

HEAD OF OPERATIONS

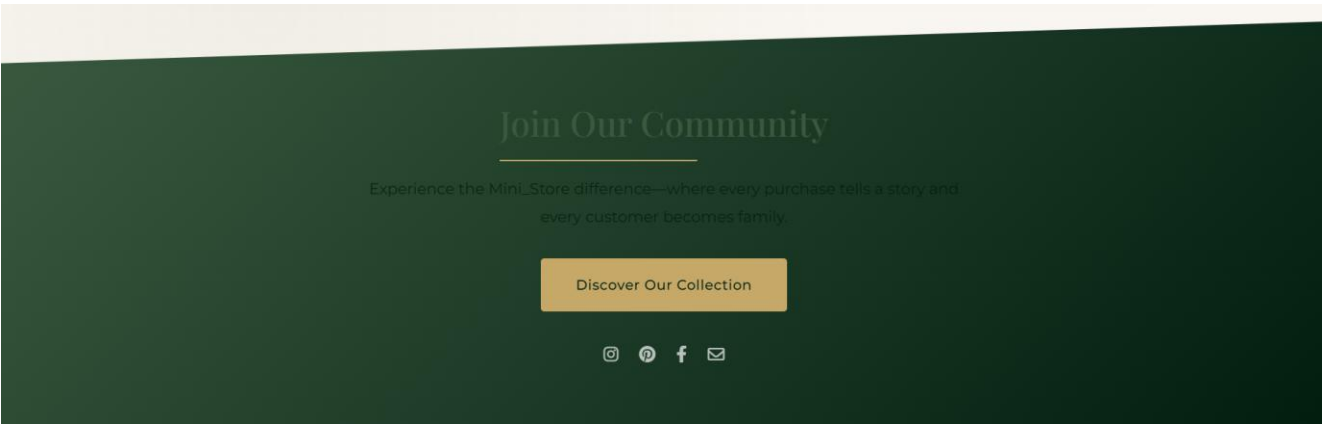
"Efficiency with elegance—that's our service promise."



Sophia Rodriguez

CUSTOMER EXPERIENCE

"Your satisfaction is our greatest achievement."



AVERIST

HomeShopAboutContact

Your Cart

Product	Quantity	Total	
 White Printed T-Shirt Size: S \$18.99	-1+	\$18.99	Remove
 Black Cotton T-Shirt Size: M \$15.99	-3+	\$47.97	Remove

Subtotal

\$66.96

Shipping & taxes calculated at checkout

Clear Cart

Checkout

Checkout

Shipping Information

Full Name:

Phone Number:

Address:

City:

Postal Code:

Country:

Payment Method

Payment Method:

Pay with Card (Stripe) 

Place Order

Order Summary

Product ID: 2 | Size: S | Quantity: 1 | Price: \$18.99

Product ID: 1 | Size: M | Quantity: 3 | Price: \$15.99

Total: \$66.96

←  TEST MODE

Pay

\$66.95

Product ID: 2 | Size: S

Qty 1

\$18.98

Product ID: 1 | Size: M

Qty 3

\$47.97

\$15.99 each

Pay with  link

Or

Email

email@example.com

Payment method

Card information

1234 1234 1234 1234



MM / YY

CVC



Cardholder name

Full name on card

Country or region

Armenia



Securely save my information for 1-click checkout

Pay faster on this site and everywhere Link is accepted.

Pay

 **Order placed successfully (Card)**

Thank you for your order!

Order Details

Order ID: 19

Name: DAVIT HOVHANNISYAN

Phone: 

Address: , ՎԱՅՈՑ ԶՈՐ, 0000, Armenia

Total: \$66.96

Payment Method: Credit Card

You will be redirected to the homepage in 5 seconds...

Գլուխ 3 Back-End

3.1.Login.php

```
1. <?php
2. session_start();
3.
4. // Database connection
5. include('../db.php'); // Include your database connection
6.
7. // Check if user is already logged in
8. if (isset($_SESSION['isAuth']) && $_SESSION['isAuth']) {
9.     header("Location: profile/profile.php"); // Redirect to profile if already
authenticated
10.     exit();
11. }
12.
13. $errors = [];
14. if ($_SERVER['REQUEST_METHOD'] === 'POST') {
15.     // Retrieve form data
16.     $input = $_POST['username']; // User input (could be username or email)
17.     $password = $_POST['password'];
18.
19.     // Check if the input is an email or username
20.     if (filter_var($input, FILTER_VALIDATE_EMAIL)) {
21.         // Input is an email
22.         $stmt = $conn->prepare("SELECT * FROM users WHERE email = ?");
23.     } else {
24.         // Input is a username
25.         $stmt = $conn->prepare("SELECT * FROM users WHERE username = ?");
26.     }
27.
28.     $stmt->bind_param("s", $input); // Bind parameter for either email or username
29.     $stmt->execute();
30.     $result = $stmt->get_result();
31.
32.     if ($result->num_rows > 0) {
33.         // User exists, check password
34.         $user = $result->fetch_assoc();
35.         if (password_verify($password, $user['password'])) { // Verify hashed
password
36.             $_SESSION['isAuth'] = true;
37.             $_SESSION['user_id'] = $user['id']; // Store user_id in session
38.             $_SESSION['username'] = $user['username']; // Store username in session
39.             $_SESSION['email'] = $user['email']; // Store email in session for future
use
40.             header("Location: ../profile/profile.php"); // Redirect to profile page on
successful login
41.             exit();
42.         } else {
43.             $errors[] = 'Invalid password';
44.         }
45.     } else {
```

```

46.         $errors[] = 'Invalid username or email';
47.     }
48. }
49. ?>
50.
51. <!DOCTYPE html>
52. <html lang="en">
53. <head>
54.     <meta charset="UTF-8">
55.     <meta name="viewport" content="width=device-width, initial-scale=1.0">
56.     <title>Login - Mini Store</title>
57.     <link rel="stylesheet" href="login.css">
58.     <link rel="stylesheet" href="../header.css">
59.     <link rel="stylesheet" href="../footer.css">
60. </head>
61. <body>
62.     <?php include '../header.php'; ?>
63.     <div class="login-form">
64.         <h2>Login</h2>
65.
66.         <!-- Display errors -->
67.         <?php if (!empty($errors)): ?>
68.             <div class="errors">
69.                 <?php foreach ($errors as $error): ?>
70.                     <p><?= $error ?></p>
71.                 <?php endforeach; ?>
72.             </div>
73.         <?php endif; ?>
74.
75.         <form action="login.php" method="POST">
76.             <label for="username">Username or Email</label>
77.             <input type="text" id="username" name="username" required>
78.
79.             <label for="password">Password</label>
80.             <input type="password" id="password" name="password" required>
81.
82.             <button type="submit">Login</button>
83.         </form>
84.
85.         <p>Don't have an account? <a href="register.php">Register here</a></p>
86.     </div>
87.     <?php include '../footer.php'; ?>
88. </body>
89. </html>
90.

```

3.2. Logout.php

```
<?php
session_start();
session_destroy();
header("Location: ../auth/login.php");
exit();
?>
```

3.3. Register.php

```
1. <?php
2. session_start();
3. include('../db.php');
4.
5. $errors = [];
6.
7. if ($_SERVER['REQUEST_METHOD'] === 'POST') {
8.     $username = trim($_POST['username']);
9.     $email = trim($_POST['email']);
10.    $password = $_POST['password'];
11.    $confirmPassword = $_POST['confirm_password'];
12.
13.    // Validation
14.    if (empty($username) || empty($email) || empty($password) ||
empty($confirmPassword)) {
15.        $errors[] = 'All fields are required';
16.    } elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
17.        $errors[] = 'Invalid email format';
18.    } elseif ($password !== $confirmPassword) {
19.        $errors[] = 'Passwords do not match';
20.    } else {
21.        // Check if username or email is taken
22.        $stmt = $conn->prepare("SELECT id FROM users WHERE username = ? OR
email = ?");
23.        $stmt->bind_param("ss", $username, $email);
24.        $stmt->execute();
25.        $result = $stmt->get_result();
26.
27.        if ($result->num_rows > 0) {
28.            $errors[] = 'Username or email already taken';
29.        } else {
30.            // Hash password
31.            $hashedPassword = password_hash($password, PASSWORD_DEFAULT);
32.
33.            // Insert into database
```

```

34.         $stmt = $conn->prepare("INSERT INTO users (username, email,
password) VALUES (?, ?, ?)");
35.         $stmt->bind_param("sss", $username, $email, $hashedPassword);
36.         if ($stmt->execute()) {
37.             // Get the user_id of the newly inserted user
38.             $user_id = $stmt->insert_id; // This retrieves the last
inserted ID
39.
40.             // Set session variables
41.             $_SESSION['isAuth'] = true;
42.             $_SESSION['user_id'] = $user_id; // Store user_id in session
43.             $_SESSION['username'] = $username;
44.             $_SESSION['email'] = $email;
45.
46.             header("Location: /profile");
47.             exit();
48.         } else {
49.             $errors[] = 'Failed to register user';
50.         }
51.     }
52. }
53. }
54. ?>
55.
56. <!DOCTYPE html>
57. <html lang="en">
58. <head>
59.     <meta charset="UTF-8">
60.     <meta name="viewport" content="width=device-width, initial-scale=1.0">
61.     <title>Register - Mini Store</title>
62.     <link rel="stylesheet" href="register.css">
63. </head>
64. <body>
65.     <div class="register-form">
66.         <h2>Register</h2>
67.
68.         <!-- Display errors -->
69.         <?php if (!empty($errors)): ?>
70.             <div class="errors">
71.                 <?php foreach ($errors as $error): ?>
72.                     <p><?= $error ?></p>
73.                 <?php endforeach; ?>
74.             </div>
75.         <?php endif; ?>
76.
77.         <form action="register.php" method="POST">
78.             <label for="username">Username</label>
79.             <input type="text" id="username" name="username" required>
80.
81.             <label for="email">Email</label>

```

```

82.         <input type="email" id="email" name="email" required> <!-- Email
field -->
83.
84.         <label for="password">Password</label>
85.         <input type="password" id="password" name="password" required>
86.
87.         <label for="confirm_password">Confirm Password</label>
88.         <input type="password" id="confirm_password"
name="confirm_password" required>
89.
90.         <button type="submit">Register</button>
91.     </form>
92.
93.     <p>Already have an account? <a href="login.php">Login here</a></p>
94. </div>
95. </body>
96. </html>
97.

```

Գլուխ 4 Database (MySQL)

```

1. -- phpMyAdmin SQL Dump
2. -- version 5.2.1
3. -- https://www.phpmyadmin.net/
4. --
5. -- Host: 127.0.0.1
6. -- Generation Time: May 19, 2025 at 09:10 PM
7. -- Server version: 10.4.32-MariaDB
8. -- PHP Version: 8.2.12
9.
10. SET SQL_MODE = "NO_AUTO_VALUE_ON_ZERO";
11. START TRANSACTION;
12. SET time_zone = "+00:00";
13.
14. /*!40101 SET @OLD_CHARACTER_SET_CLIENT=@@CHARACTER_SET_CLIENT */;
15. /*!40101 SET @OLD_CHARACTER_SET_RESULTS=@@CHARACTER_SET_RESULTS */;
16. /*!40101 SET @OLD_COLLATION_CONNECTION=@@COLLATION_CONNECTION */;
17. /*!40101 SET NAMES utf8mb4 */;
18.
19. --
20. -- Database: `mini_store_db`
21. --
22.
23. -- -----
24.
25. --
26. -- Table structure for table `admins`
27. --
28.
29. CREATE TABLE `admins` (

```

```

30. `id` int(11) NOT NULL,
31. `user_id` int(11) NOT NULL,
32. `role` varchar(50) DEFAULT 'admin',
33. `created_at` timestamp NOT NULL DEFAULT current_timestamp(),
34. `updated_at` timestamp NOT NULL DEFAULT current_timestamp() ON UPDATE
current_timestamp()
35. ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
36.
37. --- -----
38.
39. --
40. -- Table structure for table `cart`
41. --
42.
43. CREATE TABLE `cart` (
44. `id` int(11) NOT NULL,
45. `user_id` int(11) NOT NULL,
46. `product_id` int(11) NOT NULL,
47. `quantity` int(11) NOT NULL DEFAULT 1,
48. `image` varchar(255) DEFAULT NULL,
49. `created_at` timestamp NOT NULL DEFAULT current_timestamp(),
50. `updated_at` timestamp NOT NULL DEFAULT current_timestamp() ON UPDATE
current_timestamp(),
51. `size` varchar(255) DEFAULT NULL
52. ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
53.
54. --- -----
55.
56. --
57. -- Table structure for table `orders`
58. --
59.
60. CREATE TABLE `orders` (
61. `id` int(11) NOT NULL,
62. `user_id` int(11) NOT NULL,
63. `full_name` varchar(255) DEFAULT NULL,
64. `address` text DEFAULT NULL,
65. `phone` varchar(20) DEFAULT NULL,
66. `total_price` decimal(10,2) NOT NULL,
67. `created_at` timestamp NOT NULL DEFAULT current_timestamp(),
68. `payment_method` varchar(50) NOT NULL,
69. `city` varchar(255) NOT NULL,
70. `postal_code` varchar(255) NOT NULL,
71. `country` varchar(255) NOT NULL,
72. `status` varchar(20) DEFAULT 'Completed'
73. ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
74.
100. --- -----
101.
102. --
103. -- Table structure for table `order_items`
104. --
105.
106. CREATE TABLE `order_items` (
107. `id` int(11) NOT NULL,
108. `order_id` int(11) NOT NULL,

```

```

109. `product_id` int(11) NOT NULL,
110. `size` varchar(10) DEFAULT NULL,
111. `quantity` int(11) DEFAULT NULL,
112. `price` decimal(10,2) DEFAULT NULL
113. ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
114.
115. --
116. -- Dumping data for table `order_items`
117. --
118.
119. INSERT INTO `order_items` (`id`, `order_id`, `product_id`, `size`, `quantity`,
`price`) VALUES
120. (1, 1, 1, 'M', 2, 15.99),
121. (2, 2, 1, 'S', 25, 15.99),
122. (3, 3, 3, 'M', 3, 20.99),
123. (4, 7, 2, 'S', 2, 18.99),
124. (5, 8, 3, 'M', 3, 20.99),
125. (6, 9, 3, 'M', 3, 20.99),
126. (7, 10, 1, 'M', 1, 15.99),
127. (8, 11, 1, 'M', 1, 15.99),
128. (9, 12, 2, 'M', 2, 18.99),
129. (10, 13, 2, 'M', 2, 18.99),
130. (11, 14, 2, 'M', 3, 18.99),
131. (12, 15, 2, 'S', 1, 18.99),
132. (13, 16, 2, 'S', 1, 18.99),
133. (14, 17, 2, 'L', 1, 18.99),
134. (15, 18, 2, 'S', 4, 18.99),
135. (16, 18, 1, 'S', 3, 15.99),
136. (17, 19, 2, 'S', 1, 18.99),
137. (18, 19, 1, 'M', 3, 15.99);
138.
139. -- -----
140.
141. --
142. -- Table structure for table `payments`
143. --
144.
145. CREATE TABLE `payments` (
146. `id` int(11) NOT NULL,
147. `user_id` int(11) NOT NULL,
148. `order_id` int(11) NOT NULL,
149. `payment_status` enum('pending','completed','failed') DEFAULT 'pending',
150. `transaction_id` varchar(255) DEFAULT NULL,
151. `created_at` timestamp NOT NULL DEFAULT current_timestamp(),
152. `updated_at` timestamp NOT NULL DEFAULT current_timestamp() ON UPDATE
current_timestamp()
153. ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
154.
155. -- -----
156.
157. --
158. -- Table structure for table `products`
159. --
160.
161. CREATE TABLE `products` (
162. `id` int(11) NOT NULL,

```

```

163. `name` varchar(255) NOT NULL,
164. `category` varchar(50) NOT NULL,
165. `description` text NOT NULL,
166. `brand` varchar(100) NOT NULL,
167. `image` longtext CHARACTER SET utf8mb4 COLLATE utf8mb4_bin DEFAULT NULL CHECK
(json_valid(`image`)),
168. `sold_out` tinyint(1) DEFAULT 0,
169. `made_in` varchar(100) DEFAULT NULL,
170. `product_code` varchar(50) NOT NULL,
171. `model_size` varchar(10) DEFAULT NULL,
172. `complete_look` text DEFAULT NULL,
173. `created_at` timestamp NOT NULL DEFAULT current_timestamp(),
174. `updated_at` timestamp NOT NULL DEFAULT current_timestamp() ON UPDATE
current_timestamp()
175. ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
176.
177. --
178. -- Dumping data for table `products`
179. --
180.
181. INSERT INTO `products` (`id`, `name`, `category`, `description`, `brand`, `image`,
`sold_out`, `made_in`, `product_code`, `model_size`, `complete_look`, `created_at`,
`updated_at`) VALUES
182. (1, 'Black Cotton T-Shirt', 't-shirt', 'A premium cotton t-shirt with a classic black
color.', 'Nike',
'["https://blenderartists.org/uploads/default/original/4X/5/7/8/578db100f40fcdac766dc020d
b08c754ad08c444.png", "black_tshirt2.jpg"]', 0, 'USA', 'TSH1001', 'M', '["2", "3"]',
'2025-02-16 12:55:13', '2025-05-19 18:08:22'),
183. (2, 'White Printed T-Shirt', 't-shirt', 'Soft white t-shirt with stylish print.',
'Adidas', '["https://www.trueclassictees.com/cdn/shop/files/4000_CARBON_2_6792bbe7-7dc1-
47b6-8f43-9b7484909d71_800x.jpg", "white_tshirt2.jpg"]', 0, 'India', 'TSH1002', 'L',
'["1", "4"]', '2025-02-16 12:55:13', '2025-02-16 13:03:30'),
184. (3, 'Graphic Oversized T-Shirt', 't-shirt', 'Trendy oversized t-shirt with graphic
design.', 'Zara', '["https://www.trueclassictees.com/cdn/shop/files/4000_NAVY_2_e3036ca0-
faf1-48cc-81da-f3c4b0af82bb_800x.jpg", "graphic_tshirt2.jpg"]', 0, 'Bangladesh',
'TSH1003', 'XL', '["2", "5"]', '2025-02-16 12:55:13', '2025-02-16 13:03:55'),
185. (4, 'V-Neck Slim Fit T-Shirt', 't-shirt', 'Slim fit t-shirt with a comfortable V-neck
design.', 'Puma',
'["https://www.trueclassictees.com/cdn/shop/files/4000_MILITARYGREEN_2_15f28142-93cd-
4854-b5b7-f94be39abec1_800x.jpg", "vneck_tshirt2.jpg"]', 0, 'Vietnam', 'TSH1004', 'M',
'["3", "5"]', '2025-02-16 12:55:13', '2025-02-16 13:04:10'),
186. (5, 'Long Sleeve T-Shirt', 't-shirt', 'Comfortable long sleeve t-shirt for casual
wear.', 'Levi's',
'["https://www.trueclassictees.com/cdn/shop/files/4000_WHITE_2_1a4b3f1d-1093-4b35-bbfd-
bf413d2d12be_800x.jpg", "longsleeve_tshirt2.jpg"]', 0, 'China', 'TSH1005', 'L',
'["1", "3"]', '2025-02-16 12:55:13', '2025-02-16 13:04:42');
187.
188. -----
189.
190. --
191. -- Table structure for table `product_sizes`
192. --
193.
194. CREATE TABLE `product_sizes` (
195. `id` int(11) NOT NULL,
196. `product_id` int(11) NOT NULL,

```

```

197. `size` varchar(10) NOT NULL,
198. `quantity` int(11) NOT NULL,
199. `price` decimal(10,2) NOT NULL,
200. `created_at` timestamp NOT NULL DEFAULT current_timestamp(),
201. `updated_at` timestamp NOT NULL DEFAULT current_timestamp() ON UPDATE
current_timestamp()
202. ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
203.
204. --
205. -- Dumping data for table `product_sizes`
206. --
207.
208. INSERT INTO `product_sizes` (`id`, `product_id`, `size`, `quantity`, `price`,
`created_at`, `updated_at`) VALUES
209. (1, 1, 'S', 27, 15.99, '2025-02-16 12:55:13', '2025-04-08 09:56:15'),
210. (2, 1, 'M', 22, 15.99, '2025-02-16 12:55:13', '2025-05-19 18:24:32'),
211. (3, 1, 'L', 20, 15.99, '2025-02-16 12:55:13', '2025-02-16 12:55:13'),
212. (4, 1, 'XL', 10, 17.99, '2025-02-16 12:55:13', '2025-02-16 12:55:13'),
213. (5, 2, 'S', 13, 18.99, '2025-02-16 12:55:13', '2025-05-19 18:24:32'),
214. (6, 2, 'M', 18, 18.99, '2025-02-16 12:55:13', '2025-02-16 12:55:13'),
215. (7, 2, 'L', 14, 18.99, '2025-02-16 12:55:13', '2025-04-06 13:56:20'),
216. (8, 3, 'M', 22, 20.99, '2025-02-16 12:55:13', '2025-02-16 12:55:13'),
217. (9, 3, 'L', 18, 20.99, '2025-02-16 12:55:13', '2025-02-16 12:55:13'),
218. (10, 3, 'XL', 12, 22.99, '2025-02-16 12:55:13', '2025-02-16 12:55:13'),
219. (11, 4, 'S', 25, 16.99, '2025-02-16 12:55:13', '2025-02-16 12:55:13'),
220. (12, 4, 'M', 20, 16.99, '2025-02-16 12:55:13', '2025-02-16 12:55:13'),
221. (13, 4, 'L', 15, 16.99, '2025-02-16 12:55:13', '2025-02-16 12:55:13'),
222. (14, 5, 'M', 18, 19.99, '2025-02-16 12:55:13', '2025-02-16 12:55:13'),
223. (15, 5, 'L', 12, 19.99, '2025-02-16 12:55:13', '2025-02-16 12:55:13'),
224. (16, 5, 'XL', 8, 21.99, '2025-02-16 12:55:13', '2025-02-16 12:55:13');
225.
226. -- -----
227.
228. --
229. -- Table structure for table `reviews`
230. --
231.
232. CREATE TABLE `reviews` (
233. `id` int(11) NOT NULL,
234. `user_id` int(11) NOT NULL,
235. `product_id` int(11) NOT NULL,
236. `rating` int(11) DEFAULT NULL CHECK (`rating` between 1 and 5),
237. `comment` text DEFAULT NULL,
238. `created_at` timestamp NOT NULL DEFAULT current_timestamp(),
239. `updated_at` timestamp NOT NULL DEFAULT current_timestamp() ON UPDATE
current_timestamp()
240. ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
241.
242. -- -----
243.
244. --
245. -- Table structure for table `shipping_addresses`
246. --
247.
248. CREATE TABLE `shipping_addresses` (
249. `id` int(11) NOT NULL,

```

```

250. `user_id` int(11) NOT NULL,
251. `full_name` varchar(255) NOT NULL,
252. `address` text NOT NULL,
253. `city` varchar(100) NOT NULL,
254. `postal_code` varchar(20) NOT NULL,
255. `country` varchar(100) NOT NULL,
256. `phone` varchar(20) NOT NULL,
257. `created_at` timestamp NOT NULL DEFAULT current_timestamp(),
258. `updated_at` timestamp NOT NULL DEFAULT current_timestamp() ON UPDATE
current_timestamp()
259. ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
260.
261. -- -----
262.
263. --
264. -- Table structure for table `users`
265. --
266.
267. CREATE TABLE `users` (
268. `id` int(11) NOT NULL,
269. `username` varchar(255) NOT NULL,
270. `full_name` varchar(60) NOT NULL,
271. `email` varchar(255) NOT NULL,
272. `password` varchar(255) NOT NULL,
273. `created_at` timestamp NOT NULL DEFAULT current_timestamp(),
274. `updated_at` timestamp NOT NULL DEFAULT current_timestamp() ON UPDATE
current_timestamp()
275. ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
276.
277. --
278. -- Dumping data for table `users`
279. --
280.
281. INSERT INTO `users` (`id`, `username`, `full_name`, `email`, `password`,
`created_at`, `updated_at`) VALUES
282. (1, 'admin', 'Davit Hovhannisyan', 'dav.hovhannisyan.0102@gmail.com',
'$2y$10$RonH2AfoKmyEldqPccsAg.tq0s2laTsQLKwg9ekdQL0Yq9SZX..n0', '2025-03-06 14:07:15',
'2025-04-06 14:00:50');
283.
284. -- -----
285.
286. --
287. -- Table structure for table `wishlist`
288. --
289.
290. CREATE TABLE `wishlist` (
291. `id` int(11) NOT NULL,
292. `user_id` int(11) NOT NULL,
293. `product_id` int(11) NOT NULL,
294. `created_at` timestamp NOT NULL DEFAULT current_timestamp()
295. ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
296.
297. --
298. -- Indexes for dumped tables
299. --
300.

```

```

301. --
302. -- Indexes for table `admins`
303. --
304. ALTER TABLE `admins`
305.     ADD PRIMARY KEY (`id`),
306.     ADD UNIQUE KEY `user_id` (`user_id`);
307.
308. --
309. -- Indexes for table `cart`
310. --
311. ALTER TABLE `cart`
312.     ADD PRIMARY KEY (`id`),
313.     ADD KEY `user_id` (`user_id`),
314.     ADD KEY `product_id` (`product_id`);
315.
316. --
317. -- Indexes for table `orders`
318. --
319. ALTER TABLE `orders`
320.     ADD PRIMARY KEY (`id`),
321.     ADD KEY `user_id` (`user_id`);
322.
323. --
324. -- Indexes for table `order_items`
325. --
326. ALTER TABLE `order_items`
327.     ADD PRIMARY KEY (`id`),
328.     ADD KEY `order_id` (`order_id`),
329.     ADD KEY `product_id` (`product_id`);
330.
331. --
332. -- Indexes for table `payments`
333. --
334. ALTER TABLE `payments`
335.     ADD PRIMARY KEY (`id`),
336.     ADD UNIQUE KEY `transaction_id` (`transaction_id`),
337.     ADD KEY `user_id` (`user_id`),
338.     ADD KEY `order_id` (`order_id`);
339.
340. --
341. -- Indexes for table `products`
342. --
343. ALTER TABLE `products`
344.     ADD PRIMARY KEY (`id`),
345.     ADD UNIQUE KEY `product_code` (`product_code`);
346.
347. --
348. -- Indexes for table `product_sizes`
349. --
350. ALTER TABLE `product_sizes`
351.     ADD PRIMARY KEY (`id`),
352.     ADD KEY `product_id` (`product_id`);
353.
354. --
355. -- Indexes for table `reviews`
356. --

```

```

357. ALTER TABLE `reviews`
358.     ADD PRIMARY KEY (`id`),
359.     ADD KEY `user_id` (`user_id`),
360.     ADD KEY `product_id` (`product_id`);
361.
362. --
363. -- Indexes for table `shipping_addresses`
364. --
365. ALTER TABLE `shipping_addresses`
366.     ADD PRIMARY KEY (`id`),
367.     ADD KEY `user_id` (`user_id`);
368.
369. --
370. -- Indexes for table `users`
371. --
372. ALTER TABLE `users`
373.     ADD PRIMARY KEY (`id`),
374.     ADD UNIQUE KEY `email` (`email`);
375.
376. --
377. -- Indexes for table `wishlist`
378. --
379. ALTER TABLE `wishlist`
380.     ADD PRIMARY KEY (`id`),
381.     ADD KEY `user_id` (`user_id`),
382.     ADD KEY `product_id` (`product_id`);
383.
384. --
385. -- AUTO_INCREMENT for dumped tables
386. --
387.
388. --
389. -- AUTO_INCREMENT for table `admins`
390. --
391. ALTER TABLE `admins`
392.     MODIFY `id` int(11) NOT NULL AUTO_INCREMENT;
393.
394. --
395. -- AUTO_INCREMENT for table `cart`
396. --
397. ALTER TABLE `cart`
398.     MODIFY `id` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=43;
399.
400. --
401. -- AUTO_INCREMENT for table `orders`
402. --
403. ALTER TABLE `orders`
404.     MODIFY `id` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=20;
405.
406. --
407. -- AUTO_INCREMENT for table `order_items`
408. --
409. ALTER TABLE `order_items`
410.     MODIFY `id` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=19;
411.
412. --

```

```

413. -- AUTO_INCREMENT for table `payments`
414. --
415. ALTER TABLE `payments`
416.   MODIFY `id` int(11) NOT NULL AUTO_INCREMENT;
417.
418. --
419. -- AUTO_INCREMENT for table `products`
420. --
421. ALTER TABLE `products`
422.   MODIFY `id` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=6;
423.
424. --
425. -- AUTO_INCREMENT for table `product_sizes`
426. --
427. ALTER TABLE `product_sizes`
428.   MODIFY `id` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=17;
429.
430. --
431. -- AUTO_INCREMENT for table `reviews`
432. --
433. ALTER TABLE `reviews`
434.   MODIFY `id` int(11) NOT NULL AUTO_INCREMENT;
435.
436. --
437. -- AUTO_INCREMENT for table `shipping_addresses`
438. --
439. ALTER TABLE `shipping_addresses`
440.   MODIFY `id` int(11) NOT NULL AUTO_INCREMENT;
441.
442. --
443. -- AUTO_INCREMENT for table `users`
444. --
445. ALTER TABLE `users`
446.   MODIFY `id` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=2;
447.
448. --
449. -- AUTO_INCREMENT for table `wishlist`
450. --
451. ALTER TABLE `wishlist`
452.   MODIFY `id` int(11) NOT NULL AUTO_INCREMENT;
453.
454. --
455. -- Constraints for dumped tables
456. --
457.
458. --
459. -- Constraints for table `admins`
460. --
461. ALTER TABLE `admins`
462.   ADD CONSTRAINT `admins_ibfk_1` FOREIGN KEY (`user_id`) REFERENCES `users` (`id`) ON
  DELETE CASCADE;
463.
464. --
465. -- Constraints for table `cart`
466. --
467. ALTER TABLE `cart`

```

```

468.  ADD CONSTRAINT `cart_ibfk_1` FOREIGN KEY (`user_id`) REFERENCES `users` (`id`) ON
DELETE CASCADE,
469.  ADD CONSTRAINT `cart_ibfk_2` FOREIGN KEY (`product_id`) REFERENCES `products`
(`id`) ON DELETE CASCADE;
470.
471. --
472. -- Constraints for table `orders`
473. --
474. ALTER TABLE `orders`
475.  ADD CONSTRAINT `orders_ibfk_1` FOREIGN KEY (`user_id`) REFERENCES `users` (`id`) ON
DELETE CASCADE;
476.
477. --
478. -- Constraints for table `order_items`
479. --
480. ALTER TABLE `order_items`
481.  ADD CONSTRAINT `order_items_ibfk_1` FOREIGN KEY (`order_id`) REFERENCES `orders`
(`id`) ON DELETE CASCADE,
482.  ADD CONSTRAINT `order_items_ibfk_2` FOREIGN KEY (`product_id`) REFERENCES
`products` (`id`);
483.
484. --
485. -- Constraints for table `payments`
486. --
487. ALTER TABLE `payments`
488.  ADD CONSTRAINT `payments_ibfk_1` FOREIGN KEY (`user_id`) REFERENCES `users` (`id`)
ON DELETE CASCADE,
489.  ADD CONSTRAINT `payments_ibfk_2` FOREIGN KEY (`order_id`) REFERENCES `orders`
(`id`) ON DELETE CASCADE;
490.
491. --
492. -- Constraints for table `product_sizes`
493. --
494. ALTER TABLE `product_sizes`
495.  ADD CONSTRAINT `product_sizes_ibfk_1` FOREIGN KEY (`product_id`) REFERENCES
`products` (`id`) ON DELETE CASCADE;
496.
497. --
498. -- Constraints for table `reviews`
499. --
500. ALTER TABLE `reviews`
501.  ADD CONSTRAINT `reviews_ibfk_1` FOREIGN KEY (`user_id`) REFERENCES `users` (`id`)
ON DELETE CASCADE,
502.  ADD CONSTRAINT `reviews_ibfk_2` FOREIGN KEY (`product_id`) REFERENCES `products`
(`id`) ON DELETE CASCADE;
503.
504. --
505. -- Constraints for table `shipping_addresses`
506. --
507. ALTER TABLE `shipping_addresses`
508.  ADD CONSTRAINT `shipping_addresses_ibfk_1` FOREIGN KEY (`user_id`) REFERENCES
`users` (`id`) ON DELETE CASCADE;
509.
510. --
511. -- Constraints for table `wishlist`
512. --

```

```
513. ALTER TABLE `wishlist`
514.   ADD CONSTRAINT `wishlist_ibfk_1` FOREIGN KEY (`user_id`) REFERENCES `users` (`id`)
ON DELETE CASCADE,
515.   ADD CONSTRAINT `wishlist_ibfk_2` FOREIGN KEY (`product_id`) REFERENCES `products`
(`id`) ON DELETE CASCADE;
516. COMMIT;
517.
518. /*!40101 SET CHARACTER_SET_CLIENT=@OLD_CHARACTER_SET_CLIENT */;
519. /*!40101 SET CHARACTER_SET_RESULTS=@OLD_CHARACTER_SET_RESULTS */;
520. /*!40101 SET COLLATION_CONNECTION=@OLD_COLLATION_CONNECTION */;
521.
```

Եզրակացություն

Իմ վեբ ծրագրավորման կուրսային աշխատանքը հանդիսանում է լիարժեք ֆունկցիոնալությամբ հազեցած միևնույն առցանց խանութի ստեղծման արդյունք, որը մշակվել է արդի վեբ տեխնոլոգիաների հիման վրա՝ ներառելով Frontend, Backend, տվյալների բազայի կառավարում (SQL) և որոնողական համակարգի օպտիմալացում (SEO):

Frontend (Ձևավորում և օգտագործողի ինտերֆեյս)

Ձեռնարկված նախագծման ընթացքում Frontend-ի մասը ստեղծվել է HTML, CSS (Flexbox և Grid Layout) և JavaScript լեզուներով: Օգտագործվել են պատասխանատու (responsive) դիզայնի սկզբունքներ՝ ապահովելու համար վեբ կայքի ճիշտ աշխատունակությունը ինչպես դեքտոպ, այնպես էլ բջջային սարքերում: Հստակ կառուցված նավիգացիան, մաքուր ինտերֆեյսը և օգտատերերին հասկանալի ձևաչափով ներկայացված ապրանքները նպաստում են կայքի օգտագործման հեշտությանը:

Backend (Սերվերային մաս)

Backend-ը իրականացվել է PHP լեզվով՝ ապահովելով կայքի գործառնությունների սերվերային մշակում: Արժեք առանձնացնել հետևյալ գործառնությունները.

- Օգտատերերի գրանցում և մուտք
- Ապրանքների ցուցադրում և ֆիլտրում ըստ կատեգորիաների
- Չամբյուղի կառավարում (ավելացում, ջնջում, քանակի փոփոխություն)
- Վճարման գործընթացի նախապատրաստում

Սերվերային մասում ստեղծվել են անվտանգ և օպտիմալացված սցենարներ, որոնք թույլ են տալիս կայքին աշխատել հուսալի և արդյունավետ կերպով:

Տվյալների բազա (SQL)

Տվյալների պահպանման համար կիրառվել է MySQL/PostgreSQL տվյալների բազայի կառավարիչը: Մշակվել են հարաբերական աղյուսակներ, որոնք ապահովում են տվյալների նորմալիզացիան, բազմապիսի հարցումների օպտիմալ կատարումը և տվյալների ինտեգրիտետը: Աղյուսակներն ընդգրկում են օգտատերեր, ապրանքներ, չափսեր, Չամբյուղ, պատվերներ և վճարումներ: Ապահովված է համապատասխան կապերի միջոցով (foreign key) տվյալների բազայի ճշգրտությունը և կապվածությունը:

SEO (Որոնողական համակարգերի օպտիմալացում)

Կայքի տեսանելիությունը որոնողական համակարգերում բարձրացնելու նպատակով կիրառվել են մի շարք SEO լավագույն փորձեր՝

- Օգտագործվել են semantic HTML տարրեր (<header>, <main>, <section>, <article>, <footer>)՝ բովանդակության կառուցվածքը ճշգրիտ ներկայացնելու համար:
- Ավելացվել են meta թեգեր (title, description, keywords)՝ յուրաքանչյուր էջի համար՝ համապատասխան բովանդակությամբ:
- Արժանահավատ URL կառուցվածք՝ SEO-ին բարենպաստ լինող հղումներով:
- Ապահովվել է կայքի արագ բեռնման ժամանակ՝ CSS և JS ֆայլերի օպտիմալացման միջոցով:
- Ավելացվել է ALT հատկություն պատկերների համար՝ ապահովելով ինչպես մատչելիություն, այնպես էլ որոնողական ճանաչելիություն:

Ընդհանուր գնահատական

Կուրսային աշխատանքը հաջողությամբ ինտեգրում է կայքաշինության հիմնական բաղադրիչները՝ ապահովելով տեխնիկական, դիզայնային և գործառնության բարձր մակարդակ: Ստացված արդյունքը վկայում է ստացած տեսական գիտելիքների և գործնական հմտությունների արդյունավետ կիրառման մասին: Այս նախագիծը կարող է ծառայել որպես հիմք հետագա ավելի բարդ վեբ լուծումների մշակման և կատարելագործման համար:

Գրականություն

Frontend (HTML, CSS, JavaScript)

1. Duckett, Jon. *HTML and CSS: Design and Build Websites*. Wiley, 2011.
 2. Duckett, Jon. *JavaScript and jQuery: Interactive Front-End Web Development*. Wiley, 2014.
 3. Keith, Jeremy. *HTML5 for Web Designers*. A Book Apart, 2010.
 4. Marcotte, Ethan. *Responsive Web Design*. A Book Apart, 2011.
 5. Mozilla Developer Network (MDN) Documentation:
 - HTML: <https://developer.mozilla.org/en-US/docs/Web/HTML>
 - CSS: <https://developer.mozilla.org/en-US/docs/Web/CSS>
 - JavaScript: <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
-

Backend (PHP)

1. Ullman, Larry. *PHP and MySQL for Dynamic Web Sites: Visual QuickPro Guide*. Peachpit Press, 2017. [amazon.com](https://www.amazon.com/dp/B000APR004)
 2. Lerdorf, Rasmus, Tatroe, Kevin, and MacIntyre, Peter. *Programming PHP*. O'Reilly Media, 2013. [amazon.com+1books.google.com+1](https://www.amazon.com/dp/B000APR004)
 3. Sklar, David. *Learning PHP 7: A Pain-Free Introduction to Building Interactive Web Sites*. O'Reilly Media, 2016.
 4. PHP Official Documentation: <https://www.php.net/docs.php>
-

Տվյալների բազա / SQL (MySQL / PostgreSQL)

1. Oppel, Andy. *Databases: A Beginner's Guide*. McGraw-Hill Education, 2015.

2. Beaulieu, Alan. *Learning SQL*. O'Reilly Media, 2020.
 3. Van der Lans, Rick F. *Introduction to SQL*. Addison-Wesley, 2006.
 4. MySQL Documentation: <https://dev.mysql.com/doc/>
 5. PostgreSQL Documentation: <https://www.postgresql.org/docs/>
-

SEO (Search Engine Optimization)

1. Fishkin, Rand, and Høgenhaven, Thomas. *The Art of SEO: Mastering Search Engine Optimization*. O'Reilly Media, 2015.
 2. Ledford, Jerri L. *SEO: Search Engine Optimization Bible*. Wiley, 2009.
 3. Google Search Central (Previously Webmaster Guidelines):
<https://developers.google.com/search/docs>
 4. Moz SEO Guide: <https://moz.com/beginners-guide-to-seo>
-

Ընդհանուր / Լրացուցիչ աղբյուրներ

1. W3Schools Tutorials: <https://www.w3schools.com>
2. freeCodeCamp: <https://www.freecodecamp.org/>
3. Traversy Media (YouTube): Լավագույն ուսուցողական վիդեոներ վեբ ծրագրավորման համար:
4. Stack Overflow: Օգտակար է խնդիրների լուծման և հարցուպատասխանի նպատակով:
<https://stackoverflow.com>
5. GeeksforGeeks – Web Development Section: <https://www.geeksforgeeks.org/web-development/>